

UNIVERSIDAD DEL VALLE



TRABAJO PROFESIONAL

**Implementación y Evaluación Experimental de un
Modelo de Lenguaje Basado en Arquitecturas
MatMul-Free y Cuantización Ternaria**

Juan Camilo Valencia Rivas
juan.valencia.rivas@correounivalle.edu.co

Director:
Oscar Fernando Bedoya Leiva, PhD.

Co-director:
Raúl Ernesto Gutiérrez de Piñerez, PhD.

Universidad del Valle
Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación
Programa Académico de Ingeniería de Sistemas
Cali, 2026

Resumen

Los Grandes Modelos de Lenguaje (LLMs) basados en arquitecturas Transformer convencionales enfrentan limitaciones estructurales significativas que comprometen su escalabilidad y versatilidad. Estas limitaciones se manifiestan principalmente en dos dimensiones: el costo computacional prohibitivo asociado a las operaciones de multiplicación de matrices (MatMul) y la dependencia crítica de procesos de tokenización que introducen sesgos lingüísticos y obstaculizan el procesamiento eficiente de secuencias no convencionales. Este trabajo de grado aborda estas problemáticas mediante el diseño, implementación y evaluación de una arquitectura innovadora que integra sinérgicamente dos paradigmas emergentes: modelos MatMul-Free y mecanismos de Dynamic Chunking. La arquitectura propuesta, denominada HNetBit, elimina completamente las operaciones MatMul mediante la implementación de pesos ternarios (-1, 0, +1) y operaciones elemento a elemento, reduciendo drásticamente los requerimientos computacionales y energéticos. Paralelamente, incorpora un sistema de chunking dinámico multi-nivel que permite el aprendizaje automático de estrategias de segmentación adaptativa, eliminando la necesidad de tokenización predefinida y facilitando el procesamiento directo de secuencias de bytes crudos. La evaluación comprehensiva del modelo se realizó mediante un benchmark compuesto por texto natural en español, utilizando métricas estandarizadas de precisión y eficiencia computacional. Los resultados obtenidos en trabajos seminales sugirieron que esta integración sinérgica permitió alcanzar un rendimiento competitivo con arquitecturas Transformer tradicionales, ofreciendo a su vez ventajas significativas en eficiencia computacional y capacidad de generalización a través de dominios estructuralmente diversos, estableciendo así las bases para LLMs más sostenibles y versátiles.

Palabras clave: Grandes Modelos de Lenguaje, arquitectura MatMul-Free, cuantización ternaria, chunking dinámico, eficiencia computacional.

Índice general

1	Introducción	1
1.1	Planteamiento del Problema	2
1.1.1	Descripción del Problema	2
1.1.2	Formulación del Problema	2
1.2	Justificación del Problema	3
1.2.1	Justificación Académica	3
1.2.2	Justificación Social	3
1.2.3	Justificación Económica	4
1.3	Objetivos	4
1.3.1	Objetivo General	4
1.3.2	Objetivos Específicos	4
1.4	Alcance	4
2	Marco Referencial	6
2.1	Glosario	6
2.2	Marco Teórico	7
2.2.1	Fundamentos de Arquitecturas Eficientes de LLMs	7
2.2.2	Teoría de Procesamiento sin Tokenización	8
2.2.3	Fundamentos Matemáticos de Leyes de Escalabilidad	9
2.2.4	Modelos de Lenguaje Pequeños (SLMs)	9
3	Estado del Arte	11
3.1	Modelado de Lenguaje Escalable Libre de Multiplicación de Matrices	11
3.2	Chunking Dinámico para Modelado Jerárquico de Secuencias de Extremo a Extremo	14
3.3	La Era de los LLMs de 1 bit: Todos los Modelos de Lenguaje de Gran Tamaño son de 1.58 Bits	16
3.4	SpaceByte: Hacia la Eliminación de la Tokenización en el Modelado de Lenguaje de Gran Tamaño	18
3.5	MEGABYTE: Prediciendo Secuencias de Millones de Bytes con Transformers Multiescala	21
4	Arquitectura del modelo propuesto HNetBit	24
4.1	Visión general de la arquitectura	24
4.2	Cuantización ternaria y capa BitLinear	25
4.2.1	Fundamentos de la cuantización ternaria	25

4.2.2	Cuantización de pesos	25
4.2.3	Cuantización de activaciones	27
4.2.4	Capa BitLinear	27
4.2.5	Variante fusionada: FusedBitLinear	28
4.2.6	Normalización RMSNorm	28
4.3	Recurrencia HGRN	28
4.3.1	Motivación	29
4.3.2	Formulación matemática	29
4.3.3	Mecanismo multi-cabeza	30
4.3.4	Función de activación SwiGLU	30
4.3.5	Normalización de compuerta fusionada	30
4.3.6	Convolución corta (<i>Short Convolution</i>)	30
4.3.7	Kernels de recurrencia	31
4.4	Chunking dinámico multi-etapa	32
4.4.1	Fundamento teórico	32
4.4.2	Módulo de enrutamiento (<i>RoutingModuleBit</i>)	32
4.4.3	Capa de chunking (<i>ChunkLayer</i>)	33
4.4.4	Capa de reconstrucción (<i>DeChunkLayer</i>)	34
4.4.5	Pérdida de balanceo de carga (<i>Load-Balancing Loss</i>)	34
4.5	Estructura de bloques	35
4.5.1	Bloque HGRNBitAttention	35
4.5.2	Bloque HGRNBitMLP	35
4.5.3	Bloque HGRNBitBlock	36
4.6	Arquitectura jerárquica recursiva	36
4.6.1	Principio de recursividad	36
4.6.2	Etapas no interna	36
4.6.3	Etapas más interna	37
4.6.4	Pila HGRNBitStack	37
4.7	Variante con atención deslizante: Hybrid-Attn	38
4.7.1	Motivación	38
4.7.2	Atención causal con ventana deslizante	38
4.7.3	Embeddings rotatorios (RoPE)	38
4.7.4	Patrón de intercalado	39
4.8	Caché de inferencia	39
4.8.1	Estructura de la caché	39
4.8.2	Paso de generación (<i>step mode</i>)	40
4.8.3	Reordenamiento para búsqueda de haz	40
4.9	Pipeline completo: de bytes a logits	40
4.9.1	Codificación byte-level	40
4.9.2	Forward completo	41
4.9.3	Cabecera de lenguaje (<i>LM head</i>)	41
4.9.4	Cálculo de pérdida	41
4.9.5	Componentes por régimen de operación	41

5 Implementación y optimizaciones 44

5.1	Organización del código y arquitectura de software	44
5.2	Infraestructura de entrenamiento	45
5.2.1	Bucle principal de entrenamiento	45
5.2.2	Precisión mixta	46
5.2.3	Pérdida de balanceo de carga	46
5.2.4	Tracking de compresión por etapa	46
5.3	Sistema de checkpointing y resume	46
5.3.1	Contenido del checkpoint	46
5.3.2	Jerarquía de checkpoints	47
5.3.3	Mecanismo de resume	47
5.4	Sistema de logging	48
5.4.1	Training steps log	48
5.4.2	Validation log	48
5.4.3	Training stats JSON	48
5.5	Kernels Triton y gestión de memoria GPU	49
5.5.1	Detección de Triton y fallbacks	49
5.6	Optimizaciones de memoria para entrenamiento	49
5.6.1	Ausencia de gradient checkpointing	50
5.6.2	Estrategias de mitigación	50
5.6.3	Consumo de memoria del optimizador	51
5.7	Sistema de caché para generación	51
5.7.1	HGRNBlockCache	51
5.7.2	Manejo de continuidad de batch en step mode	51
5.7.3	Soporte para búsqueda de haz	52
5.8	Exportación para despliegue	52
5.8.1	Proceso de exportación	52
5.8.2	Ratios de compresión	53
5.9	Integración con Hugging Face	53
5.9.1	Manejo de caché en generación	53
5.9.2	Manejo de tamaño de lote variable	54
5.9.3	Compatibilidad con beam search	54
6	Modelos y método experimental	55
6.1	Conjunto de datos	55
6.1.1	Procesamiento según el esquema de tokenización	55
6.1.2	Construcción de los datasets	56
6.2	Arquitecturas implementadas	56
6.2.1	Transformer (línea base)	56
6.2.2	MatMul-Free plano (línea base recurrente)	57
6.2.3	Modelo híbrido propuesto (HNetBit)	57
6.2.4	Configuraciones y conteo de parámetros	57
6.3	Metodología de entrenamiento	59
6.4	Métricas de evaluación	60
6.4.1	Calidad: bits por byte (BPB)	60
6.4.2	Eficiencia: tamaño en despliegue	60

6.4.3	Eficiencia: rendimiento de inferencia	60
7	Resultados	62
7.1	Calidad de los modelos	62
7.2	Eficiencia en parámetros y despliegue	64
7.3	Rendimiento de inferencia	65
7.4	Costo de entrenamiento	66
7.5	Análisis del chunking dinámico	67
7.6	Síntesis	69
8	Discusión	70
8.1	Viabilidad de eliminar MatMul y tokenización	70
8.2	Hallazgos inesperados	71
8.2.1	La curva no monótona del híbrido de 150M	71
8.2.2	La compresión aprendida es estable y lingüísticamente plausible . . .	71
8.2.3	El cuello de botella de los modelos ternarios es la velocidad, no la memoria	71
8.3	Validez interna	72
8.3.1	Igualdad del presupuesto de bytes	72
8.3.2	Desigualdad de parámetros	72
8.3.3	Asimetría en gradient checkpointing	72
8.3.4	Comparabilidad de la métrica BPB	72
8.4	Validez externa y limitaciones	73
8.4.1	Una sola ejecución por configuración	73
8.4.2	Comparación no emparejada en FLOPs	73
8.5	Implicaciones	73
9	Conclusiones	74
9.1	Respuesta a la pregunta de investigación	74
9.2	Evaluación de la hipótesis	75
9.3	Contribuciones	75
9.4	Síntesis final	76
10	Trabajo Futuro	77
10.1	Evaluación empírica más amplia	77
10.2	Mejoras arquitectónicas	78
10.3	Mejoras de implementación y eficiencia	78
10.4	Optimizaciones de velocidad de inferencia	79
	Referencias	81

Índice de figuras

Figura 3.1.	Comparación de ley de escalamiento entre modelos MatMul-Free LM y Transformer++.	12
Figura 3.2.	Precisión zero-shot de modelos MatMulfree LM y Transformer++.	13
Figura 3.3.	Resumen arquitectónico de H-Net.	15
Figura 3.4.	Bits por byte (BPB) de validación durante el entrenamiento.	16
Figura 3.5.	Precisión zero-shot de BitNet b1.58 y LLaMA LLM.	17
Figura 3.6.	Perplejidad y costo de inferencia de BitNet b1.58 y LLaMA LLM.	18
Figura 3.7.	Resumen arquitectónico de SpaceByte.	19
Figura 3.8.	Frontera de Pareto de la entropía cruzada (bits por byte) frente a los FLOPs por byte durante la inferencia para diferentes arquitecturas en el dataset Github.	20
Figura 3.9.	Resumen arquitectónico de MegaByte con $P = 4$.	22
Figura 3.10.	Rendimiento en bit por byte según dataset.	22
Figura 4.1.	Arquitectura del modelo HNetBit.	26
Figura 7.1.	BPB de validación durante el entrenamiento.	63
Figura 7.2.	Proyección de escala del BPB hasta 800M de parámetros.	64
Figura 7.3.	Evolución de la compresión del chunking dinámico.	68

Índice de tablas

Tabla 4.1.	Kernels de recurrencia HGRN	32
Tabla 4.2.	Componentes arquitectónicos por régimen de operación	42
Tabla 6.1.	Configuraciones de las arquitecturas evaluadas	58
Tabla 6.2.	Conteo de parámetros (millones)	58
Tabla 6.3.	Hiperparámetros de entrenamiento comunes	59
Tabla 7.1.	BPB de validación final (200 lotes; menor es mejor)	62
Tabla 7.2.	Tamaño de despliegue y compresión (150M)	64
Tabla 7.3.	Métricas de inferencia (A100 80 GB)	65
Tabla 7.4.	Costo de entrenamiento (presupuesto de 25B bytes)	66
Tabla 7.5.	FLOPs estimados por muestra (bloques HGRN; excluye enrutamiento, <i>embeddings</i> y pérdida)	66
Tabla 7.6.	Síntesis de resultados	69

Capítulo 1

Introducción

Los Grandes Modelos de Lenguaje (LLMs) han demostrado capacidades transformadoras en el procesamiento de secuencias multimodales, convirtiéndose en componentes fundamentales para aplicaciones que van desde la generación de texto hasta el análisis de código y secuencias biológicas. No obstante, su arquitectura convencional basada en Transformers presenta dos limitaciones fundamentales que obstaculizan su escalabilidad y versatilidad: un alto costo computacional, impulsado principalmente por las operaciones de multiplicación de matrices (MatMul), y una dependencia crítica de la tokenización, que introduce sesgos lingüísticos y obstaculiza el procesamiento eficiente de *inputs* no convencionales como lenguajes pictográficos, código de programación o secuencias de ADN.

En respuesta al desafío computacional, han emergido arquitecturas innovadoras como el MatMul-Free LM (Zhu et al., 2024), que elimina completamente las operaciones MatMul reemplazándolas con pesos ternarios y operaciones elemento a elemento, logrando una drástica reducción en el consumo energético y de memoria. Adicionalmente, los recientes avances en modelos de baja precisión, como BitNet b1.58 (Ma et al., 2024), demuestran que es posible entrenar modelos de lenguaje donde cada parámetro es ternario ($-1, 0, +1$), equivalente aproximadamente a 1.58 bits por peso. Este enfoque no solo reduce drásticamente los requerimientos de memoria y energía, sino que mantiene un rendimiento comparable al de modelos en precisión completa, estableciendo un nuevo paradigma de computación basado en operaciones enteras.

Paralelamente, para abordar las limitaciones impuestas por la tokenización, modelos como H-Net (Hwang et al., 2025) incorporan un mecanismo de *Dynamic Chunking*, permitiendo un modelado jerárquico de extremo a extremo que aprende a segmentar secuencias de bytes crudos de forma adaptativa. Esto elimina la necesidad de una tokenización predefinida, facilitando el procesamiento de dominios con estructuras no lingüísticas donde los tokenizadores tradicionales fallan en capturar relaciones significativas.

Este trabajo diseñó e implementó un marco experimental modular que facilita el desarrollo, entrenamiento, inferencia y evaluación de SLMs bajo diversas estrategias de eficiencia, proveyendo un enfoque más robusto frente a las arquitecturas convencionales existentes.

1.1. Planteamiento del Problema

1.1.1. Descripción del Problema

La arquitectura Transformer, fundamento de los modelos de lenguaje actuales, presenta dos limitaciones estructurales que comprometen su eficiencia y capacidad de generalización: las operaciones de multiplicación de matrices (MatMul) y los procesos de tokenización.

Las operaciones MatMul constituyen el principal cuello de botella computacional, demandando un consumo excesivo de energía y memoria que se acentúa en modelos que escalan a miles de millones de parámetros. Esta limitación no solo restringe la implementación en entornos con recursos computacionales limitados, sino que también afecta la sostenibilidad y accesibilidad de los LLMs.

Paralelamente, la tokenización introduce una capa de preprocesamiento que fragmenta las secuencias de entrada en unidades discretas basadas en estadísticas de corpus lingüísticos. Este enfoque, aunque efectivo para lenguaje natural, resulta inadecuado para dominios con estructuras no lingüísticas como código fuente, secuencias de ADN o sistemas simbólicos, donde la división de unidades semánticamente relevantes genera pérdida de información estructural y rendimiento subóptimo.

La conjunción de estas dependencias, MatMul y tokenización, limita tanto la eficiencia computacional como la versatilidad de los LLMs actuales. Superar este desafío dual requiere replantear los fundamentos arquitectónicos, explorando alternativas que eliminen las multiplicaciones de matrices y prescindan de la tokenización tradicional, sin comprometer el rendimiento ni la escalabilidad del modelo.

1.1.2. Formulación del Problema

El problema central que se abordó en este trabajo es la dependencia estructural de los LLMs en dos mecanismos fundamentales que limitan su eficiencia y versatilidad: las operaciones de multiplicación de matrices (MatMul) y los procesos de tokenización. Estas limitaciones no solo imponen barreras computacionales y energéticas, sino que restringen la capacidad de los modelos para aprender representaciones verdaderamente de extremo a extremo sobre diversos tipos de secuencias.

Para superar estos obstáculos, se desarrolló una arquitectura alternativa de LLM que integra enfoques innovadores en modelado eficiente y procesamiento directo de datos brutos. En particular, este trabajo combinó los principios de los modelos MatMul-Free, que reemplazan las multiplicaciones de matrices con operaciones elementales y pesos ternarios cuantizados a 1.58 bits, con el mecanismo de Dynamic Chunking, que permite el aprendizaje automático de estrategias de segmentación sin requerir tokenización previa. La integración sinérgica de estos enfoques buscó no solo reducir drásticamente el costo computacional, sino también ampliar la capacidad del modelo para procesar secuencias con estructuras no lingüísticas o de naturaleza diversa.

La pregunta de investigación que guía este trabajo es: ¿cómo diseñar e implementar una arquitectura de LLM que, mediante la integración de mecanismos MatMul-Free y Dynamic

Chunking, logre un rendimiento competitivo en tareas de modelado de lenguaje con un costo computacional sustancialmente reducido y una capacidad mejorada para procesar secuencias no tokenizables?

La hipótesis central sostiene que la combinación de una arquitectura libre de multiplicaciones de matrices con cuantización a 1.58 bits y un mecanismo de segmentación dinámica permitirá construir modelos más eficientes, escalables y versátiles. Esta aproximación podría sentar las bases para una nueva generación de LLMs capaces de aprender directamente de datos crudos, sin depender de procesos de tokenización manuales, ofreciendo además un desempeño comparable o superior al de las arquitecturas Transformer tradicionales.

1.2. Justificación del Problema

1.2.1. Justificación Académica

Este trabajo representa una contribución fundamental al campo emergente de los modelos de lenguaje eficientes al abordar sistemáticamente dos limitaciones estructurales de los Transformers convencionales. La integración sinérgica de arquitecturas MatMul-Free con mecanismos de Dynamic Chunking establece un nuevo paradigma para el desarrollo de LLMs sostenibles y verdaderamente end-to-end. Académicamente, este proyecto avanza en la comprensión teórica de las leyes de escalabilidad en arquitecturas libres de MatMul, extendiendo el trabajo fundacional existente al incorporar mecanismos de segmentación adaptativa. Explora formalmente los límites del modelado sin tokenización mediante el análisis comparativo de estrategias de chunking dinámico frente al estático, y establece comparaciones rigurosas para evaluar el rendimiento de modelos eficientes en dominios no convencionales como código fuente, secuencias biológicas y notación matemática, llenando un vacío significativo en las tecnologías existentes.

1.2.2. Justificación Social

La democratización del acceso a tecnologías de IA avanzadas requiere eliminar las barreras computacionales y energéticas que actualmente concentran estas capacidades en organizaciones con recursos significativos. Al desarrollar arquitecturas sustancialmente más eficientes, este trabajo promueve activamente la accesibilidad a LLMs potentes para instituciones educativas con presupuestos limitados, investigadores en países en desarrollo y desarrolladores de aplicaciones en dispositivos de bajo consumo. La capacidad inherente de procesar secuencias no lingüísticas como ADN, lenguajes simbólicos tradicionales o notaciones científicas especializadas amplía considerablemente el impacto social hacia dominios críticos como la bioinformática, preservación digital de contenidos en riesgo, análisis de código legacy en sistemas heredados, y educación STEM en contextos de recursos limitados. Este enfoque se alinea con los objetivos de desarrollo sostenible al reducir la huella de carbono de las tecnologías de IA mientras expande sus aplicaciones benéficas para la sociedad en su conjunto.

1.2.3. Justificación Económica

Los costos exponenciales asociados al entrenamiento e inferencia de LLMs tradicionales representan una barrera económica insorteable para muchas organizaciones. Este proyecto aborda este desafío de manera integral al lograr una reducción en el impacto computacional de los LLMs mediante la sustitución completa de operaciones MatMul por alternativas basadas en operaciones elemento a elemento y acumulaciones ternarias, combinada con una disminución de $3-4\times$ que permite el despliegue de modelos más grandes en hardware convencional. Adicionalmente, elimina los costos operativos asociados al desarrollo, mantenimiento y actualización de tokenizadores especializados para diferentes dominios. Para empresas y organizaciones que dependen de procesamiento de lenguaje a escala industrial, estos avances pueden traducirse en ahorros operativos del orden de millones de dólares anuales en infraestructura computacional, mientras se mantienen las capacidades funcionales. La eficiencia mejorada resultante permite nuevos modelos de negocio basados en inferencia en dispositivos de bajo consumo y aplicaciones en tiempo real previamente inviables, creando oportunidades económicas en sectores donde los costos computacionales actualmente limitan la innovación.

1.3. Objetivos

1.3.1. Objetivo General

Implementar y evaluar experimentalmente una arquitectura base de SLM, basada en enfoques MatMul-Free y cuantización ternaria, con el fin de analizar su viabilidad computacional y su comportamiento en tareas básicas de modelado de lenguaje sobre un conjunto acotado de datos textuales.

1.3.2. Objetivos Específicos

1. Implementar una versión simplificada de un SLM eficiente, basada en una arquitectura MatMul-Free con pesos ternarios.
2. Realizar un conjunto reducido de experimentos de entrenamiento e inferencia sobre un dominio específico de texto natural, empleando datasets públicos de tamaño controlado.
3. Evaluar el modelo implementado mediante métricas básicas de rendimiento y métricas de eficiencia computacional.

1.4. Alcance

Este trabajo de grado se centró en el diseño, implementación y evaluación de una arquitectura eficiente para SLMs que integró de manera sinérgica los principios de modelos MatMul-Free, cuantización a 1.58 bits y el mecanismo de Dynamic Chunking. El alcance abarcó la implementación de una arquitectura base libre de multiplicaciones de matrices utilizando pesos ternarios y operaciones elemento a elemento, así como la integración de un sistema

de segmentación adaptativa que permite el procesamiento directo de secuencias de bytes sin dependencia de tokenizadores predefinidos.

El benchmark de evaluación incluyó como dominio representativo el texto natural (español), pero no se extendió a otros dominios como audio, imágenes o video.

Metodológicamente, el proyecto se restringió a técnicas de modelado *in silico*, utilizando datasets públicos y herramientas de código abierto. Si bien se evaluó la eficiencia computacional, no se incluyó el desarrollo de hardware especializado ni la implementación en dispositivos de bajo consumo específicos, aunque se analizó su potencial viabilidad. El entrenamiento y la evaluación se realizaron utilizando infraestructura computacional accesible, sin contemplar el escalado a modelos de centenares de miles de millones de parámetros.

Capítulo 2

Marco Referencial

2.1. Glosario

- **Cuantización 1.58 bits:** esquema de representación paramétrica que utiliza valores ternarios (-1, 0, +1), logrando una entropía de 1.58 bits por peso mediante la fórmula $\log_2(3) \approx 1.58$, permitiendo sustituir multiplicaciones por operaciones de conteo (Ma et al., 2024).
- **Dynamic Chunking:** mecanismo de segmentación adaptativa que permite aprender estrategias de agrupamiento de secuencias basadas en similitud semántica y contexto, sin dependencia de reglas predefinidas o estadísticas de corpus específicos (Hwang et al., 2025).
- **End-to-end learning:** paradigma de entrenamiento donde los modelos aprenden representaciones directamente de datos crudos (bytes, caracteres) sin capas de preprocesamiento no diferenciables, permitiendo optimización conjunta de todas las capacidades del modelo (Yu et al., 2023).
- **Leyes de escalabilidad:** relaciones matemáticas que describen cómo varía el rendimiento del modelo (loss, accuracy) en función del aumento de recursos computacionales, parámetros del modelo, y tamaño del dataset de entrenamiento (Hoffmann et al., 2022; Kaplan et al., 2020).
- **LLMs (Large Language Models):** grandes modelos de lenguaje típicamente basados en la arquitectura Transformer, con capacidad de comprensión contextual y generación coherente de texto, entrenados con corpus masivos de datos textuales (Vaswani et al., 2017).
- **MatMul-Free:** arquitectura de modelo que elimina completamente las operaciones de multiplicación de matrices, reemplazándolas con operaciones elemento a elemento y acumulaciones ternarias, reduciendo complejidad computacional en operaciones dominantes (Zhu et al., 2024).
- **Operaciones elemento a elemento:** operaciones matemáticas aplicadas indepen-

dientemente a cada elemento de un tensor, como suma, multiplicación Hadamard o funciones de activación, con complejidad computacional lineal $O(n)$ frente a $O(n^3)$ de MatMul (Zhu et al., 2024).

- **SLMs (Small Language Models):** modelos de lenguaje de escala reducida (generalmente <10B parámetros) que ofrecen un balance óptimo entre rendimiento y eficiencia computacional, siendo más accesibles para entornos con recursos limitados y permitiendo fine-tuning especializado en dominios específicos. Incluyen arquitecturas como Phi-2 (Microsoft Research, 2023), DistilBERT (Sanh et al., 2019), Gemma-2B (Google DeepMind, 2024), TinyLlama (P. Zhang et al., 2024) y RWKV (Peng et al., 2023).
- **Tokenización:** proceso de división de texto en unidades discretas (tokens) mediante algoritmos como BPE (HuggingFace, 2024a), WordPiece (HuggingFace, 2024b) o SentencePiece (Google, 2024), introduciendo una capa de preprocesamiento no diferenciable que limita el aprendizaje end-to-end.

2.2. Marco Teórico

El desarrollo de este trabajo se fundamenta en tres pilares teóricos interconectados que proporcionan las bases formales para la arquitectura propuesta: los fundamentos de arquitecturas eficientes para LLMs, la teoría de procesamiento sin tokenización y los fundamentos de leyes de escalabilidad. A continuación se describe cada uno de ellos.

2.2.1. Fundamentos de Arquitecturas Eficientes de LLMs

Las arquitecturas eficientes de LLMs representan un campo de investigación dedicado a rediseñar los fundamentos computacionales de los modelos de lenguaje para superar las limitaciones de escalabilidad y consumo energético de la arquitectura Transformer convencional. La evolución teórica en este dominio ha transitado desde optimizaciones incrementales hasta reformulaciones radicales de las operaciones matemáticas subyacentes. El punto de partida lo establece la arquitectura Transformer original (Vaswani et al., 2017), cuya dependencia de mecanismos de autoatención con complejidad cuadrática $O(n^2 \cdot d)$ y operaciones intensivas de multiplicación de matrices constituye el principal cuello de botella para el escalado.

La teoría de las arquitecturas eficientes postula que es posible desacoplar la capacidad expresiva del modelo de su costo computacional mediante la reformulación de las operaciones fundamentales. Emergen así dos líneas teóricas principales: por un lado, los modelos que reemplazan operaciones MatMul por alternativas computacionalmente más eficientes, y por otro, los esquemas de cuantización extrema que reducen la precisión numérica sin comprometer el rendimiento. En la primera categoría, el MatMul-Free LM (Zhu et al., 2024) establece un paradigma basado en el marco Metaformer, que descompone la arquitectura en mezcla de tokens y mezcla de canales. La mezcla de tokens se implementa mediante una Unidad Recurrente Lineal con Puertas sin MatMul (MLGRU) que utiliza exclusivamente productos Hadamard y acumulaciones, mientras que la mezcla de canales emplea una Unidad Lineal con Puertas (GLU) construida con capas BitLinear.

Paralelamente, la teoría de cuantización ternaria desarrollada en BitNet b1.58 (Ma et al., 2024) demuestra que es posible representar parámetros con valores en $\{-1, 0, +1\}$ manteniendo capacidad expresiva equivalente a precisiones superiores. La función de cuantización `absmean`, que escala la matriz de pesos por su valor absoluto promedio antes del redondeo ternario, preserva la distribución estadística de las activaciones mientras transforma multiplicaciones en operaciones de conteo. Esta aproximación teórica se sustenta en principios de teoría de la información, donde la entropía de $\log_2(3) \approx 1.58$ bits por parámetro representa un óptimo para representaciones discretas de tres estados. La integración de técnicas de normalización como RMSNorm y la eliminación sistemática de sesgos constituyen elementos teóricos esenciales para la estabilidad del entrenamiento en regímenes de ultra baja precisión.

La confluencia de estos avances teóricos sugiere un cambio de paradigma hacia modelos basados en operaciones enteras y acumulaciones, donde la eficiencia computacional deja de ser una optimización secundaria para convertirse en un principio de diseño fundamental.

2.2.2. Teoría de Procesamiento sin Tokenización

El procesamiento sin tokenización constituye un paradigma teórico alternativo en el modelado de lenguaje que cuestiona la necesidad de métodos de segmentación predefinidos, argumentando que la representación óptima de secuencias debe emerger de patrones estadísticos inherentes a los datos más que de estructuras lingüísticas externas. La teoría se fundamenta en la crítica a la tokenización convencional (BPE, WordPiece, SentencePiece) como capa de preprocesamiento no diferenciable que introduce sesgos lingüísticos irreductibles y limita la aplicabilidad a dominios no textuales. Desde una perspectiva teórica, la tokenización viola el principio de aprendizaje extremo a extremo al introducir una discontinuidad no diferenciable en el pipeline de procesamiento.

La teoría del procesamiento sin tokenización postula que los modelos deben aprender representaciones jerárquicas directamente de secuencias de bytes o caracteres, capturando automáticamente los distintos niveles de abstracción semántica. SpaceByte (Slagle, 2024) formaliza este enfoque mediante bloques globales de Transformer activados dinámicamente después de bytes “spacelike”, implementando un modelado multiescala donde el procesamiento de alta capacidad se alinea con límites lingüísticos naturales. Esta aproximación teórica se basa en el principio de que la incertidumbre predictiva es máxima en las transiciones entre unidades semánticas, justificando la asignación adaptativa de recursos computacionales.

MEGABYTE (Yu et al., 2023) desarrolla la teoría de arquitecturas multiescala explícitas, donde un modelo global opera sobre parches de bytes mientras un modelo local procesa dependencias intra-parche. La reducción de complejidad a $O(T^{4/3})$ para tamaño de parche $P = T^{1/3}$ representa una contribución teórica significativa al problema del modelado de secuencias largas. H-Net (Hwang et al., 2025) avanza la teoría mediante Dynamic Chunking, donde un módulo de enrutamiento basado en similitud coseno aprende estrategias de segmentación end-to-end. Su arquitectura jerárquica tipo U-Net autoregresiva implementa formalmente el principio de compresión recursiva, permitiendo que diferentes niveles de abstracción (caracteres $\rightarrow$ palabras $\rightarrow$ frases) emerjan naturalmente del proceso de entrenamiento.

Estos desarrollos teóricos comparten el principio fundamental de que la segmentación óptima es inherentemente contextual y debe ser aprendida en lugar de predefinida. Nuestro trabajo se fundamenta en estos principios para desarrollar un marco unificado que combine eficiencia computacional con flexibilidad estructural.

2.2.3. Fundamentos Matemáticos de Leyes de Escalabilidad

Las leyes de escalabilidad en modelos de lenguaje constituyen un marco teórico-matemático que describe cuantitativamente cómo el rendimiento evoluciona en función de recursos computacionales, estableciendo principios predictivos fundamentales para el desarrollo eficiente de LLMs. La ley de Chinchilla (Hoffmann et al., 2022) representa un hito teórico al establecer la relación óptima $L(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta}$ entre parámetros (N), datos (D) y pérdida (L), donde A, B, α, β son constantes empíricas. Trabajos fundacionales como Kaplan et al. (2020) demuestran que la pérdida escala como ley de potencias en N y D , proporcionando las bases matemáticas para entender el comportamiento de escalado en arquitecturas convencionales.

La teoría de escalabilidad se extiende más allá de las relaciones empíricas para abarcar principios fundamentales de optimización recursos. El análisis de complejidad computacional establece que para arquitecturas Transformer estándar, el costo escala con $O(n^2 \cdot d)$ en atención y $O(n \cdot d^2)$ en capas feed-forward, creando restricciones fundamentales al crecimiento. Las leyes de escalabilidad también incorporan consideraciones energéticas, donde el consumo escala linealmente con FLOPs pero con constantes proporcionales que dependen críticamente de la precisión numérica y las operaciones específicas implementadas en hardware.

Para arquitecturas no convencionales, las leyes de escalabilidad requieren reformulaciones profundas. Los modelos MatMul-Free alteran las relaciones fundamentales al reemplazar operaciones $O(n^2 \cdot d)$ con alternativas lineales $O(n \cdot d)$, modificando la trayectoria de escalado óptimo. Similarmente, la cuantización ternaria introduce discontinuidades en las curvas de escalado al reducir drásticamente los requerimientos de memoria ($\sim 3,5\times$ según Ma et al. (2024)) y cambiar la naturaleza de las operaciones aritméticas de punto flotante a enteras. Los mecanismos de chunking dinámico añaden dimensiones adicionales al introducir longitud efectiva de secuencia y eficiencia de segmentación como variables en las ecuaciones de escalado.

2.2.4. Modelos de Lenguaje Pequeños (SLMs)

Los Modelos de Lenguaje Pequeños (SLMs) han emergido como una alternativa eficiente a los LLMs tradicionales, ofreciendo capacidades comparables con requerimientos computacionales significativamente reducidos. Estos modelos se caracterizan por su menor número de parámetros, mayor eficiencia en inferencia y facilidad de ajuste para dominios específicos.

Entre los SLMs más representativos se encuentra **Phi-2** (Microsoft Research, 2023), un modelo de 2.7B parámetros desarrollado por Microsoft que demuestra un rendimiento excepcional en tareas de razonamiento y programación, a pesar de su tamaño compacto. Su arquitectura Transformer convencional permite un preentrenamiento eficiente y fine-tuning especializado. Por otro lado, **DistilBERT** (Sanh et al., 2019) implementa una técnica de destilación

que reduce en un 40 % el tamaño de BERT mientras mantiene el 97 % de su rendimiento, optimizando especialmente para inferencia en dispositivos con recursos limitados.

Google DeepMind (2024) introdujo la familia **Gemma**, donde Gemma-2B destaca por su equilibrio entre tamaño y capacidades, empleando una arquitectura Transformer modificada con mecanismos de atención mejorados. **TinyLlama** (P. Zhang et al., 2024), con 1.1B parámetros, demuestra que un preentrenamiento extensivo en datasets diversificados puede compensar las limitaciones de escala, logrando un rendimiento competitivo en múltiples benchmarks.

En el ámbito de arquitecturas alternativas, **RWKV** (Peng et al., 2023) representa un paradigma innovador al combinar las ventajas de Transformers y RNNs, eliminando la atención cuadrática mediante un mecanismo basado en atención con complejidad lineal. Esto permite un preentrenamiento eficiente y una inferencia sustancialmente más rápida, especialmente en secuencias largas.

Respecto a las técnicas avanzadas, varios SLMs incorporan arquitecturas de **Mezcla de Expertos (MoE)**. Por ejemplo, modelos como Gshard de Lepikhin et al. (2020) demostraron que MoE puede escalar eficientemente la capacidad del modelo sin aumentar proporcionalmente los costos computacionales. Esta aproximación permite mantener recuentos de parámetros reducidos mientras se preserva la capacidad expresiva mediante la activación selectiva de sub-redes especializadas.

Capítulo 3

Estado del Arte

A continuación se presentan los trabajos más relevantes que constituyen el estado del arte en las dos líneas de investigación que confluyen en este trabajo: arquitecturas libres de multiplicaciones de matrices con cuantización ternaria y modelos de lenguaje que operan directamente sobre bytes sin tokenización. Se describen cinco propuestas fundamentales que abordan estos paradigmas desde distintas perspectivas.

3.1. Modelado de Lenguaje Escalable Libre de Multiplicación de Matrices

Zhu et al. (2024) proponen una arquitectura innovadora para Modelos de Lenguaje (LM) que elimina completamente las operaciones de multiplicación de matrices (MatMul), creando lo que se conoce como MatMul-Free LM. Este enfoque surge como respuesta al creciente costo computacional que representa la multiplicación de matrices en modelos que escalan hacia miles de millones de parámetros, a pesar de estar altamente optimizada para GPUs.

La arquitectura Transformer convencional depende fundamentalmente de MatMul en dos áreas clave: las capas densas y los mecanismos de auto-atención. Los autores construyen su modelo MatMul-Free siguiendo el framework Metaformer, descomponiendo el modelo en mezcla de tokens y mezcla de canales. Para reemplazar la auto-atención, implementan una Unidad Recurrente Lineal con Puertas sin MatMul (MLGRU), que utiliza exclusivamente productos elemento a elemento y acumulaciones ternarias. Paralelamente, todas las capas densas se sustituyen con módulos BitLinear que emplean pesos ternarios $(-1, 0, +1)$, transformando así las multiplicaciones en simples sumas o restas.

La Figura 3.1 presenta un análisis de leyes de escalamiento entre el modelo MatMul-free LM y el Transformer++ estándar, donde el eje X representa el cómputo utilizado durante el entrenamiento medido en FLOPs, mientras que el eje Y representa la pérdida (Loss) del modelo. Las líneas rojas trazan la evolución de la pérdida para los diferentes tamaños del MatMul-free LM (370M, 1.3B y 2.7B de parámetros), y las azules hacen lo propio para los modelos Transformer++ equivalentes; las líneas de ley de escalamiento punteadas extrapolan

esta tendencia. Puede observarse que la extrapolación del resultado de MatMul-free LM presenta una pendiente de descenso más pronunciada, y se espera que su curva intersecte y supere el rendimiento del Transformer++ alrededor de los 10^{23} FLOPs, sugiriendo que la arquitectura MatMul-Free no solo es más eficiente, sino que puede alcanzar un menor nivel de pérdida al ser escalada a modelos de gran tamaño.

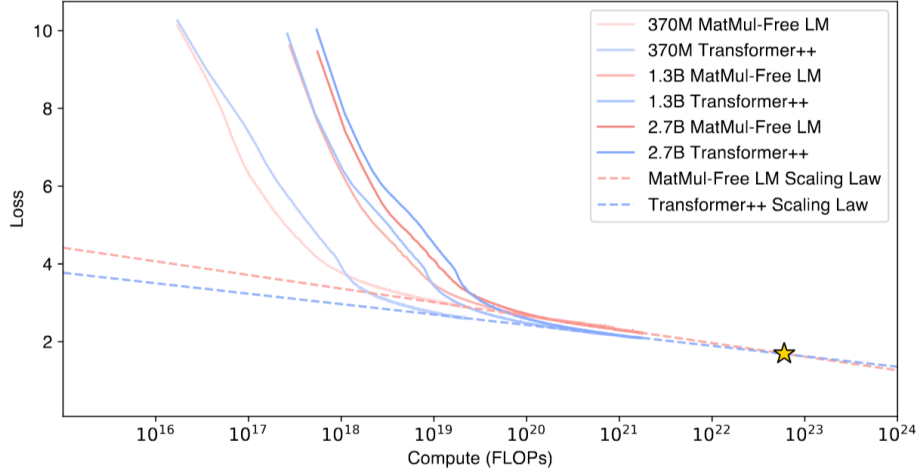


Figura 3.1. Comparación de ley de escalamiento entre modelos MatMul-Free LM y Transformer++. Tomado de Zhu et al. (2024).

La eficiencia en entrenamiento e inferencia se ve notablemente mejorada en GPUs. Una implementación fusionada de BitLinear reduce la memoria de entrenamiento en un 61 % y acelera las iteraciones en un 25.6 %. Durante la inferencia, MatMul-Free LM utiliza significativamente menos memoria y latencia que Transformer++, especialmente a escalas más grandes (por ejemplo, 4.19 GB frente a 48.50 GB para un modelo de 13B).

Además, el modelo se adapta naturalmente a hardware neuromórfico como Loihi 2 de Intel, un chip optimizado para computación escasa, de baja precisión y basada en eventos. Con pesos de 8 bits y activaciones de 16 bits, el modelo mantiene una precisión casi idéntica a la original. En un sistema Loihi multi-chip, alcanza 59.4 tokens/seg a 70.8 mJ/token durante la generación, superando a Jetson Orin Nano por al menos $4\times$ en rendimiento y $10\times$ en eficiencia energética.

La Figura 3.2 presenta una evaluación comparativa del rendimiento en tareas de lenguaje de cero disparos entre el modelo propuesto libre de multiplicación de matrices (MatMul-free LM), denominado “Ours”, y dos líneas de base: el Transformer++ estándar y el modelo MatMul-free RWKV-4, para tres escalas de parámetros (370M, 1.3B y 2.7B). Las columnas representan las métricas de precisión (%) en diferentes benchmarks: ARCe (ARC-Easy), ARCc (ARC-Challenge), HS (HellaSwag), OQ (OpenbookQA), PQ (PIQA), WGe (Wino-grande) y el promedio (Avg.) de todas ellas. Las filas detallan los resultados para cada modelo y tamaño, mostrando que el modelo MatMul-free LM compite de manera cercana con el Transformer++, observándose que la brecha de rendimiento se reduce a medida que aumenta el tamaño del modelo; por ejemplo, en el modelo de 2.7B parámetros, MatMul-Free

LM alcanza un promedio de 49.9%, muy próximo al 50.7% del Transformer++, e incluso lo supera en tareas específicas como ARCe (29.7% frente a 27.4%) y OQ (35.4% frente a 34.4%), lo que demuestra la efectividad de la arquitectura propuesta a pesar de eliminar todas las operaciones de multiplicación de matrices.

Models	Size	ARCe	ARCc	HS	OQ	PQ	WGe	Avg.
<i>370M parameters with 15B training tokens, Layer=24, d=1024</i>								
Transformer++	370M	45.0	24.0	34.3	29.2	64.0	49.9	41.1
MatMul-free RWKV-4	370M	44.7	22.8	31.6	27.8	63.0	50.3	40.0
Ours	370M	42.6	23.8	32.8	28.4	63.0	49.2	40.3
<i>1.3B parameters with 100B training tokens, Layer=24, d=2048</i>								
Transformer++	1.3B	54.1	27.1	49.3	32.4	70.3	54.9	48.0
MatMul-free RWKV-4	1.3B	52.4	25.6	45.1	31.0	68.2	50.5	45.5
Ours	1.3B	54.0	25.9	44.9	31.4	68.4	52.4	46.2
<i>2.7B parameters with 100B training tokens, Layer=32, d=2560</i>								
Transformer++	2.7B	59.7	27.4	54.2	34.4	72.5	56.2	50.7
Ours	2.7B	58.5	29.7	52.3	35.4	71.1	52.1	49.9

Figura 3.2. Precisión zero-shot de modelos MatMulfree LM y Transformer++. Tomado de Zhu et al. (2024).

Los métodos técnicos implementados son cruciales para este éxito. Las capas densas sin MatMul utilizan pesos ternarios ($-1, 0, +1$) que reemplazan la multiplicación por acumulación: para cada valor de peso, la entrada se suma, se omite o se niega, eliminando así la multiplicación en todas las capas densas. La capa BitLinear fusionada combina RMSNorm y cuantización en memoria compartida mediante un kernel especializado, reduciendo la E/S entre HBM y SRAM y optimizando significativamente el tiempo de entrenamiento y uso de memoria.

La arquitectura sin MatMul integra un mezclador de tokens basado en MLGRU, una versión lineal y ternaria de la GRU, que emplea solo productos elemento a elemento y acumulaciones, mientras que el mezclador de canales utiliza una Unidad Lineal con Puertas (GLU) con capas BitLinear. Ambos componentes están completamente libres de MatMul. Para el entrenamiento, se emplea el estimador de paso directo (STE) para manejar la cuantización no diferenciable y tasas de aprendizaje más grandes ($4e - 3$ a $1.5e - 3$) para evitar que las actualizaciones de pesos sean demasiado pequeñas. Adicionalmente, se utiliza un programador coseno con reducción a mitad del entrenamiento, resultando en una curva de pérdida en forma de S.

En la implementación para Loihi 2, el modelo se cuantiza a pesos de 8 bits y activaciones de 16 bits, utilizando aproximaciones personalizadas de punto fijo para funciones como sigmoide y raíz cuadrada inversa. El modelo se divide estratégicamente entre los neurocores de Loihi 2, aprovechando la ejecución asíncrona y la memoria local para minimizar el consumo energético y la latencia.

En conclusión, el MatMul-Free LM representa un avance significativo hacia modelos de lenguaje eficientes y escalables. Al reemplazar MatMul con operaciones ternarias y productos

elemento a elemento, el modelo logra un rendimiento sólido con costos computacionales y energéticos dramáticamente reducidos. Su inherente compatibilidad con hardware neuromórfico subraya aún más su potencial para los futuros sistemas de IA de baja potencia. El código está disponible en: <https://github.com/ridgerchu/matmulfreellm>.

3.2. Chunking Dinámico para Modelado Jerárquico de Secuencias de Extremo a Extremo

Hwang et al. (2025) proponen H-Net, una arquitectura jerárquica que incorpora un mecanismo de chunking dinámico (DC) para aprender estrategias de segmentación dependientes del contenido y contexto de forma conjunta con el resto del modelo, eliminando así la necesidad de tokenización. El Dynamic Chunking permite que el modelo procese datos brutos, como bytes, aprendiendo automáticamente cómo agruparlos en segmentos significativos sin depender de métodos predefinidos. Este enfoque aborda limitaciones fundamentales en los modelos de lenguaje modernos, donde la tokenización convencional introduce problemas como pobre comprensión a nivel de carácter, falta de adaptabilidad a diferentes lenguajes y modalidades, y sesgos inherentes. Aunque arquitecturas como el Transformer han avanzado hacia el aprendizaje de extremo a extremo, la tokenización sigue siendo un componente artesanal que impide la modelización eficiente de secuencias sin preprocesamiento.

H-Net adopta una arquitectura jerárquica similar a una U-Net autoregresiva, compuesta por tres componentes principales: las redes codificadoras (Encoder), que procesan los datos brutos y comprimen la secuencia; la red principal (Main Network), que opera sobre las representaciones comprimidas (chunks) y puede ser cualquier arquitectura estándar como Transformer, conteniendo la mayor parte de los parámetros; y las redes decodificadoras (Decoder), que reconstruyen la secuencia a su resolución original. La clave de H-Net es su diseño recursivo, donde la red principal puede ser a su vez otra H-Net, permitiendo la construcción de jerarquías multi-nivel que capturan diferentes niveles de abstracción, como de caracteres a palabras y luego a frases.

La Figura 3.3 presenta un resumen arquitectónico de H-Net con un diseño jerárquico de dos etapas ($S = 2$). La arquitectura sigue un esquema similar a U-Net, donde los datos brutos de entrada son procesados por las redes codificadoras ($\mathcal{E}^0$ y $\mathcal{E}^1$), que comprimen la secuencia mediante el mecanismo de chunking dinámico. Los chunks resultantes son procesados por la red principal ($\mathcal{M}$), que contiene la mayor parte de los parámetros del modelo y opera sobre secuencias comprimidas. Finalmente, las redes decodificadoras ($\mathcal{D}^1$ y $\mathcal{D}^0$) reconstruyen la secuencia a su resolución original utilizando la información de las conexiones residuales. El chunking dinámico, detallado en la parte derecha de la figura, consiste en un módulo de enrutamiento que predice los límites entre elementos adyacentes basándose en la similitud del coseno, y un downsampler que selecciona los vectores en las posiciones de límite para formar los chunks. Esta arquitectura permite que H-Net aprenda estrategias de segmentación de forma end-to-end, reemplazando la tokenización tradicional.

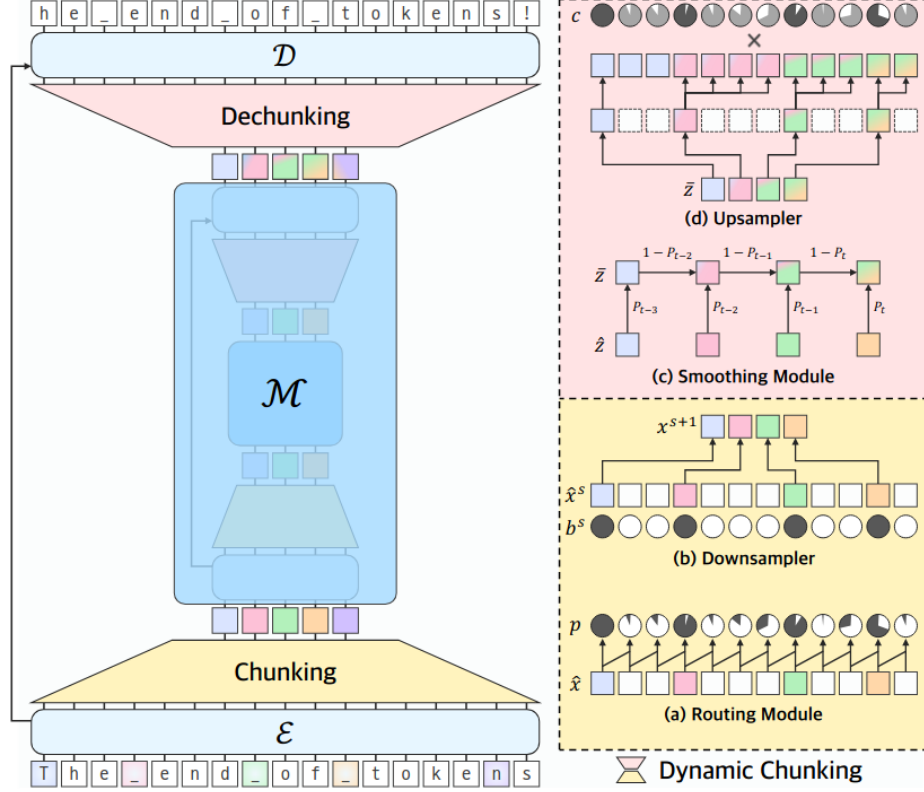


Figura 3.3. Resumen arquitectónico de H-Net. Tomado de Hwang et al. (2025).

Para mejorar la propagación de señales y la estabilidad del entrenamiento en arquitecturas jerárquicas, se introducen varias técnicas clave. El balance de normalización utiliza RMSNorm al final de cada componente de la red para equilibrar las magnitudes de las características entre las etapas jerárquicas y evitar que las señales residuales sean dominadas por las salidas de las redes más profundas. Además, la modulación de la tasa de aprendizaje asigna tasas más altas a las etapas externas, que procesan secuencias más largas, en comparación con las etapas internas, asegurando un entrenamiento equilibrado en toda la jerarquía. Los codificadores y decodificadores se implementan preferentemente con capas Mamba-2 (State Space Models), aprovechando su fuerte sesgo inductivo para la compresión y su eficiencia en el procesamiento de secuencias de grano fino.

La Figura 3.4 muestra la evolución del bits por byte (BPB) de validación durante el entrenamiento para modelos a escalas Large (760M parámetros) y XL (1.3B parámetros), todos equiparados en FLOPs y datos. Las curvas comparan H-Net de 1 y 2 etapas con líneas de base como el Transformer tokenizado con BPE, MambaByte, LlamaByte, SpaceByte, y variantes de H-Net con chunking estático o heurístico. En la escala Large, se observa que H-Net (2-stage) logra el menor BPB, superando a todos los demás modelos, y en la escala XL, H-Net (2-stage) cruza por debajo del Transformer después de solo 30B bytes de entrenamiento, demostrando una escalabilidad superior con los datos. Las líneas punteadas indican los puntos de cruce donde H-Net supera al Transformer, resaltando la ventaja del chunking dinámico aprendido end-to-end.

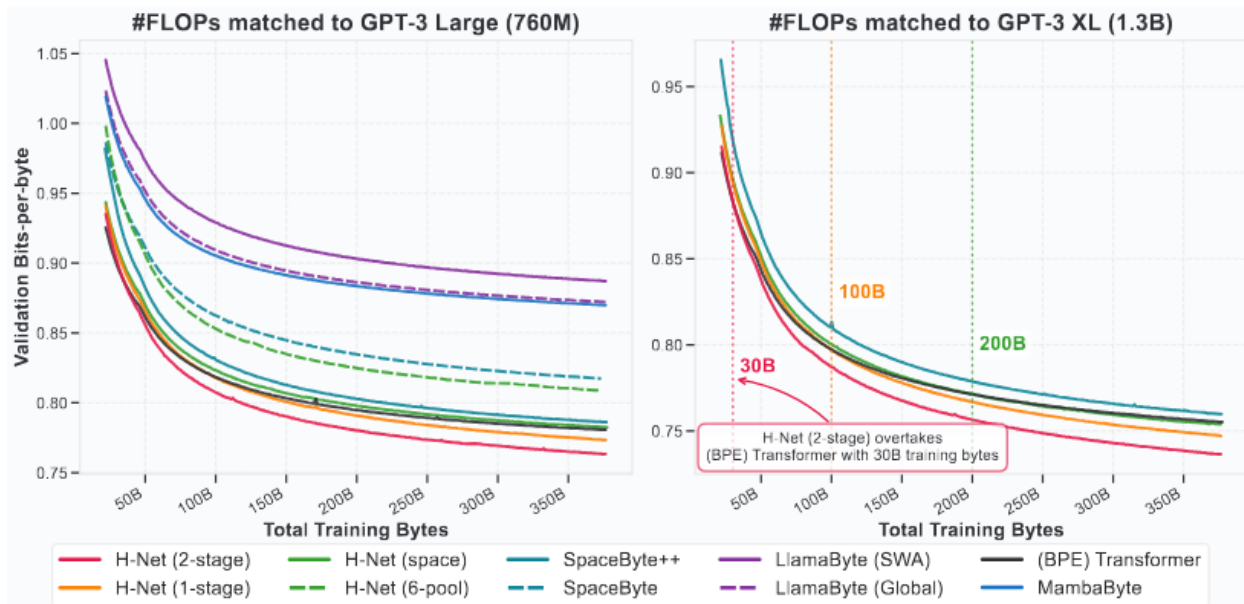


Figura 3.4. Bits por byte (BPB) de validación durante el entrenamiento. Tomado de Hwang et al. (2025).

H-Net representa un avance significativo hacia modelos de lenguaje verdaderamente de extremo a extremo, eliminando la dependencia de la tokenización mediante un mecanismo de chunking dinámico que aprende a segmentar datos brutos de forma adaptativa. Al aprender jerarquías de abstracción de forma recursiva, H-Net no solo iguala sino que supera el rendimiento de los pipelines tokenizados, especialmente a medida que escala en datos y parámetros. Su robustez, interpretabilidad y versatilidad en diferentes lenguajes y modalidades lo posicionan como una arquitectura fundamental prometedora para la próxima generación de modelos de fundación que aprenden más con menos preprocesamiento.

3.3. La Era de los LLMs de 1 bit: Todos los Modelos de Lenguaje de Gran Tamaño son de 1.58 Bits

Ma et al. (2024) proponen BitNet b1.58, un modelo de lenguaje grande donde cada parámetro es ternario, tomando valores en $-1, 0, +1$, lo que equivale a 1.58 bits por peso. La motivación surge de la necesidad de reducir el costo computacional, de memoria y energético de los LLM tradicionales en precisión completa, como FP16 o BF16, sin sacrificar su rendimiento. A diferencia de la cuantización posterior al entrenamiento, BitNet b1.58 se entrena desde cero con esta representación de baja precisión, lo que define un nuevo paradigma de computación basado en operaciones enteras en lugar de operaciones de punto flotante.

El artículo propone una arquitectura basada en Transformer que reemplaza las capas lineales estándar por una versión cuantizada denominada BitLinear. Entre las modificaciones clave se incluye una función de cuantización que utiliza un método absmean, el cual escala la matriz de pesos por su valor absoluto promedio y luego redondea cada valor a $-1, 0, +1$. Por

otro lado, las activaciones se cuantizan a 8 bits y se escalan por token al rango $[-Q_b, Q_b]$, eliminando la necesidad de un punto cero y simplificando la implementación. Además, se adoptan componentes estándar como RMSNorm, SwiGLU, rotary embeddings y se eliminan los sesgos para facilitar la integración con herramientas de código abierto.

En cuanto a los resultados experimentales, los autores comparan BitNet b1.58 con LLaMA en precisión completa bajo las mismas condiciones, incluyendo tamaño del modelo y tokens de entrenamiento. La Figura 3.5 presenta una evaluación comparativa del rendimiento en tareas de lenguaje de cero disparos entre BitNet b1.58 y LLaMA LLM para tres escalas de parámetros (700M, 1.3B y 3B). Las columnas representan las métricas de precisión (%) en diferentes benchmarks: ARCe (ARC-Easy), ARCc (ARC-Challenge), HS (HellaSwag), BQ (BoolQ), OQ (OpenbookQA), PQ (PIQA), WGe (Winogrande) y el promedio (Avg.) de todas ellas. Puede observarse que la brecha de rendimiento se reduce a medida que aumenta el tamaño del modelo; en el modelo de 3B parámetros, BitNet b1.58 alcanza un promedio de 50.2%, superando ligeramente el 49.7% de LLaMA LLM, e incluso lo supera en tareas específicas como ARCc (28.3% frente a 25.6%) y OQ (26.6% frente a 24.6%), lo que demuestra la efectividad de la arquitectura propuesta a pesar de emplear pesos ternarios.

Models	Size	ARCe	ARCc	HS	BQ	OQ	PQ	WGe	Avg.
LLaMA LLM	700M	54.7	23.0	37.0	60.0	20.2	68.9	54.8	45.5
BitNet b1.58	700M	51.8	21.4	35.1	58.2	20.0	68.1	55.2	44.3
LLaMA LLM	1.3B	56.9	23.5	38.5	59.1	21.6	70.0	53.9	46.2
BitNet b1.58	1.3B	54.9	24.2	37.7	56.7	19.6	68.8	55.8	45.4
LLaMA LLM	3B	62.1	25.6	43.3	61.8	24.6	72.1	58.2	49.7
BitNet b1.58	3B	61.4	28.3	42.9	61.5	26.6	71.5	59.3	50.2
BitNet b1.58	3.9B	64.2	28.7	44.2	63.5	24.2	73.2	60.5	51.2

Figura 3.5. Precisión zero-shot de BitNet b1.58 y LLaMA LLM. Tomado de Ma et al. (2024).

La Figura 3.6 resume la perplejidad así como el costo en memoria y latencia de BitNet b1.58 y LLaMA LLM. El eje izquierdo presenta la reducción en el consumo de memoria de GPU (GB) y la latencia (ms) por token, mientras que el eje derecho muestra la perplejidad (PPL) en los conjuntos de validación. Las barras rojas trazan la ventaja en memoria y latencia de BitNet b1.58 sobre LLaMA LLM para los diferentes tamaños de modelo (700M, 1.3B y 3B), y las líneas azules representan la perplejidad correspondiente. Se observa que BitNet b1.58 iguala o supera la perplejidad de LLaMA LLM a partir del tamaño de 3B (9.91 frente a 10.04), mientras que reduce la memoria en 3.55 veces (2.22 GB frente a 7.89 GB) y la latencia en 2.71 veces (1.87 ms frente a 5.07 ms). Esta ventaja se acentúa con el escalado, donde el modelo de 70B es 4.1 veces más rápido.

Models	Size	Memory (GB)↓	Latency (ms)↓	PPL↓
LLaMA LLM	700M	2.08 (1.00x)	1.18 (1.00x)	12.33
BitNet b1.58	700M	0.80 (2.60x)	0.96 (1.23x)	12.87
LLaMA LLM	1.3B	3.34 (1.00x)	1.62 (1.00x)	11.25
BitNet b1.58	1.3B	1.14 (2.93x)	0.97 (1.67x)	11.29
LLaMA LLM	3B	7.89 (1.00x)	5.07 (1.00x)	10.04
BitNet b1.58	3B	2.22 (3.55x)	1.87 (2.71x)	9.91
BitNet b1.58	3.9B	2.38 (3.32x)	2.11 (2.40x)	9.62

Figura 3.6. Perplejidad y costo de inferencia de BitNet b1.58 y LLaMA LLM. Tomado de Ma et al. (2024).

Respecto a la energía, el consumo en operaciones aritméticas se reduce en 71.4 veces gracias al reemplazo de multiplicaciones FP16 por sumas enteras. En throughput, con 70B de parámetros, BitNet b1.58 soporta un batch size 11 veces mayor y un throughput 8.9 veces superior al de LLaMA.

BitNet b1.58 establece una nueva ley de escalado que relaciona el rendimiento y el costo de inferencia. Por ejemplo, un BitNet b1.58 de 13B es más eficiente que un LLaMA de 3B, y un BitNet b1.58 de 70B es más eficiente que un LLaMA de 13B. Además, al entrenar con 2 billones de tokens, BitNet b1.58 supera a modelos de referencia como StableLM-3B en múltiples benchmarks, demostrando una fuerte capacidad de generalización.

En la discusión y trabajo futuro, los autores identifican varias direcciones prometedoras. Los modelos de Mixture-of-Experts pueden beneficiarse de la reducción de memoria y comunicación entre dispositivos. La reducción de bits en las activaciones permite duplicar la longitud de contexto sin incrementar el uso de memoria, lo que facilita el soporte nativo para secuencias largas. La eficiencia de BitNet b1.58 también posibilita la ejecución en dispositivos edge y móviles con recursos limitados. Finalmente, se propone el diseño de nuevo hardware especializado optimizado para operaciones con 1 bit, similar a lo realizado por Groq con sus LPUs.

En conclusión, BitNet b1.58 representa un avance significativo hacia LLMs más sostenibles y escalables. Al redefinir el paradigma de computación desde operaciones en punto flotante hacia operaciones enteras de 1.58 bits, no solo se iguala el rendimiento de los modelos en precisión completa, sino que se logran mejoras sustanciales en eficiencia. Esto sienta las bases para una nueva generación de modelos de lenguaje más accesibles y aplicables en entornos con restricciones de recursos.

3.4. SpaceByte: Hacia la Eliminación de la Tokenización en el Modelado de Lenguaje de Gran Tamaño

Slagle (2024) propone SpaceByte, una nueva arquitectura byte-level que cierra la brecha de rendimiento entre los modelos basados en bytes y los basados en subpalabras, argumentando

que la tokenización, aunque ampliamente utilizada en Modelos de Lenguaje a Gran Escala (LLMs), introduce importantes desventajas como sesgos de rendimiento, vulnerabilidad adversarial, menor rendimiento en modelado a nivel de carácter y complejidad adicional.

SpaceByte es un modelo Transformer a nivel de bytes que incorpora bloques Transformer más grandes, denominados bloques globales, insertados estratégicamente en posiciones específicas de la secuencia, particularmente después de bytes considerados "spacelike" (como espacios), que suelen denotar límites de palabras. Los experimentos controlados por costo computacional muestran que SpaceByte supera a otras arquitecturas basadas en bytes y se equipara en rendimiento con los Transformers tokenizados.

La Figura 3.7 presenta un resumen de la arquitectura SpaceByte. En esencia, puede conceptualizarse como un Transformer a nivel de bytes estándar, pero con bloques transformadores "globales" adicionales (con una dimensión de modelo más grande) insertados en medio de las capas, los cuales se aplican solo una fracción del tiempo. Mientras que arquitecturas como MegaByte aplican estos bloques globales cada $P \sim 8$ bytes de forma fija, SpaceByte utiliza una regla dinámica basada en el contenido del texto. La intuición subyacente es que el primer carácter de una palabra es típicamente más difícil de predecir que los caracteres siguientes; por lo tanto, se espera que el rendimiento mejore aplicando los bloques globales principalmente en los límites de las palabras, donde reside la mayor incertidumbre predictiva.

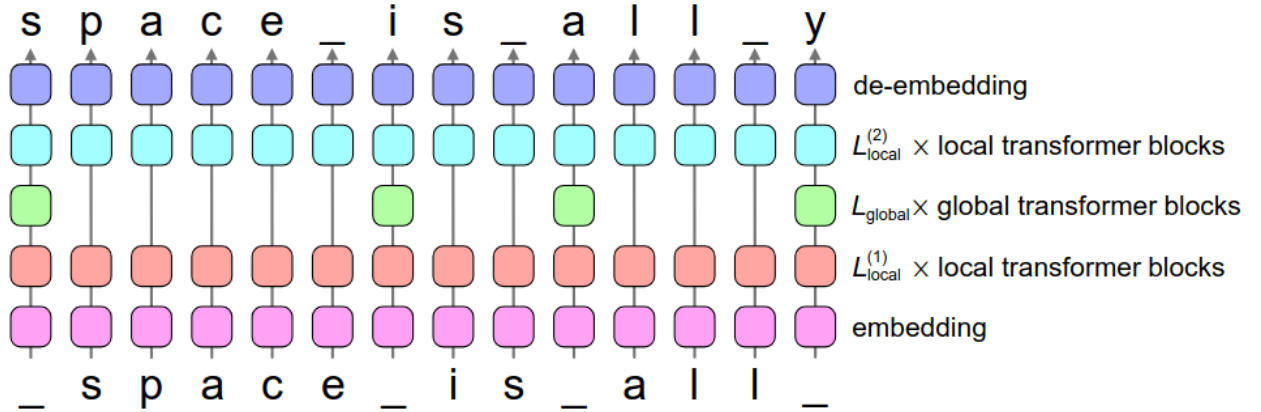


Figura 3.7. Resumen arquitectónico de SpaceByte. Tomado de Slagle (2024).

La Figura 3.8 presenta la frontera de Pareto que compara la entropía cruzada (medida en bits por byte) y el costo computacional de inferencia (medido en FLOPs por byte) para las diferentes arquitecturas evaluadas en el conjunto de datos Github, entrenadas con presupuestos de 10^{18} (líneas finas) y 10^{19} (líneas gruesas) FLOPs. Cada punto representa un modelo con diferentes dimensiones y/o número de capas. Puede observarse que SpaceByte (línea roja) supera consistentemente a todas las demás arquitecturas a nivel de bytes en toda la frontera de Pareto para todos los conjuntos de datos. Además, su rendimiento es aproximadamente equivalente al del Transformer con subpalabras que utiliza el tokenizer SentencePiece (línea negra), e incluso lo supera en los conjuntos de datos arXiv y Github. Es notable también que el rendimiento de SpaceByte mejora más rápidamente que el del Transformer con subpalabras a medida que aumenta el presupuesto de entrenamiento. La

versión de SpaceByte con parches de tamaño fijo (línea naranja) muestra un rendimiento significativamente inferior, lo que subraya la importancia crítica de la partición dinámica alineada con los límites lingüísticos.

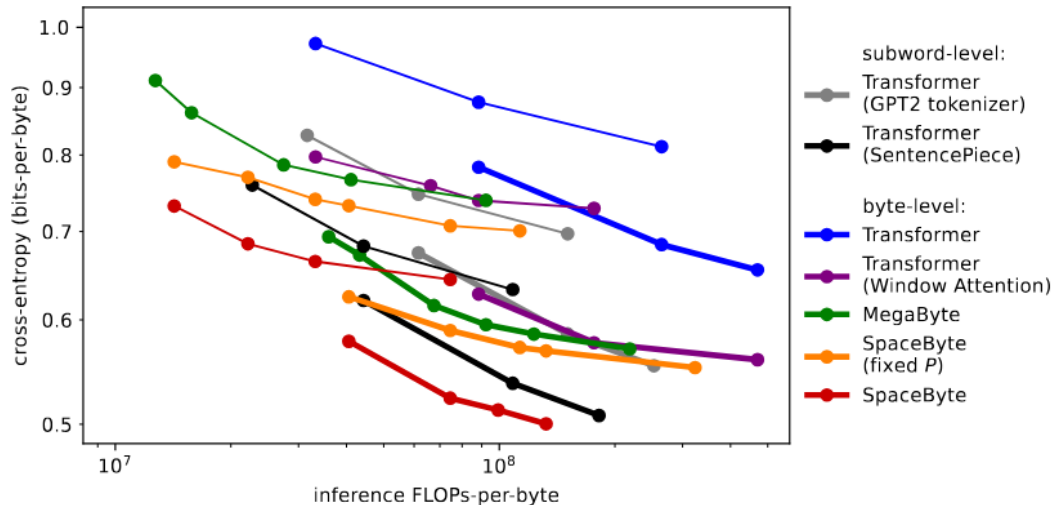


Figura 3.8. Frontera de Pareto de la entropía cruzada (bits por byte) frente a los FLOPs por byte durante la inferencia para diferentes arquitecturas en el dataset Github. Tomado de Slagle (2024).

Para una comparación justa, todos los modelos se entrenaron con presupuestos computacionales fijos (10^{18} o 10^{19} FLOPs) y se evaluaron en términos de bits por byte (BPB) y FLOPs por byte. Los conjuntos de datos incluyeron PG-19 (libros en inglés anteriores a 1919), arXiv (artículos científicos en LaTeX) y Github (código fuente de repositorios públicos). Se compararon múltiples arquitecturas: Transformers a nivel de bytes (base y con ventana de atención), MegaByte (con parches de tamaño fijo), SpaceByte (con parches dinámicos y fijos) y Transformers tokenizados (con tokenizadores GPT-2 y SentencePiece).

Los resultados experimentales demuestran que SpaceByte supera consistentemente a todas las demás arquitecturas basadas en bytes y se equipara con el Transformer tokenizado con SentencePiece, incluso superándolo en los conjuntos de datos arXiv y Github. La versión de SpaceByte con parches de tamaño fijo tiene un rendimiento significativamente peor, lo que resalta la importancia de alinear los bloques globales con los límites lingüísticos. SpaceByte logra un mejor equilibrio entre bits por byte y FLOPs por byte, especialmente en contextos largos, y supera a MegaByte y MambaByte, incluso cuando estos últimos fueron entrenados con más datos y computación. SpaceByte logra los mejores bits por byte en arXiv (0.748) y Github (0.500), y se mantiene competitivo en PG-19 (1.009), siempre dentro del 2.5 % del mejor modelo por conjunto de datos.

Cabe señalar que la regla heurística basada en bytes spacelike funciona bien en textos en inglés, LaTeX y código, pero puede no ser óptima para otros idiomas o modalidades. Por ejemplo, en chino (que no usa espacios entre palabras), el rendimiento de SpaceByte es inferior al de los modelos tokenizados. Además, la naturaleza variable de los parches dinámicos

complica la implementación de inferencia en lote eficiente.

Como trabajo futuro, los autores sugieren extender el modelado multiescala a niveles superiores (oraciones, párrafos), integrar bloques Mamba en lugar de Transformers para mayor eficiencia, optimizar la regla de inserción de bloques globales mediante aprendizaje automático en lugar de heurísticas, y mejorar la eficiencia en inferencia con secuencias de longitud variable.

En conclusión, SpaceByte representa un avance significativo hacia la eliminación de la tokenización en LLMs. Al introducir un modelado multiescala dinámico que alinea el procesamiento global con los límites lingüísticos naturales, logra un rendimiento competitivo con los modelos tokenizados, al tiempo que evita sus desventajas inherentes. Esto sienta las bases para LLMs más robustos, eficientes y menos sesgados en el futuro.

3.5. MEGABYTE: Prediciendo Secuencias de Millones de Bytes con Transformers Multiescala

Yu et al. (2023) proponen MegaByte, una arquitectura de decodificador multiescala que permite el modelado diferenciable de secuencias de más de un millón de bytes. MegaByte divide las secuencias en parches y utiliza un modelo local dentro de cada parche y un modelo global entre parches. Esto permite autoatención subcuadrática, capas feedforward más grandes para el mismo coste computacional y un mayor paralelismo durante la generación. Una ventaja clave de MegaByte es que opera a nivel de bytes, eliminando la necesidad de tokenización, lo que simplifica el preprocesamiento y evita problemas de generalización a nuevos dominios o modos.

La Figura 3.9 ilustra la arquitectura de MegaByte con un tamaño de parche $P = 4$. La secuencia de entrada se divide en parches de P bytes, cada uno de los cuales es embedido y luego procesado por el modelo global. El modelo global, un transformador autoregresivo, contextualiza las representaciones de los parches anteriores y produce una salida que se proyecta y combina con los embeddings de los bytes anteriores para alimentar el modelo local. El modelo local, a su vez, es un transformador más pequeño que predice autoregresivamente cada byte dentro del parche, condicionado por la salida del modelo global. Esta arquitectura permite una autoatención subcuadrática, ya que el modelo global opera sobre una secuencia de T/P parches, reduciendo la complejidad a $O(T^{4/3})$ cuando $P = T^{1/3}$. Además, las capas feedforward se aplican a nivel de parche, lo que permite utilizar capas más grandes y expresivas sin incrementar el coste computacional. Durante la generación, el modelo global puede producir representaciones de múltiples parches en paralelo, reduciendo las operaciones seriales y acelerando la inferencia.

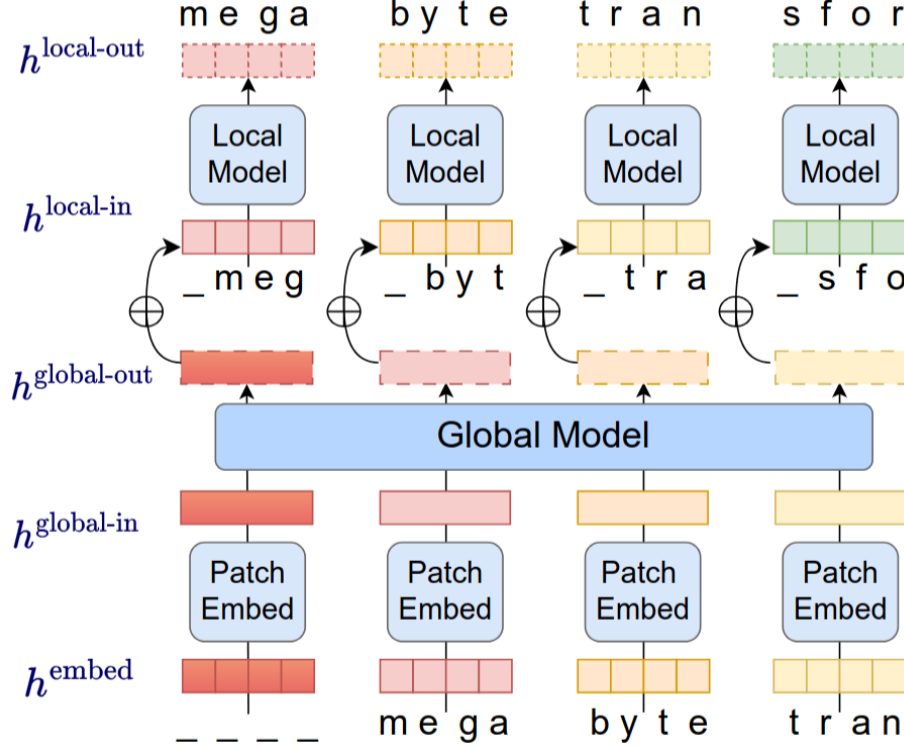


Figura 3.9. Resumen arquitectónico de MegaByte con $P = 4$. Tomado de Yu et al. (2023)

La Figura 3.10 presenta una evaluación comparativa del rendimiento de MegaByte frente a dos líneas base, Transformer y PerceiverAR, en cinco conjuntos de datos de texto largo: PG-19, Stories, Books, arXiv y Code. La métrica reportada es bits por byte (bpb), donde un valor más bajo indica mejor desempeño. Puede observarse que MegaByte supera consistentemente a ambos baselines en todos los conjuntos de datos, logrando el menor bpb en cada uno de ellos. Por ejemplo, en el dataset de Code, MegaByte alcanza un bpb de 0.411, mientras que Transformer y PerceiverAR obtienen 0.575 y 0.546 respectivamente. Estos resultados demuestran la efectividad de la arquitectura multiescala de MegaByte para el modelado de lenguaje a nivel de bytes, sin necesidad de tokenización.

	PG-19	Stories	Books	arXiv	Code
Transformer	1.057	1.064	1.097	0.816	0.575
PerceiverAR	1.104	1.070	1.104	0.791	0.546
MEGABYTE	1.000	0.978	1.007	0.678	0.411

Figura 3.10. Rendimiento en bit por byte según dataset. Tomado de Yu et al. (2023)

En el análisis y ablaciones, se identifican componentes críticos: la eliminación del modelo local o global degrada significativamente el rendimiento, confirmando que ambos son esenciales. Para mitigar la caída de rendimiento al final de cada parche, se propone una inferencia

strideada con dos pasadas desplazadas, mejorando el bpb a costa de duplicar el coste. El rendimiento es robusto a variaciones en el tamaño del parche, aunque el óptimo varía según la modalidad. También es beneficioso asignar más parámetros al modelo global que al local.

MegaByte establece un nuevo paradigma para el modelado autoregresivo de secuencias largas a nivel de bytes, eliminando la necesidad de tokenización y ofreciendo ventajas en eficiencia, escalabilidad y generalidad. Sus resultados competitivos en texto, imágenes y audio sugieren que los modelos basados en bytes pueden rivalizar con los basados en tokens, abriendo la puerta a futuros modelos de lenguaje totalmente libres de tokenización. Sin embargo, los experimentos realizados están aún por debajo de la escala de los modelos de lenguaje más grandes, por lo que un futuro trabajo debería explorar el escalado de MegaByte a modelos y conjuntos de datos masivos.

Capítulo 4

Arquitectura del modelo propuesto HNetBit

En este capítulo se describe en detalle la arquitectura del modelo híbrido propuesto, denominado HNetBit (*Hierarchical Network with BitLinear ternary weights*), que constituye el núcleo de esta investigación. La exposición se organiza desde los componentes más básicos hasta el sistema completo, presentando las innovaciones en cuantización ternaria, recurrencia HGRN y chunking dinámico jerárquico, así como sus fundamentos matemáticos y computacionales.

4.1. Visión general de la arquitectura

HNetBit es un modelo de lenguaje autorregresivo que opera directamente sobre bytes, eliminando por completo la necesidad de tokenización. La arquitectura integra tres innovaciones principales:

1. **Cuantización ternaria:** todas las proyecciones lineales del modelo (con excepción de la tabla de *embeddings* y la proyección residual de cada etapa) utilizan la capa **BitLinear**, que reemplaza las multiplicaciones de punto flotante por acumulaciones ternarias con pesos en $\{-1, 0, +1\}$, equivalente a aproximadamente 1.58 bits por parámetro.
2. **Recurrencia HGRN** (*Hierarchically Gated Recurrent Network*): un mecanismo de mezcla de secuencia con complejidad $O(L)$ en entrenamiento y $O(1)$ por paso en inferencia, que sustituye a la autoatención cuadrática del Transformer.
3. **Chunking dinámico multi-etapa:** un sistema de segmentación jerárquica aprendida que comprime la secuencia recursivamente, eliminando la necesidad de un tokenizador predefinido.

El modelo se organiza como una jerarquía de etapas (*stages*), donde cada etapa no interna contiene un módulo de enrutamiento que decide dinámicamente qué posiciones de la secuencia constituyen límites de fragmentos (*chunks*). Los *chunks* resultantes se procesan

recursivamente por la siguiente etapa, permitiendo que el modelo aprenda representaciones cada vez más abstractas.

HNetBit está diseñado para operar en dos regímenes distintos: **entrenamiento** e **inferencia autorregresiva**. La mayoría de los componentes arquitectónicos (codificadores, decodificadores, bloques HGRN, capas BitLinear, módulos de chunking) se utilizan en ambos. Sin embargo, ciertos elementos son exclusivos de cada régimen:

- **Exclusivos de entrenamiento:** el estimador STE dentro de BitLinear (cuyo efecto desaparece bajo `torch.no_grad()`), la pérdida de balanceo de carga $\mathcal{L}_{LB}$, la entropía cruzada $\mathcal{L}_{CE}$, y el kernel `chunk_hgrn` para recurrencia paralela en secuencias largas.
- **Exclusivos de inferencia:** la estructura de caché HNetBitCache y todas sus sub-cachés, el kernel `fused_recurrent_hgrn` para recurrencia secuencial paso a paso, los estados `RoutingModuleState` y `DeChunkState`, y la lógica de omisión condicional de la etapa interna para tokens no-límite. En la variante de despliegue compacto (*deployment*), los pesos ternarios se congelan y precuantizan, eliminando la necesidad de re-ejecutar `weight_quant` en cada paso.

Las secciones siguientes detallan cada componente, indicando explícitamente cuándo su uso es exclusivo de un régimen o compartido.

4.2. Cuantización ternaria y capa BitLinear

La cuantización ternaria constituye el mecanismo principal de eficiencia computacional en el modelo propuesto. A continuación se describen los fundamentos teóricos y la implementación matemática de este esquema.

4.2.1. Fundamentos de la cuantización ternaria

La cuantización ternaria representa cada peso $w \in \mathbb{R}$ mediante un valor en el conjunto discreto $\{-1, 0, +1\}$. La entropía de esta representación es $\log_2 3 \approx 1.58$ bits por parámetro (Ma et al., 2024). Esta reducción drástica en precisión numérica permite reemplazar las operaciones de multiplicación de punto flotante por simples sumas, restas u omisiones: para cada peso $w_q \in \{-1, 0, +1\}$, la contribución a la salida es $w_q \cdot x$, que equivale a sumar x , restar x o no hacer nada.

4.2.2. Cuantización de pesos

Dada una matriz de pesos $W \in \mathbb{R}^{d_{out} \times d_{in}}$, el proceso de cuantización ternaria sigue el método `absmean` (Ma et al., 2024):

$$\alpha = \frac{1}{\text{mean}(|W|)} \in \mathbb{R}, \quad (4.1)$$

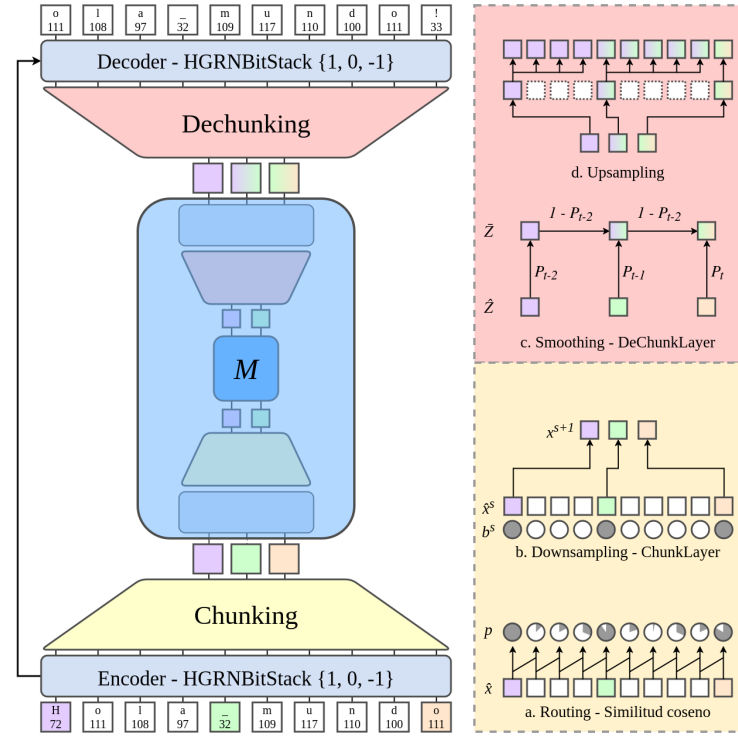


Figura 4.1. Arquitectura del modelo HNetBitForCausalLM. El panel izquierdo muestra el pipeline principal desde la entrada de bytes hasta los logits de salida, con el flujo ascendente a través del codificador, chunking, modelo central, dechunking y decodificador. El panel derecho detalla los cuatro componentes del mecanismo de *dynamic chunking*: (a) módulo de enrutamiento basado en similitud coseno, (b) submuestreador que selecciona los tokens límite mediante argsort, (c) módulo de suavizado mediante media móvil exponencial (EMA) y (d) sobre-muestreador que reconstruye la secuencia completa. Los colores consistentes (azul, verde, naranja) representan cuatro fragmentos distintos a través de todo el proceso.

$$W_q = \frac{\text{round_clamp}(W \cdot \alpha, -1, 1)}{\alpha} \in \left\{ -\frac{1}{\alpha}, 0, \frac{1}{\alpha} \right\}^{d_{\text{out}} \times d_{\text{in}}}, \quad (4.2)$$

donde $\text{round_clamp}(x, a, b)$ redondea cada elemento de x al entero más cercano y luego lo limita al rango $[a, b]$. El factor de escala α se calcula por tensor completo y se almacena implícitamente: durante el entrenamiento, la división por α se cancela en la multiplicación lineal subsiguiente, por lo que el cómputo efectivo utiliza los valores enteros $\{-1, 0, +1\}$.

La función de cuantización es no diferenciable. Para permitir el entrenamiento mediante backpropagation, se utiliza el estimador de paso directo (*Straight-Through Estimator*, STE), que aproxima el gradiente de la cuantización como la identidad (Bengio et al., 2013):

$$W_{\text{forward}} = W_q, \quad \frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial W_q}. \quad (4.3)$$

Durante el paso backward, los gradientes fluyen a través de los pesos completos en precisión original, permitiendo que el optimizador actualice los pesos de punto flotante que luego serán recuantizados en el siguiente paso forward.

4.2.3. Cuantización de activaciones

Las activaciones de entrada a cada capa **BitLinear** se normalizan primero con RMSNorm y luego se cuantizan a 8 bits enteros. La cuantización es simétrica (sin punto cero) y se aplica por token:

$$x_{\text{norm}} = \text{RMSNorm}(x) \in \mathbb{R}^{B \times L \times d}, \quad (4.4)$$

$$\beta_t = \frac{127}{\max(|x_{\text{norm},t}|)} \in \mathbb{R}, \quad x_{q,t} = \frac{\text{round_clamp}(x_{\text{norm},t} \cdot \beta_t, -128, 127)}{\beta_t}, \quad (4.5)$$

donde t indexa la dimensión del token (posición en la secuencia). La cuantización simétrica simplifica la implementación en hardware al eliminar la necesidad de un punto cero, manteniendo la precisión suficiente para la tarea de modelado de lenguaje.

4.2.4. Capa BitLinear

La capa **BitLinear** integra la normalización y la doble cuantización (pesos y activaciones) en un módulo unificado. Dada una entrada $x \in \mathbb{R}^{B \times L \times d_{\text{in}}}$ y una matriz de pesos $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, el paso forward es:

$$\begin{aligned} x_{\text{norm}} &= \text{RMSNorm}(x), \\ \tilde{x} &= x_{\text{norm}} + (\text{activation_quant}(x_{\text{norm}}) - x_{\text{norm}}).\text{detach}(), \\ \tilde{W} &= W + (\text{weight_quant}(W) - W).\text{detach}(), \\ y &= \text{Linear}(\tilde{x}, \tilde{W}). \end{aligned} \quad (4.6)$$

La operación **detach()** en ambos términos implementa el STE: durante el paso forward se utilizan los valores cuantizados; durante el backward, el gradiente fluye a través de la versión completa (no cuantizada) de x y W , ignorando la no-diferenciabilidad de las funciones de cuantización. El STE es un mecanismo **exclusivo de entrenamiento**: durante la inferencia autorregresiva el modelo opera bajo **torch.no_grad()**, por lo que no se calculan gradientes y el patrón **detach()** es inocuo.

En la variante de despliegue compacto (*deployment*), los pesos se congelan y precuantizan una sola vez al exportar el modelo, almacenándose directamente en su forma ternaria empaquetada ($\approx 2,1$ bits/parámetro). Esto elimina la necesidad de ejecutar **weight_quant** en cada paso de inferencia, reduciendo tanto el cómputo como la memoria requerida.

Esta doble cuantización convierte la multiplicación de matrices en una acumulación ternaria. Dado que los pesos cuantizados pertenecen a $\{-1, 0, +1\}$, la operación lineal $y = \tilde{W} \cdot \tilde{x}$ se

reduce a sumas y restas selectivas de los elementos de $\tilde{x}$, eliminando las multiplicaciones de punto flotante que constituyen el principal costo computacional en las capas lineales estándar.

4.2.5. Variante fusionada: FusedBitLinear

Para maximizar la eficiencia en GPU, se implementó una variante **FusedBitLinear** que fusiona la normalización RMSNorm, la cuantización de activaciones y la multiplicación ternaria en un solo kernel Triton (Tillet et al., 2019). Este kernel reduce los viajes de ida y vuelta entre la memoria HBM (*High Bandwidth Memory*) y la SRAM (*Shared Memory*) en un factor de $3\times$, al procesar todas las operaciones en una sola pasada de datos.

El kernel utiliza un esquema de bloque por fila/columna con autotuning sobre el número de *warps* $\{1, 2, 4, 8, 16, 32\}$, seleccionando la configuración óptima según la dimensión del modelo. En el paso backward, el kernel implementa recomputación de la salida para ahorrar memoria, evitando almacenar los tensores intermedios de activación cuantizada.

Sin Triton (CPU o Docker sin privilegios), **FusedBitLinear** usa `BitLinear.forward()` puro. La variable `HNETBIT_DISABLE_TRITON` controla este fallback.

4.2.6. Normalización RMSNorm

La normalización RMSNorm (*Root Mean Square Layer Normalization*) (B. Zhang & Sennrich, 2019) se utiliza como alternativa a LayerNorm en todo el modelo, debido a su menor costo computacional al eliminar la estimación de la media:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \odot \gamma, \quad (4.7)$$

donde $\gamma \in \mathbb{R}^d$ es un vector de escala aprendible y ϵ es una constante de estabilidad numérica. RMSNorm se aplica siguiendo el patrón de pre-normalización residual (*pre-norm residual*): cada subcapa (atención o MLP) recibe la entrada normalizada antes de la transformación, y la salida se suma a la conexión residual:

$$x' = x + f(\text{RMSNorm}(x)), \quad (4.8)$$

donde f representa la subcapa correspondiente.

4.3. Recurrencia HGRN

El mecanismo de mezcla de secuencia principal del modelo es la *Hierarchically Gated Recurrent Network* (HGRN), que reemplaza completamente a la autoatención del Transformer. Esta sección describe su formulación matemática, componentes auxiliares e implementación computacional.

4.3.1. Motivación

La autoatención en el Transformer tiene complejidad $O(L^2 \cdot d)$ en tiempo y $O(L^2)$ en memoria, lo que limita el escalado a secuencias largas. La recurrencia HGRN ofrece tres ventajas fundamentales:

- **Complejidad lineal:** $O(L)$ en entrenamiento y $O(1)$ por paso en inferencia.
- **Estado oculto de tamaño fijo:** a diferencia del *KV cache* del Transformer, que crece linealmente con la longitud de secuencia, el estado recurrente ocupa $O(d)$ independientemente de L .
- **Eficiencia en hardware:** las operaciones elemento a elemento se benefician de la arquitectura SIMT (*Single Instruction, Multiple Threads*) de las GPUs modernas.

4.3.2. Formulación matemática

Dado un token $x_t \in \mathbb{R}^d$ en la posición t , el HGRN actualiza un estado oculto $h_t \in \mathbb{R}^d$ mediante tres compuertas principales:

$$\begin{aligned} f_t &= \sigma(\text{BitLinear}_f(x_t)) \in (0, 1)^d, \\ i_t &= \text{SwiGLU}(\text{BitLinear}_i(x_t), \mathbf{1} - f_t), \\ h_t &= f_t \odot h_{t-1} + i_t, \\ o_t &= \text{FusedRMSNormSwishGate}(\text{BitLinear}_g(x_t), h_t). \end{aligned} \tag{4.9}$$

donde σ es la función sigmoide, $\odot$ denota el producto de Hadamard, $\mathbf{1}$ es un vector de unos, y f_t , i_t son las compuertas de olvido (*forget gate*) y entrada (*input gate*) respectivamente.

La compuerta de olvido f_t controla qué fracción del estado anterior se retiene. La compuerta de entrada utiliza un mecanismo de compuerta complementaria: i_t se obtiene aplicando SwiGLU sobre la proyección de entrada y el complemento de la compuerta de olvido $1 - f_t$. Esto crea una dependencia directa entre la retención y la incorporación de nueva información: cuando f_t es alto (mucha retención), $1 - f_t$ es bajo, por lo que se incorpora poca información nueva, y viceversa.

El estado oculto h_t es una interpolación lineal entre el estado anterior h_{t-1} y la entrada transformada i_t , ponderada por las compuertas:

$$h_t = f_t \odot h_{t-1} + (\mathbf{1} - f_t) \odot \tilde{i}_t, \tag{4.10}$$

donde $\tilde{i}_t$ es la entrada después de SwiGLU. Esta formulación es similar a la de una GRU (*Gated Recurrent Unit*) (Cho et al., 2014), pero con la diferencia fundamental de que todas las proyecciones utilizan cuantización ternaria en lugar de pesos completos.

4.3.3. Mecanismo multi-cabeza

Para aumentar la capacidad expresiva, HGRN soporta múltiples cabezas recurrentes. El estado oculto se redimensiona de $(B, L, d_{\text{hidden}})$ a $(B, H, L, D_{\text{head}})$, donde H es el número de cabezas, $D_{\text{head}} = (d_{\text{hidden}} \cdot r)/H$ es la dimensión por cabeza y r es el factor de expansión del estado (`expand_ratio`). La recurrencia se aplica independientemente en cada cabeza:

$$h_t^{(c)} = f_t^{(c)} \odot h_{t-1}^{(c)} + i_t^{(c)}, \quad c = 1, \dots, H. \quad (4.11)$$

La paralelización a través de las cabezas permite un uso eficiente de las GPUs, ya que cada hilo puede procesar una cabeza diferente de forma independiente.

4.3.4. Función de activación SwiGLU

La función SwiGLU (*Sigmoid-weighted Linear Unit with Gating*) (Shazeer, 2020) combina la activación Swish con una compuerta multiplicativa:

$$\text{SwiGLU}(x, g) = \text{Swish}(x) \odot g = \frac{x \odot g}{1 + e^{-x}}. \quad (4.12)$$

La implementación en el modelo utiliza un kernel CUDA optimizado mediante la interfaz jiterator, con funciones forward y backward fusionadas para minimizar la sobrecarga de lanzamiento de kernels.

4.3.5. Normalización de compuerta fusionada

La salida del HGRN se procesa mediante `FusedRMSNormSwishGate`, un módulo que fusiona la normalización RMSNorm y una compuerta Swish en un solo kernel Triton. La operación combinada es:

$$o_t = \text{RMSNorm}(\text{BitLinear}_g(x_t)) \odot h_t \odot \sigma(h_t). \quad (4.13)$$

Esta fusión cuadruplica la eficiencia al reducir cuatro operaciones separadas (normalización, proyección g , multiplicación por h , y activación compuerta sigmoide) a un solo lanzamiento de kernel con autotuning.

4.3.6. Convolución corta (*Short Convolution*)

Opcionalmente, cada bloque HGRN puede incluir una convolución causal unidimensional profunda (*depthwise*) antes de las proyecciones recurrentes. Esta convolución, de tamaño de kernel 4, proporciona un sesgo inductivo local que ayuda al modelo a capturar patrones de corto alcance (como bigramas y trigramas a nivel de bytes) antes de la mezcla recurrente global:

$$x'_t = \text{Conv1D}_{\text{causal}}(x_{0:t}, \text{kernel_size} = 4). \quad (4.14)$$

El modelo soporta dos modos de convolución: **compartido** (una convolución sobre los estados ocultos antes de proyectar, que es más eficiente) y **separado** (convoluciones independientes sobre cada flujo proyectado i , f y g , que ofrece mayor capacidad). La convolución corta es particularmente útil en el modelo híbrido (habilitada por defecto) para compensar la ausencia de atención local en las etapas externas.

4.3.7. Kernels de recurrencia

La recurrencia HGRN se implementa mediante dos kernels Triton especializados, optimizados para diferentes regímenes de operación:

Kernel recurrente fusionado (`fused_recurrent_hgrn`): implementa la recurrencia secuencial pura, iterando sobre la dimensión temporal en un solo kernel:

$$h_t = g_t \odot h_{t-1} + x_t, \quad (4.15)$$

donde $g_t = f_t$ (la compuerta de olvido ya transformada por sigmoide). Este kernel paraleliza a través de la dimensión de cabezas y el tamaño de lote, pero procesa el tiempo secuencialmente. Opera en espacio de probabilidad ($g_t \in (0, 1)$), lo que es numéricamente estable para pasos individuales. **Es el kernel utilizado durante la inferencia autorregresiva**, donde la longitud de secuencia es 1, y también para secuencias cortas durante el entrenamiento. Los estados recurrentes se mantienen en registros de SRAM, evitando accesos a HBM.

Kernel por fragmentos (`chunk_hgrn`): implementa un algoritmo de escaneo paralelo que divide la secuencia en fragmentos de tamaño $C = 128$:

$$h_t = \exp(g_t) \odot h_{t-1} + x_t, \quad (4.16)$$

donde g_t está en espacio logarítmico (*log-space*). La razón de esta diferencia es que la multiplicación repetida de probabilidades $g_t \in (0, 1)$ causa *underflow* numérico en secuencias largas; en espacio logarítmico, las multiplicaciones se convierten en sumas ($\exp(a) \cdot \exp(b) = \exp(a + b)$), lo que permite acumular las compuertas de forma estable a través de cientos de posiciones. El algoritmo procesa intra-fragmento de forma secuencial dentro de cada kernel, pero en paralelo a través de fragmentos. Luego, un segundo kernel aplica la corrección entre fragmentos utilizando la acumulación de compuertas logarítmicas. Este enfoque reduce la latencia de $O(L)$ a $O(L/C)$ pasos seriales, acelerando significativamente el entrenamiento en secuencias largas. **Es el kernel utilizado durante el entrenamiento** cuando la longitud de secuencia supera el umbral del fragmento.

La Tabla 4.1 resume las diferencias entre ambos kernels.

Tabla 4.1. Kernels de recurrencia HGRN

Propiedad	<code>fused_recurrent</code>	<code>chunk</code>
Fórmula	$h_t = g_t \cdot h_{t-1} + x_t$	$h_t = \exp(g_t) \cdot h_{t-1} + x_t$
Representación de g	Probabilidad $(0, 1)$	<i>Log-space</i> $\mathbb{R}$
Complejidad serial	$O(L)$	$O(L/C)$
Uso principal	Inferencia	Entrenamiento

4.4. Chunking dinámico multi-etapa

El mecanismo de chunking dinámico constituye la segunda innovación central del modelo, permitiendo el aprendizaje de estrategias de segmentación jerárquica sin dependencia de un tokenizador predefinido.

4.4.1. Fundamento teórico

El chunking dinámico se basa en el principio de que la segmentación óptima de una secuencia es inherentemente contextual y debe ser aprendida a partir de los datos, no impuesta por reglas estadísticas fijas (Hwang et al., 2025). La intuición subyacente es que la incertidumbre predictiva es máxima en las transiciones entre unidades semánticas (caracteres, subpalabras, palabras, frases), y que el modelo debe aprender a identificar estos límites de extremo a extremo.

Conceptualmente, la jerarquía multi-etapa comprime progresivamente la secuencia de entrada. En la etapa más externa, cada posición corresponde a un byte individual. La primera etapa de chunking agrupa bytes en fragmentos que se aproximan a caracteres o sílabas. Las etapas sucesivas agrupan estos fragmentos en unidades cada vez más abstractas (palabras, frases).

4.4.2. Módulo de enrutamiento (*RoutingModuleBit*)

El módulo de enrutamiento decide dinámicamente qué posiciones de la secuencia constituyen límites de fragmentos. Opera en dos modos: **modo lote** (entrenamiento y prefill), donde recibe la secuencia completa de una vez, y **modo paso a paso** (generación autorregresiva), donde recibe un token a la vez manteniendo un estado. Opera en precisión completa (`nn.Linear`, no `BitLinear`) porque las decisiones de límite son sensibles al ruido de cuantización. Las proyecciones se inicializan como matrices identidad para estabilizar el inicio del entrenamiento.

Dado un estado oculto $H \in \mathbb{R}^{B \times L \times d}$ a la salida del codificador de una etapa, el enrutamiento calcula la similitud coseno entre proyecciones de tokens consecutivos:

$$\begin{aligned}
q_t &= \text{normalize}(\text{Linear}_q(H[t])) \in \mathbb{R}^d, \\
k_{t+1} &= \text{normalize}(\text{Linear}_k(H[t+1])) \in \mathbb{R}^d, \\
\cos_sim_t &= \langle q_t, k_{t+1} \rangle \in [-1, 1], \\
p_t &= \text{clamp}\left(\frac{1 - \cos_sim_t}{2}, 0, 1\right).
\end{aligned} \tag{4.17}$$

La probabilidad $p_t \in [0, 1]$ mide la disimilitud semántica entre tokens consecutivos: un valor cercano a 1 indica alta disimilitud, sugiriendo que existe un límite de fragmento entre t y $t + 1$. La primera posición se fuerza como límite ($p_0 = 1$) para garantizar que todos los fragmentos tengan al menos un elemento.

La decisión binaria se obtiene mediante **argmax**:

$$m_t = \mathbb{I}[\text{argmax}([1 - p_t, p_t]) = 1], \tag{4.18}$$

donde $\mathbb{I}$ es la función indicadora. Esta operación es no diferenciable, pero el gradiente fluye a través de p_t mediante el STE en la conexión residual posterior.

Durante la generación paso a paso, el módulo usa un estado (**RoutingModuleState**) con el último estado oculto y un booleano de tokens vistos. La probabilidad de límite se calcula comparando la proyección k del token actual con la proyección q del anterior:

$$p_{\text{step}} = \text{clamp}\left(\frac{1 - \langle q_{\text{prev}}, k_{\text{curr}} \rangle}{2}, 0, 1\right). \tag{4.19}$$

4.4.3. Capa de chunking (*ChunkLayer*)

La capa **ChunkLayer** no tiene parámetros aprendibles; es una operación pura de selección que extrae los tokens en las posiciones marcadas como límites por la máscara m_t .

En modo rellenado (*padded mode*), que es el modo por defecto durante el entrenamiento, la capa utiliza un ordenamiento (**argsort**) para llevar los tokens límite al inicio del tensor:

$$\text{token_idx}_t = t + (-m_t) \cdot L, \quad \text{indices} = \text{argsort}(\text{token_idx}), \tag{4.20}$$

donde L es la longitud original. Este procedimiento asigna claves pequeñas ($< L$) a los tokens límite y claves grandes ($\geq L$) a los no límite, permitiendo recoger los primeros M tokens como entrada para la siguiente etapa, donde $M = \max_i \sum_t m_{i,t}$ es el máximo número de límites entre todas las secuencias del lote. La máscara de salida M_{chunk} se construye truncando la máscara original a las primeras M posiciones y enmascarando como inválidas aquellas que exceden el número real de límites de cada secuencia.

En modo empaquetado (*packed mode*), utilizado para secuencias de longitud variable concatenadas, la selección es directa mediante indexación booleana.

4.4.4. Capa de reconstrucción (*DeChunkLayer*)

La capa **DeChunkLayer** reconstruye la secuencia original a partir de los fragmentos procesados por la etapa interna, utilizando una media móvil exponencial (*Exponential Moving Average*, EMA). La intuición es que la información de cada fragmento debe propagarse suavemente a todas las posiciones que abarca.

El EMA se define como:

$$\text{out}_t = p_t \cdot \text{chunk}_t + (1 - p_t) \cdot \text{out}_{t-1}, \quad (4.21)$$

donde p_t es la probabilidad de límite en la posición t y chunk_t es el vector del fragmento correspondiente. En las posiciones donde $p_t = 1$ (límites), la salida se iguala al valor del fragmento; en las posiciones donde $p_t \approx 0$ (no límites), la salida es aproximadamente el valor del fragmento anterior.

Esta formulación es equivalente a la recurrencia HGRN:

$$h_t = g_t \odot h_{t-1} + x_t, \quad (4.22)$$

con la correspondencia $g_t = 1 - p_t$ y $x_t = p_t \cdot \text{hidden}_t$. Por lo tanto, la reconstrucción puede implementarse eficientemente utilizando el mismo kernel **fused_recurrent_hgrn** que la recurrencia principal, operando sobre los M tokens límite.

La propagación a todas las L posiciones se realiza mediante un índice de retroceso (*plug-back*):

$$\text{plug_back_idx}_t = \text{cumsum}(m, t) - 1 \in \{0, \dots, M - 1\}, \quad (4.23)$$

que mapea cada posición t al fragmento que la contiene, permitiendo una difusión paralela de los valores EMA a través de **torch.gather**.

4.4.5. Pérdida de balanceo de carga (*Load-Balancing Loss*)

Para evitar que el módulo de enrutamiento colapse a soluciones degeneradas (todos los tokens son límites o ningún token es límite), se introduce una pérdida auxiliar de balanceo de carga (Hwang et al., 2025) que se utiliza **exclusivamente durante el entrenamiento**. Dado el factor de submuestreo esperado N (que representa la longitud promedio de fragmento en bytes), la pérdida para una etapa se define como:

$$\mathcal{L}_{\text{LB}} = \frac{N}{N-1} [(1 - \hat{m}) \cdot (1 - \bar{p}) + \hat{m} \cdot \bar{p} \cdot (N - 1)], \quad (4.24)$$

donde $\hat{m} = \frac{1}{L} \sum_t m_t$ es la fracción de límites predichos y $\bar{p} = \frac{1}{L} \sum_t p_t$ es la probabilidad promedio de límite. Esta función tiene un único mínimo cuando la fracción de límites es aproximadamente $1/N$ (es decir, la longitud promedio de fragmento es N).

La pérdida total del modelo en entrenamiento es:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda_{\text{LB}} \sum_s \mathcal{L}_{\text{LB}}^{(s)}, \quad (4.25)$$

donde s indexa las etapas de chunking y λ_{LB} es un hiperparámetro que controla la fuerza del balanceo.

4.5. Estructura de bloques

Los componentes anteriores se organizan en tres tipos de bloques que forman las unidades computacionales básicas del modelo.

4.5.1. Bloque HGRNBitAttention

El bloque `HGRNBitAttention` implementa la recurrencia completa con todos sus componentes auxiliares. El flujo de procesamiento es:

1. **Convolución corta** (opcional): $\tilde{x} = \text{Conv1D}_{\text{causal}}(x)$.
2. **Proyecciones ternarias**: $i = \text{BitLinear}_i(\tilde{x})$, $f = \text{BitLinear}_f(\tilde{x})$, $g = \text{BitLinear}_g(\tilde{x})$.
3. **Compuerta de olvido**: $f = \sigma(f)$.
4. **Compuerta de entrada**: $i = \text{SwiGLU}(i, 1 - f)$.
5. **Recurrencia**: $h = \text{HGRN}(i, f, h_{\text{prev}})$.
6. **Normalización de salida**: $o = \text{FusedRMSNormSwishGate}(g, h)$.
7. **Proyección de salida**: $o = \text{BitLinear}_o(o)$.

Cada una de las proyecciones utiliza la capa `BitLinear` con cuantización ternaria. El bloque mantiene un estado recurrente h_{prev} que se actualiza en cada paso temporal.

4.5.2. Bloque HGRNBitMLP

El bloque `HGRNBitMLP` implementa la mezcla de canales mediante una red feed-forward con activación `SwiGLU`:

$$\begin{aligned} y &= \text{BitLinear}_{\text{gate}}(x), \\ y_{\text{gate}}, y_{\text{hidden}} &= \text{chunk}(y, 2), \\ z &= \text{SwiGLU}(y_{\text{gate}}, y_{\text{hidden}}), \\ o &= \text{BitLinear}_{\text{down}}(z). \end{aligned} \quad (4.26)$$

La dimensión intermedia se calcula como $\frac{2}{3} \cdot d_{\text{hidden}} \cdot \text{hidden_ratio}$, redondeada al múltiplo de 256 más cercano para eficiencia en memoria.

4.5.3. Bloque HGRNBitBlock

Cada bloque `HGRNBitBlock` combina atención recurrente y MLP en una estructura residual de pre-normalización (*pre-norm residual*):

$$\begin{aligned} x_1 &= x_0 + \text{HGRNBitAttention}(\text{RMSNorm}(x_0)), \\ x_2 &= x_1 + \text{HGRNBitMLP}(\text{RMSNorm}(x_1)). \end{aligned} \quad (4.27)$$

La normalización de la segunda subcapa utiliza una versión fusionada de `RMSNorm` con suma residual (`prenorm=True`), que combina la adición residual y la normalización en una sola operación para mejorar la eficiencia.

4.6. Arquitectura jerárquica recursiva

La jerarquía del modelo se construye mediante composición recursiva del módulo `HNetBit`: cada etapa no interna contiene otra instancia de `HNetBit` como su red principal.

4.6.1. Principio de recursividad

Cada etapa s del modelo recibe una secuencia de entrada y la transforma a través de tres fases: codificación, procesamiento interno y decodificación. La etapa más interna implementa solo la fase de procesamiento interno (una pila de bloques `HGRNBitBlock`), mientras que las etapas externas añaden los módulos de chunking.

Formalmente, sea $\mathcal{H}_s$ la etapa s con dimensión d_s . Para una secuencia $H \in \mathbb{R}^{B \times L_s \times d_{s-1}}$ (donde $d_{-1} = d_0$):

$$\mathcal{H}_s(H) = \begin{cases} \text{HGRNBitStack}_s^{\text{inner}}(H) & \text{si } s = S - 1, \\ \text{Decoder}_s(\text{Dechunk}(\mathcal{H}_{s+1}(\text{Chunk}(\text{Encoder}_s(H)))))) & \text{en otro caso.} \end{cases} \quad (4.28)$$

4.6.2. Etapa no interna

Para una etapa s no interna con entrada $H \in \mathbb{R}^{B \times L \times d}$, el flujo de procesamiento completo es:

1. **Acolchonamiento de dimensión:** si $d_s > d_{s-1}$, se concatenan ceros aprendibles para igualar la dimensión.
2. **Codificador:** $H_{\text{enc}} = \text{HGRNBitStack}_{\text{enc}}(H)$ — procesa la secuencia a la resolución actual.
3. **Proyección residual:** $R = \text{Linear}_{\text{res}}(H_{\text{enc}})$ — proyección lineal en FP32 inicializada a cero, siguiendo la práctica de H-Net para estabilizar las conexiones residuales en jerarquías profundas.
4. **Enrutamiento:** $(p, m) = \text{RoutingModuleBit}(H_{\text{enc}})$ — predice límites de fragmentos.

5. **Chunking:** $H_{\text{chunk}} = \text{ChunkLayer}(H_{\text{enc}}, m)$ — selecciona tokens límite.
6. **Procesamiento interno:** $H_{\text{inner}} = \mathcal{H}_{s+1}(H_{\text{chunk}})$ — procesa recursivamente en la siguiente etapa.
7. **Dechunking:** $H_{\text{dechunk}} = \text{DeChunkLayer}(H_{\text{inner}}, m, p)$ — reconstruye la secuencia original mediante EMA.
8. **Conexión residual:** $H_{\text{res}} = H_{\text{dechunk}} \cdot \text{STE}(p) + R$ — combina la salida reconstruida con la proyección residual, donde $\text{STE}(p)$ permite que el gradiente fluya a través de la selección discreta.
9. **Decodificador:** $H_{\text{out}} = \text{HGRNBitStack}_{\text{dec}}(H_{\text{res}})$ — post-procesa la secuencia reconstruida.
10. **Recorte dimensional:** se trunca la salida a la dimensión d_{s-1} de la etapa padre.

La proyección residual R se inicializa a cero y se mantiene en precisión completa (FP32). Esto significa que al inicio del entrenamiento la conexión residual es inactiva, y el modelo aprende gradualmente a utilizarla según sea necesario. La función STE en el factor p permite que el gradiente fluya a través de la operación de selección discreta:

$$\frac{\partial}{\partial p}(H_{\text{dechunk}} \cdot p) = H_{\text{dechunk}}. \quad (4.29)$$

4.6.3. Etapa más interna

La etapa más interna recibe los fragmentos de la etapa anterior y los procesa mediante una pila plana de bloques `HGRNBitBlock`:

$$H_{\text{out}} = \text{HGRNBitStack}_{\text{inner}}(H_{\text{chunk}}). \quad (4.30)$$

Opcionalmente, los bloques HGRN en esta etapa pueden intercalarse con bloques de atención con ventana deslizante (*sliding-window attention*), como se describe en la Sección 4.7.

4.6.4. Pila HGRNBitStack

`HGRNBitStack` es una secuencia de N bloques `HGRNBitBlock` seguida de una normalización RMSNorm final:

$$H_{\text{stack}} = \text{RMSNorm}(\text{HGRNBitBlock}_N(\cdots \text{HGRNBitBlock}_1(H) \cdots)). \quad (4.31)$$

Cada bloque sigue la arquitectura pre-norm residual descrita anteriormente. La pila constituye el componente computacional fundamental que se repite en codificadores, decodificadores y la etapa interna.

4.7. Variante con atención deslizante: Hybrid-Attn

Para estudiar el efecto de reintroducir atención en la arquitectura, se define una variante denominada `hybrid_attn` que reemplaza selectivamente bloques HGRN por atención multi-cabeza con ventana deslizante en la etapa más interna. Esta variante permite evaluar el compromiso entre la eficiencia lineal de la recurrencia y la capacidad expresiva de la atención.

4.7.1. Motivación

Aunque la recurrencia HGRN ofrece complejidad lineal, la autoatención tiene la ventaja teórica de permitir un acceso directo a cualquier posición del contexto, sin compresión a través de un estado oculto. Al introducir atención solo en la etapa más interna (que opera sobre la secuencia más corta después del chunking jerárquico), se minimiza el impacto cuadrático: si la etapa externa comprime la secuencia por un factor N , la etapa interna opera sobre L/N tokens, reduciendo la complejidad de la atención a $O((L/N)^2)$.

4.7.2. Atención causal con ventana deslizante

La capa de atención `CausalMHABit` implementa atención multi-cabeza con ventana deslizante (Beltagy et al., 2020) y embeddings rotatorios RoPE (*Rotary Position Embeddings*) (Su et al., 2024):

$$\begin{aligned}
 Q, K, V &= \text{Linear}_q(x), \text{Linear}_k(x), \text{Linear}_v(x) \in \mathbb{R}^{B \times L \times d}, \\
 Q, K &= \text{RoPE}(Q, K, \text{offset}), \\
 \text{mask}_{i,j} &= \begin{cases} 0 & \text{si } i - w < j \leq i, \\ -\infty & \text{en otro caso,} \end{cases} \\
 A &= \text{Softmax} \left(\frac{QK^\top}{\sqrt{d_{\text{cabeza}}}} + \text{mask} \right), \\
 O &= AV,
 \end{aligned} \tag{4.32}$$

donde w es el tamaño de la ventana deslizante (64 posiciones en la configuración utilizada). La atención con ventana restringe cada token a atender solo a los w tokens anteriores, limitando la complejidad a $O(L \cdot w)$ en lugar de $O(L^2)$.

A diferencia del resto del modelo, las proyecciones Q , K , V y O utilizan precisión completa (`nn.Linear`) en lugar de cuantización ternaria, ya que los puntajes de atención requieren mayor precisión numérica para un cómputo estable (Hwang et al., 2025).

4.7.3. Embeddings rotatorios (RoPE)

RoPE codifica la posición relativa de los tokens mediante una rotación en el espacio de representación:

$$\text{RoPE}(x, m) = x \odot \cos(m\theta) + \text{rotate_half}(x) \odot \sin(m\theta), \tag{4.33}$$

donde m es la posición, $\theta_i = 10000^{-2i/d}$ es la frecuencia base para la dimensión i , y `rotate_half` intercambia y niega las mitades del vector. La caché de cosenos y senos se precalcula para el máximo contexto y se extiende dinámicamente si es necesario.

4.7.4. Patrón de intercalado

Los bloques de atención se intercalan con bloques HGRN siguiendo un patrón configurable. La variante implementada en este trabajo utiliza el patrón "xaxa", que alterna HGRN, atención, HGRN, atención en la etapa más interna. La configuración se especifica mediante el parámetro `attention_layers_pattern`, donde `x` denota un bloque HGRN y `a` denota un bloque de atención deslizante:

$$\text{Etapa interna} = [\text{HGRN}, \text{Attn}, \text{HGRN}, \text{Attn}]. \quad (4.34)$$

Esta variante podría introducir aproximadamente un 8–10% menos de parámetros que la versión puramente recurrente, debido a que las capas de atención utilizan precisión completa sin el factor de escala ternario, y no comparten el estado recurrente entre pasos.

4.8. Caché de inferencia

Durante la generación autorregresiva, el modelo mantiene una estructura de caché recursiva que almacena los estados recurrentes de todos los bloques HGRN y los estados auxiliares del chunking, permitiendo un paso de inferencia en $O(\text{profundidad})$ sin recomputación. **Esta caché es exclusiva de la inferencia autorregresiva:** durante el entrenamiento, el modelo procesa la secuencia completa de una sola vez mediante los kernels paralelos descritos en la Sección 4.3.7, sin necesidad de almacenar estados intermedios entre pasos.

4.8.1. Estructura de la caché

La caché sigue la misma estructura jerárquica que el modelo:

```
HNetBitCache (stage 0):
+-- encoder_cache: HGRNBlockCache
|   +-- layer 0: (conv_state, h_state)
|   +-- layer 1: (conv_state, h_state)
|   +-- ...
+-- routing_state: RoutingModuleState
|   +-- has_seen_tokens: (B,)
|   +-- last_hidden_state: (B, D)
+-- main_network_cache: HNetBitCache (stage 1)
|   +-- encoder_cache: ...
|   +-- decoder_cache: ...
|   +-- main_network_cache: HGRNBlockCache (stage 2, innermost)
+-- dechunk_state: DeChunkState
```

```
|   +-- last_value: (B, D)
+-- decoder_cache: HGRNBlockCache
```

A diferencia del *KV cache* del Transformer, que almacena pares (K, V) para cada capa y requiere $O(L \cdot d)$ memoria por capa, la caché HGRN almacena un estado de tamaño fijo $O(d)$ por capa, independientemente de la longitud de secuencia procesada. Esto permite una huella de memoria constante durante la generación.

4.8.2. Paso de generación (*step mode*)

Durante la generación paso a paso, el modelo recibe un solo token a la vez. El flujo es:

1. El codificador procesa el token y actualiza su caché recurrente.
2. El módulo de enrutamiento compara el token actual con el anterior almacenado en `routing_state` para decidir si es un límite.
3. Si el token **no es límite**, la etapa interna se omite completamente y el decodificador procesa la salida del EMA, que simplemente mantiene el valor anterior del fragmento.
4. Si el token **sí es límite**, se selecciona para procesamiento por la etapa interna. La caché de la etapa interna se particiona (*slice*) para incluir solo los elementos del lote que son límites. Después del procesamiento, los resultados se fusionan (*merge*) de vuelta a la caché principal.

Esta omisión condicional de la etapa interna en los tokens no-límite reduce significativamente el costo de generación, ya que la mayoría de los tokens (típicamente $> 80\%$) no constituyen límites y solo requieren procesamiento superficial.

4.8.3. Reordenamiento para búsqueda de haz

La caché implementa el método `reorder_cache(beam_idx)` que reordena recursivamente todos los estados dependientes del lote cuando se utiliza búsqueda de haz (*beam search*). Cada `HGRNBlockCache` reordena sus estados de capa, y `HNetBitCache` reordena recursivamente sus subcachés.

4.9. Pipeline completo: de bytes a logits

La clase `HNetBitForCausalLM` integra todos los componentes descritos en un modelo de lenguaje causal completo.

4.9.1. Codificación byte-level

La entrada al modelo consiste en identificadores de bytes (valores enteros en $\{0, \dots, 255\}$). El tokenizador de bytes (`ByteTokenizer`) añade un token BOS (*Beginning of Sequence*) con valor 254 al inicio de cada secuencia y un token EOS (*End of Sequence*) con valor 255 al final. La tabla de *embeddings* es una capa `nn.Embedding(256, d_0)` que mapea cada byte

a un vector denso de dimensión d_0 , donde d_0 es la dimensión del modelo en la etapa más externa.

4.9.2. Forward completo

El paso forward completo para una secuencia de entrada $X \in \mathbb{Z}^{B \times L}$ (identificadores de bytes) es:

$$\begin{aligned} H_0 &= \text{Embedding}(X) && \in \mathbb{R}^{B \times L \times d_0}, \\ H_{\text{out}}, \mathcal{R} &= \text{HNetBit}(H_0, \text{stage} = 0) && \in \mathbb{R}^{B \times L \times d_0}, \\ \text{logits} &= \text{BitLinear}_{\text{LM}}(H_{\text{out}}) && \in \mathbb{R}^{B \times L \times 256}. \end{aligned} \tag{4.35}$$

donde $\mathcal{R}$ es la lista de salidas del módulo de enrutamiento de todas las etapas, utilizada para el cómputo de la pérdida de balanceo de carga.

4.9.3. Cabecera de lenguaje (*LM head*)

La cabecera de lenguaje es una capa **BitLinear** que proyecta los estados ocultos de la etapa más externa al espacio de vocabulario de 256 bytes. Al igual que todas las demás proyecciones, utiliza cuantización ternaria con STE. No se comparten los pesos con la tabla de *embeddings* (`tie_word_embeddings=False`), ya que la cuantización ternaria de la cabecera y la precisión completa de los *embeddings* tienen diferentes propiedades numéricas.

4.9.4. Cálculo de pérdida

La pérdida de entrenamiento se calcula como entropía cruzada con desplazamiento para predicción del siguiente token:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{L-1} \sum_{t=1}^{L-1} \log \frac{e^{\text{logits}_{t-1, x_t}}}{\sum_{v=1}^{256} e^{\text{logits}_{t-1, v}}}, \tag{4.36}$$

donde x_t es el byte objetivo en la posición t . Solo se computa pérdida en las posiciones válidas (no relleno). La pérdida total es $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{LB}} \mathcal{L}_{\text{LB}}$.

4.9.5. Componentes por régimen de operación

La Tabla 4.2 consolida la pertenencia de cada componente arquitectónico a los regímenes de entrenamiento, inferencia autorregresiva, o ambos. Los componentes que están presentes en ambos ejecutan el mismo paso forward en los dos regímenes, aunque ciertos sub-mecanismos internos (como el STE) solo tengan efecto en uno de ellos.

Tabla 4.2. Componentes arquitectónicos por régimen de operación

Componente	Entrenamiento	Inferencia
ByteTokenizer + Embedding	Sí	Sí
HNetBit (jerarquía recursiva)	Sí	Sí
HGRNBitStack (encoder/decoder/inner)	Sí	Sí
HGRNBitBlock (atención + MLP)	Sí	Sí
HGRNBitAttention (proyecciones i,f,g,o, recurrencia, SwiGLU)	Sí	Sí
HGRNBitMLP (expansión + SwiGLU + contracción)	Sí	Sí
BitLinear / FusedBitLinear (RMSNorm + cuantización + lineal)	Sí	Sí
RMSNorm, FusedRMSNormSwishGate	Sí	Sí
SwiGLU	Sí	Sí
Conv1D causal (corta)	Sí	Sí
RoutingModuleBit (cálculo de p, m)	Sí (lote)	Sí (paso a paso)
ChunkLayer (argsort)	Sí	Sí
DeChunkLayer (EMA + plug-back)	Sí	Sí
CausalMHABit + RoPE (solo hybrid_attn)	Sí	Sí
BitLinear_LM (cabecera de lenguaje)	Sí	Sí
STE (mecanismo detach en BitLinear)	Sí (backprop)	No (torch.no_grad)
$\mathcal{L}_{CE}$ (entropía cruzada)	Sí	No
$\mathcal{L}_{LB}$ (balanceo de carga)	Sí	No
Kernel <code>chunk_hgrn</code> (log-space, $C=128$)	Sí (secuencias largas)	No
<code>weight_quant</code> (re-cuantización por paso)	Sí (cada forward)	No (pesos congelados en despliegue)
Kernel <code>fused_recurrent_hgrn</code> (probability-space)	No ^a	Sí
HNetBitCache (y subcachés)	No	Sí
RoutingModuleState	No	Sí
DeChunkState	No	Sí
Omisión condicional de etapa interna	No	Sí
<code>reorder_cache</code> (beam search)	No	Sí

^a El kernel `fused_recurrent_hgrn` también se usa en entrenamiento para secuencias muy cortas donde la sobrecarga de fragmentación del kernel `chunk_hgrn` no se justifica. Sin embargo, su uso principal y para lo que está optimizado es la inferencia autorregresiva.

Como resume la tabla, el esqueleto computacional del modelo (HNetBit, HGRNBitStack,

BitLinear, módulos de chunking) es compartido. Las diferencias entre regímenes se concentran en tres aspectos: (1) la presencia o ausencia de retropropagación (STE, pérdidas), (2) el kernel de recurrencia utilizado (paralelo en log-space para entrenamiento vs. secuencial en probability-space para inferencia), y (3) la infraestructura de caché para mantener el estado recurrente entre pasos autorregresivos.

Capítulo 5

Implementación y optimizaciones

Este capítulo describe los aspectos prácticos de la implementación del modelo HNetBit y la infraestructura de entrenamiento para los benchmarks, incluyendo las decisiones de ingeniería de software, las optimizaciones de bajo nivel mediante kernels Triton y las técnicas de gestión de memoria que permiten el entrenamiento eficiente en una sola GPU A100 de 80 GB.

5.1. Organización del código y arquitectura de software

El proyecto se organiza en una jerarquía de módulos con responsabilidades claramente separadas:

- `hnet_bit/models/`: clases fundamentales del modelo: `HNetBitConfig`, `HNetBit` (módulo jerárquico), `HGRNBitStack` (pila de bloques) y `HNetBitForCausalLM` (modelo completo con `HuggingFace`).
- `hnet_bit/layers/`: bloques `HGRNBitAttention`, `HGRNBitMLP`, `HGRNBitBlock` y `CausalMHABit`.
- `hnet_bit/ops/`: operaciones de bajo nivel: `BitLinear`, `FusedBitLinear`, `RMSNorm`, `RoutingModuleBit`, `ChunkLayer`, `DeChunkLayer`, activaciones y kernels Triton para HGRN.
- `hnet_bit/training/`: infraestructura: `Trainer`, `TrainingLogger`, `TrainingConfig`, optimizador y scheduler.
- `hnet_bit/utils/`: utilidades: `ByteTokenizer` (UTF-8), `HNetBitCache` (caché jerárquica), helpers.

La orquestación de los experimentos de benchmark se realiza mediante dos módulos externos a `hnet_bit/`:

- `model_factory.py`: función `build_model(nombre, tamaño)` que construye cualquiera de las cuatro arquitecturas en los tres tamaños, retornando el modelo, modo byte-level y tamaño del vocabulario.

- `train_spanish.py`: script unificado que carga la configuración, construye modelo y dataloaders, e implementa el bucle de entrenamiento con checkpointing, logging y evaluación.
- `export_deployment.py`: convierte un checkpoint en archivo de despliegue compacto con pesos ternarios empaquetados.
- `profile_inference.py`: mide latencia de prefill y throughput de decodificación.

5.2. Infraestructura de entrenamiento

El entrenamiento se realiza mediante la clase `SpanishTrainer` en `train_spanish.py`, que implementa un bucle de entrenamiento personalizado con soporte para acumulación de gradientes, precisión mixta, pérdida de balanceo de carga y tracking de bytes procesados.

5.2.1. Bucle principal de entrenamiento

El bucle de entrenamiento sigue la siguiente secuencia por paso de optimizador:

1. **Micro-batch**: se itera sobre N_{accum} micro-lotes, donde $N_{\text{accum}} = 8$ es el número de pasos de acumulación de gradientes. Para cada micro-lote:
 - a) Se transfieren los datos a GPU.
 - b) Con `autocast('cuda', dtype=bfloat16)`: forward del modelo, cálculo de $\mathcal{L}_{\text{CE}}$, y (para modelos híbridos) $\mathcal{L}_{\text{LB}}$.
 - c) $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda_{\text{LB}} \cdot \mathcal{L}_{\text{LB}}$.
 - d) $\text{loss} = \mathcal{L}_{\text{total}} / N_{\text{accum}}$.
 - e) `scaler.scale(loss).backward()`.
2. **Optimización** (cada N_{accum} micro-lotes):
 - a) `scaler.unscale_(optimizer)`.
 - b) `clip_grad_norm_(params, max_norm=1.0)`.
 - c) `scaler.step(optimizer)` y `scaler.update()`.
 - d) `optimizer.zero_grad(set_to_none=True)`.
 - e) `scheduler.step()`.
 - f) `global_step += 1`.
3. **Tracking**: se incrementa `bytes_seen` en `bytes_per_step` (131 072 para modelos byte-level, $\approx 129\,400$ para transformer).
4. **Logging**: cada 10 pasos se registran `loss`, `lr`, `grad_norm`, `tok_per_sec` y `peak_mem_mb`.

5. **Evaluación:** cada 1000 pasos se computa BPB en validación.

5.2.2. Precisión mixta

Se utiliza precisión mixta bf16 para todas las operaciones. Las activaciones y gradientes se almacenan en bf16, mientras que los pesos completos (antes de cuantización ternaria) se mantienen en FP32 para preservar la precisión de las actualizaciones del optimizador. El `GradScaler` se habilita únicamente cuando se utiliza fp16 como respaldo; con bf16, `scaler.scale()` es una operación identidad que no afecta el cómputo.

5.2.3. Pérdida de balanceo de carga

Para los modelos HNetBit (`hybrid` y `hybrid_attn`), se añade una pérdida auxiliar de balanceo de carga (ver Sección 4.4.5 del Capítulo 4) que penaliza las soluciones degeneradas del módulo de enrutamiento. En la implementación, para cada etapa con enrutamiento se extraen la máscara de límites m y las probabilidades p de la salida del `RoutingModuleBit`, y se computa $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + 0,01 \cdot \sum_s \mathcal{L}_{\text{LB}}^{(s)}$, con factor de submuestreo esperado $N = 5,0$ (longitud promedio de palabra en español).

5.2.4. Tracking de compresión por etapa

Durante el entrenamiento de modelos híbridos, se registra la tasa de compresión de cada etapa de chunking:

$$\text{compression_ratio}_s = \frac{\text{tokens de entrada}}{\text{tokens de salida}} = \frac{1}{\hat{m}_s}, \quad (5.1)$$

donde $\hat{m}_s$ es la fracción de límites predichos en la etapa s . La compresión global del modelo (producto de las compresiones de todas las etapas) se almacena en `training_stats.json` como un indicador de la eficiencia de segmentación aprendida.

5.3. Sistema de checkpointing y resume

El sistema de checkpointing está diseñado para permitir la recuperación completa del estado de entrenamiento, incluyendo el modelo, el optimizador, el scheduler y los metadatos de progreso.

5.3.1. Contenido del checkpoint

Cada checkpoint contiene las siguientes claves:

```
{  
    "model_state_dict":      # Pesos del modelo  
    "optimizer_state_dict":  # Estado de AdamW  
    "scheduler_state_dict":  # Estado del scheduler WSD
```

```

"scaler_state_dict":      # Estado del GradScaler
"global_step":            # Paso global de optimización
"bytes_seen":             # Total de bytes procesados
"best_val_bpb":           # Mejor BPB de validación hasta el momento
"milestones_passed":      # Lista de hitos de bytes ya alcanzados
"config":                 # Configuración completa de entrenamiento
}

```

Cada archivo ocupa aproximadamente 2.2 GB para el modelo de 150M, dominado por los pesos del modelo y los momentos del optimizador AdamW (que almacena dos momentos por parámetro en FP32).

5.3.2. Jerarquía de checkpoints

Durante el entrenamiento se generan cuatro tipos de checkpoints:

- **Periódicos:** cada 5 000 pasos de optimizador se guarda `checkpoint_step_{N}.pt`. Estos archivos se eliminan automáticamente al finalizar el entrenamiento para conservar espacio en disco.
- **Hitos por bytes:** al alcanzar cada hito (6.25B, 12.5B, 18.75B, 25B) se fuerza evaluación de validación y se guarda `checkpoint_milestone_{N.N}B.pt`. Esto permite reconstruir curvas de aprendizaje si el entrenamiento se interrumpe.
- **Mejor validación:** cuando el BPB de validación alcanza un nuevo mínimo, se sobrescribe `checkpoint_best.pt`.
- **Final:** al completar el presupuesto de 25B bytes, se guarda `checkpoint_final.pt`.

5.3.3. Mecanismo de resume

El resume desde un checkpoint restaura completamente el estado de entrenamiento:

1. Se restauran los estados del modelo, optimizador, scheduler y `scaler_state_dict` desde el checkpoint.
2. Se restauran `global_step`, `bytes_seen` y `best_val_bpb`.
3. Se reconstruye el conjunto de hitos pasados: si el checkpoint contiene `milestones_passed` se usa directamente; en caso contrario se infiere comparando `bytes_seen` con los umbrales.
4. Se leen los archivos CSV existentes (`training_steps_log.csv` y `validation_log.csv`) para reconstruir las listas internas de logging, de modo que las nuevas entradas se añadan sin duplicar las anteriores.

5.4. Sistema de logging

El sistema de logging registra el progreso del entrenamiento en tres formatos complementarios: CSV para análisis posterior, consola para monitoreo en vivo y JSON para métricas finales agregadas.

5.4.1. Training steps log

Cada 10 pasos de optimizador se añade una fila a `training_steps_log.csv` con las siguientes columnas:

- **step**: número de paso de optimizador.
- **bytes_seen**: bytes totales de texto procesados hasta el momento.
- **loss**: pérdida de entropía cruzada del micro-lote actual.
- **lr**: tasa de aprendizaje efectiva.
- **grad_norm**: norma L2 de los gradientes después de clipping.
- **tok_per_sec**: rendimiento en bytes por segundo (incluyendo el overhead de forward + backward).
- **peak_mem_mb**: máximo de memoria GPU asignada durante el paso.
- Para modelos híbridos: **stage_0_compression_ratio**, **stage_1_compression_ratio** y **overall_compression_ratio**.

5.4.2. Validation log

Cada 1 000 pasos, y en cada hito de bytes, se ejecuta una evaluación completa sobre 50 lotes de validación (~0.8 MB de texto) y se registra en `validation_log.csv`:

- **step**: paso de optimizador.
- **bytes_seen**: bytes procesados.
- **val_loss**: pérdida media de entropía cruzada en el conjunto de validación.
- **val_bpb**: bits por byte, calculado como $\text{val_loss} / \ln 2$ para modelos byte-level.

La evaluación final al término del entrenamiento utiliza 200 lotes (~3.3 MB) para una medición más precisa.

5.4.3. Training stats JSON

Al finalizar el entrenamiento, se escribe `training_stats.json` con un resumen de las métricas agregadas:

```
{  
  "training_time_seconds": 69300,  
  ...  
}
```

```

"training_time_hours":      19.25,
"total_steps":              190735,
"bytes_seen":               25000000000,
"param_count":              138000000,
"non_embedding_param_count": 138000000,
"disk_size_mb":             263.9,
"peak_training_memory_mb":  15500,
"peak_reserved_memory_mb":  16000,
"overall_compression_ratio": 0.31
}

```

5.5. Kernels Triton y gestión de memoria GPU

Para maximizar la eficiencia en GPU, las operaciones más costosas se implementan mediante kernels Triton personalizados que minimizan los accesos a memoria HBM mediante fusiones de operaciones y gestión cuidadosa de SRAM. Los kernels `FusedBitLinear`, `fused_recurrent_hgrn`, `chunk_hgrn`, `FusedRMSNormSwishGate` y `SwiGLU` se describen en detalle en las Secciones 4.2.5, 4.3.7, 4.3.5 y 4.3.4 del Capítulo 4, respectivamente.

5.5.1. Detección de Triton y fallbacks

Todos los kernels detectan automáticamente la disponibilidad de Triton mediante una sonda de compilación en tiempo de ejecución. Si Triton no está disponible (por ejemplo, en CPU, en contenedores Docker sin privilegios, o cuando se establece la variable de entorno `HNETBIT_DISABLE_TRITON=1`), se utilizan implementaciones de respaldo en PyTorch puro:

- `FusedBitLinear` cae a `BitLinear.forward()` estándar con bucles de normalización y cuantización separados.
- `fused_recurrent_hgrn` cae a un bucle `for` en Python sobre la dimensión temporal.
- `chunk_hgrn` cae a `_naive_chunk_hgrn`, que implementa la misma recurrencia $\exp(g)$ mediante un bucle `for` sobre t .

Estos fallbacks son entre 3 y 10 veces más lentos que las versiones Triton, pero permiten la ejecución en entornos sin soporte de compilación JIT.

5.6. Optimizaciones de memoria para entrenamiento

El modelo híbrido presenta desafíos particulares de memoria debido a que su paso forward no está instrumentado para gradient checkpointing. A continuación se describen las técnicas implementadas para reducir el consumo de VRAM.

5.6.1. Ausencia de gradient checkpointing

El gradient checkpointing (o recomputación de activaciones) es una técnica que intercambia cómputo por memoria: durante el forward, solo se almacenan las activaciones de ciertos puntos de control; durante el backward, las activaciones intermedias se recalculan a partir de estos puntos. Esta técnica funciona mediante la inserción de puntos de control en el grafo computacional de PyTorch.

En el modelo híbrido, la función `enable_gradient_checkpointing()` establece la bandera `_gradient_checkpointing = True` en los módulos de `HNetBit`, pero la implementación del paso forward en `HNetBit.forward()` y `step()` nunca lee esta bandera. Por lo tanto, todas las activaciones intermedias de la jerarquía completa (codificadores, decodificadores, módulos de chunking, etapas recursivas) se retienen en memoria hasta completar el backward.

Las consecuencias son directas: el modelo híbrido de 150M consume aproximadamente 15.5 GB de VRAM durante el entrenamiento, mientras que el transformer del mismo tamaño ocupa entre 6 y 8 GB gracias a su gradient checkpointing funcional. Para el modelo de 350M, el consumo alcanza aproximadamente 27 GB.

5.6.2. Estrategias de mitigación

Para compensar la ausencia de gradient checkpointing, se implementaron varias técnicas:

- **Precisión mixta bf16:** las activaciones y los gradientes se almacenan en bf16 (2 bytes por elemento), reduciendo el consumo a la mitad en comparación con FP32. Los pesos completos se mantienen en FP32 para preservar la precisión de las actualizaciones; tras la cuantización ternaria, los pesos efectivos son $\{-1/\alpha, 0, +1/\alpha\}$, pero la variable `weight` subyacente permanece en FP32 para el optimizador.
- **Reglas de weight decay:** el optimizador AdamW no almacena momentos (estados de primer y segundo orden) para ciertos parámetros, reduciendo el consumo de memoria del optimizador. Los parámetros excluidos del weight decay son: sesgos, parámetros de normalización (RMSNorm), proyecciones de acolchonamiento de dimensión (`pad_dimension`), tablas de *embeddings* y cualquier parámetro con dimensionalidad < 2 (como escalares). Para estos, los momentos de AdamW se almacenan pero el decaimiento de peso no se aplica.
- **Liberación manual de buffers:** después de completar el procesamiento de cada etapa (chunking, dechunking), se eliminan explícitamente los tensores intermedios mediante `del`, seguido de `torch.cuda.empty_cache()` en puntos estratégicos del código. Esto permite que el asignador de memoria de CUDA reutilice los bloques liberados para las operaciones siguientes.
- **Descomposición de lotes:** cuando la memoria es insuficiente para el tamaño de lote deseado, se reduce `batch_size` y se aumenta `gradient_accumulation_steps` proporcionalmente para mantener el mismo tamaño de lote efectivo. Por ejemplo, si el lote efectivo es 32 con 4×8 (`batch_size=4`, `acumulación=8`), se puede reconfigurar a 2×16 .

5.6.3. Consumo de memoria del optimizador

El optimizador AdamW almacena dos momentos por parámetro en FP32, lo que representa 8 bytes por parámetro (2×4 bytes). Para un modelo de 150M parámetros, solo los momentos del optimizador ocupan aproximadamente 1.1 GB adicionales a los 0.55 GB de los pesos en FP32. Este costo es independiente de la cuantización ternaria, ya que los momentos se mantienen sobre los pesos completos.

5.7. Sistema de caché para generación

Durante la inferencia autorregresiva, el modelo utiliza una estructura de caché jerárquica que evita la recomputación de estados recurrentes en cada paso (ver Sección 4.8 del Capítulo 4 para la descripción arquitectónica de HNetBitCache y el modo paso a paso).

5.7.1. HGRNBlockCache

Cada bloque HGRN almacena su estado recurrente en una tupla dentro de HGRNBlockCache. Según la convolución corta, la tupla contiene:

- Sin convolución: (`recurrent_state`,) — tensor (B, H, D_{head}) .
- Con convolución compartida: (`conv_state`, `recurrent_state`) — donde `conv_state` es $(B, d_{\text{model}}, \text{kernel_size})$.
- Con convolución separada: (`conv_state_i`, `conv_state_f`, `recurrent_state`) — tres estados de convolución independientes.

La caché soporta actualización in-place cuando las dimensiones coinciden (evitando realocaciones) y reemplazo cuando cambia el tamaño del lote.

5.7.2. Manejo de continuidad de batch en step mode

Durante la generación paso a paso, el módulo de chunking selecciona solo los elementos del lote que constituyen límites de fragmento para procesamiento por la etapa interna. Esto rompe la continuidad del lote: si en un paso dado solo 3 de 8 elementos son límites, la etapa interna debe procesar un lote de tamaño 3.

Para manejar esto, los métodos `_slice_hnet_cache` y `_merge_hnet_cache` en HNetBit recorren recursivamente todos los niveles de la caché, indexando los estados de cada capa mediante la máscara de límites:

```
def _slice_hgrn_cache(cache, mask):
    sub = HGRNBlockCache()
    for state in cache.states:
        if state is not None:
            sub.states.append(tuple(s[mask] for s in state))
        else:
            sub.states.append(None)
```

```

return sub

def _merge_hgrn_cache(cache, mask, sub):
    for i, state in enumerate(cache.states):
        if state is not None:
            cache.states[i] = tuple(
                torch.where(mask.unsqueeze(-1), s, orig_s)
                for s, orig_s in zip(sub.states[i], state)
            )

```

Este proceso se aplica recursivamente a las cachés de todas las etapas, garantizando que los estados de los elementos no-límite se preserven sin modificación.

5.7.3. Soporte para búsqueda de haz

Cuando se utiliza búsqueda de haz (*beam search*), la caché debe reordenarse para alinearse con las nuevas hipótesis. El método `reorder_cache(beam_idx)` recorre recursivamente la jerarquía de `HNetBitCache` y aplica `torch.index_select(beam_idx)` a todos los tensores de estado dependientes del lote: estados recurrentes de `HGRNBlockCache`, estados de enrutamiento y estados de dechunking.

5.8. Exportación para despliegue

Una vez entrenado, el modelo puede exportarse a un formato compacto que elimina los metadatos de entrenamiento y empaqueta los pesos ternarios a 2 bits por parámetro.

5.8.1. Proceso de exportación

La función `export_model()` en `export_deployment.py` sigue el siguiente flujo:

1. **Rebuild:** se reconstruye el modelo desde cero utilizando los parámetros de configuración almacenados en el checkpoint (`model_name` y `model_size`).
2. **Carga:** se cargan los pesos desde el `model_state_dict` del checkpoint.
3. **Congelación ternaria:** se itera sobre todos los módulos `BitLinear` y `FusedBitLinear` del modelo. Para cada uno, se aplica `weight_quant()` de forma determinista y se reemplaza el tensor de pesos en el lugar, sin STE:

$$\tilde{W} = \frac{\text{round_clamp}(W \cdot \alpha, -1, 1)}{\alpha}. \quad (5.2)$$

Después de este paso, los pesos son permanentemente ternarios y el modelo no puede seguir entrenándose.

4. **Verificación roundtrip:** se desempaquetan y recargan los pesos, se ejecuta un paso forward con una entrada sintética y se verifica que la diferencia máxima en logits sea menor a 1×10^{-5} .

5. **Empaquetado:** los pesos ternarios se empaquetan mediante `pack_ternary_tensor()`: se extrae la escala α , se codifican los tres valores $\{-1, 0, +1\}$ como $\{0, 1, 2\}$, se empaquetan 4 valores por byte en un arreglo `uint8` (2 bits por parámetro), y se almacena la escala como `float32`.
6. **Guardado en dos pasadas:** primero se guarda el archivo para medir su tamaño exacto, luego se guarda de nuevo con las estadísticas de compresión incrustadas en el nombre del archivo.

5.8.2. Ratios de compresión

El empaquetado ternario reduce significativamente el tamaño en disco. Para un modelo con P parámetros ternarios, el tamaño del archivo exportado es aproximadamente:

$$\text{Tamaño}_{\text{deploy}} \approx \frac{2P}{8} + \frac{P}{K} + \text{overhead}, \quad (5.3)$$

donde P es el número de parámetros ternarios, 2 bits por parámetro para los valores, K es la frecuencia de escalas (una por tensor) y el overhead incluye los pesos no ternarios (embeddings, normalización) almacenados en FP32. La compresión frente a FP16 es:

$$\text{Compresión} = \frac{16P}{\text{Tamaño}_{\text{deploy}} \cdot 8} \approx 7-8 \times . \quad (5.4)$$

Para el modelo híbrido de 150M, el archivo de despliegue ocupa aproximadamente 37.9 MB (2.29 bits/parámetro), frente a los 263.9 MB que ocuparían los pesos en FP16.

5.9. Integración con Hugging Face

`HNetBitForCausalLM` hereda de `PreTrainedModel` y `GenerationMixin`, permitiendo usar utilidades estándar de Hugging Face como `model.generate()` y `pipeline`.

5.9.1. Manejo de caché en generación

El método `_prepare_cache_for_generation()` sobrescribe el comportamiento por defecto de Hugging Face, que utiliza `DynamicCache` (basado en pares KV). En su lugar, crea una instancia de `HNetBitCache` mediante `backbone.allocate_inference_cache()`, que construye recursivamente la estructura jerárquica de caché con estados inicializados a cero.

El método `prepare_inputs_for_generation()` detecta la presencia de `HNetBitCache` y, si la caché ya ha procesado tokens anteriores, recorta `input_ids` al último token solamente. Esto evita reprocesar toda la secuencia en cada paso de generación.

5.9.2. Manejo de tamaño de lote variable

Durante la fase de prefill de Hugging Face, el tamaño de lote puede variar entre llamadas (por ejemplo, cuando el pipeline cambia de configuración). El forward de `HNetBitForCausalLM` detecta esta situación comparando las dimensiones de la caché existente con el lote actual, y reasigna una nueva caché si es necesario.

5.9.3. Compatibilidad con beam search

La caché implementa `reorder_cache(beam_idx)`, llamado por `GenerationMixin` durante la búsqueda de haz. Recorre recursivamente la jerarquía aplicando `index_select` a los tensores de estado, manteniendo las hipótesis alineadas.

Capítulo 6

Modelos y método experimental

En este capítulo se describe la metodología experimental del trabajo: el conjunto de datos, las arquitecturas evaluadas y sus configuraciones, el protocolo de entrenamiento común a todos los modelos y las métricas utilizadas para medir calidad y eficiencia. Todas las decisiones metodológicas buscaron garantizar una comparación justa entre arquitecturas con esquemas de tokenización distintos (BPE frente a byte-level) y representaciones de pesos distintas (punto flotante frente a ternaria).

6.1. Conjunto de datos

El conjunto de datos utilizado corresponde al *Spanish Billion Words*, disponible en HuggingFace bajo el identificador `jhonparra18/spanish_billion_words_clean`. El dataset contiene 46.9 millones de documentos; la descarga comprimida (parquet) ocupa ~ 4.7 GB y el texto plano UTF-8 resultante suma ~ 8.7 GB, proveniente de fuentes diversas (noticias, artículos académicos, páginas web). El presupuesto de entrenamiento de 25 000 millones de bytes (Sección 6.3) supera el tamaño del corpus, por lo que el `DataLoader` se recicla al agotarse y cada ejecución completa ~ 3 pasadas sobre la partición de entrenamiento (el 95 % del corpus).

Para garantizar la reproducibilidad, el corpus se dividió en entrenamiento (95 %) y validación (5 %) de manera contigua, tomando el último 5 % del archivo como conjunto de validación. No se aplicó mezcla (*shuffle*) para preservar la coherencia del orden original y evitar fugas de información entre los conjuntos.

6.1.1. Procesamiento según el esquema de tokenización

El procesamiento de los datos varía según la arquitectura:

- **Modelos byte-level** (`matmulfree`, `hybrid`): se utilizó directamente la codificación UTF-8 del texto, resultando en un vocabulario fijo de 256 símbolos. La longitud de secuencia se fijó en 4096 bytes.
- **Modelo BPE** (`transformer`): se empleó el tokenizador Byte-Pair Encoding (BPE)

de GPT-2, con un vocabulario de 50 257 tokens. La longitud de secuencia se fijó en 1280 tokens.

La tokenización BPE del corpus completo produjo un promedio medido de 3.16 bytes por token. Este valor es central para la metodología, ya que se utiliza (a) en la fórmula de presupuesto de bytes por paso de entrenamiento y (b) en la conversión de la pérdida del transformer a bits por byte. Nótese que $1280 \text{ tokens} \times 3.16 \text{ bytes/token} \approx 4043 \text{ bytes}$, es decir, la cantidad de texto subyacente que ve el transformer por muestra es prácticamente la misma que los 4096 bytes de los modelos byte-level; los contextos son casi simétricos.

6.1.2. Construcción de los datasets

La construcción de los datasets se realizó una sola vez y se almacenó en archivos mapeados en memoria (`data/spanish/`) para acelerar las cargas posteriores. Para los modelos byte-level se escribió el corpus como un único arreglo de bytes (`corpus.bin`). Para el transformer se generó además el arreglo de identificadores BPE (`corpus_bpe.npy`).

La tokenización BPE de un corpus de este tamaño requiere decenas de GB de memoria RAM intermedia. Para controlar el consumo, el proceso se ejecutó en procesos hijos independientes del intérprete principal, reciclando los trabajadores cada 50 fragmentos del corpus; de esta forma, la memoria del proceso padre se mantiene estable (del orden de 5–10 GB) y la de los hijos se libera al reciclarlos.

6.2. Arquitecturas implementadas

Se entrenaron tres arquitecturas, todas decodificadoras autorregresivas y con el mismo presupuesto de entrenamiento de 25 000 millones de bytes:

- **Transformer (línea base)**: modelo tipo Llama con tokenización BPE y pesos en FP32.
- **MatMul-Free plano (línea base recurrente)**: arquitectura recurrente HGRN con pesos ternarios, a nivel de bytes, sin jerarquía.
- **HNetBit (modelo propuesto)**: arquitectura jerárquica que combina pesos ternarios, recurrencia HGRN y chunking dinámico multi-etapa, descrita en detalle en el Capítulo 4.

Adicionalmente, se diseñó e implementó una cuarta variante, `hybrid_attn`, que intercala atención con ventana deslizante en la etapa más interna de HNetBit (Sección 4.7 del Capítulo 4). Esta variante se describe en el capítulo de arquitectura por su interés conceptual, pero **no se incluyó en las pruebas experimentales**.

6.2.1. Transformer (línea base)

Modelo estándar de tipo Llama, con atención multi-cabeza causal, normalización RMSNorm, activación SwiGLU y posiciones máximas de 2048. Se utilizó la implementación de Hugging

Face **transformers** con pesos en precisión completa (FP32). Su tokenizador BPE (GPT-2) tiene un vocabulario de 50 257 tokens, lo que implica una sobrecarga significativa de parámetros en las tablas de *embeddings* y en la cabecera de lenguaje.

6.2.2. MatMul-Free plano (línea base recurrente)

Implementación basada en el trabajo de Zhu et al. (2024), que utiliza el bloque recurrente HGRN (*Hierarchically Gated Recurrent Network*) como mezclador de tokens y capas lineales con pesos ternarios (BitLinear) para la mezcla de canales. Al igual que el modelo híbrido, opera a nivel de bytes con vocabulario de 256 símbolos. Su arquitectura es plana (sin jerarquía de chunking) y sirve como referencia para medir la contribución específica del *dynamic chunking* sobre una base MatMul-Free.

6.2.3. Modelo híbrido propuesto (HNetBit)

La arquitectura principal combina pesos ternarios, recurrencia HGRN y chunking dinámico multi-etapa, y se describe en detalle en el Capítulo 4. En resumen:

- **Pesos ternarios:** todas las proyecciones lineales (excepto la tabla de *embeddings* y la proyección residual de cada etapa) utilizan cuantización ternaria $\{-1, 0, +1\}$ mediante el estimador de paso directo (STE).
- **Recurrencia HGRN:** sustituye a la atención en todos los bloques, con complejidad $O(L)$ en entrenamiento y estado de tamaño fijo $O(d)$ en inferencia.
- **Chunking dinámico multi-etapa:** la secuencia de bytes se comprime recursivamente mediante un módulo de enrutamiento que aprende límites de fragmento basados en similitud coseno entre representaciones consecutivas. Cada etapa consta de un codificador, un módulo de enrutamiento, una red interna (que puede ser otra etapa) y un decodificador con reconstrucción suave mediante media móvil exponencial.

La variante `hybrid_attn` comparte esta arquitectura pero intercala bloques de atención multi-cabeza con ventana deslizante (tamaño 64, con embeddings rotatorios RoPE) en la etapa más interna, siguiendo el patrón "xaxa" (HGRN, atención, HGRN, atención).

6.2.4. Configuraciones y conteo de parámetros

La Tabla 6.1 resume las configuraciones de las tres arquitecturas en los dos tamaños evaluados. La notación de bloques del híbrido sigue la estructura `[pre, medio, post]` por etapa; por ejemplo, `[4,0,4] + [10]` indica una etapa externa con 4 bloques de codificador y 4 de decodificador, y una etapa interna con 10 bloques.

Tabla 6.1. Configuraciones de las arquitecturas evaluadas

Modelo	Tamaño	Capas / Bloques	d_{model}	Cabezas	Vocab.	Params tot. (M)	LR
Transformer	150M	12	768	12	BPE (50 257)	190.5	3×10^{-4}
	350M	24	1024	16	BPE (50 257)	505.6	3×10^{-4}
MatMul-Free	150M	16	768	1	Byte (256)	113.8	4×10^{-3}
	350M	24	1024	1	Byte (256)	309.1	2.5×10^{-3}
HNetBit	150M	$[4, 0, 4] + [10]$	$[576, 768]$	4	Byte (256)	138.3	4×10^{-3}
	350M	$[4, 0, 4] \times 2 + [12]$	$[640, 896, 1152]$	4	Byte (256)	419.4	2.5×10^{-3}

Además de los valores de la tabla, las configuraciones incluyen los siguientes detalles:

- **Transformer:** capas FFN intermedias de 3072 (150M) y 4096 (350M) unidades, posiciones máximas 2048, *embeddings* no compartidos con la cabecera de lenguaje.
- **MatMul-Free:** modo de recurrencia fusionado, factor de expansión del estado 1, razón de la capa FFN 4, sin convolución corta, y posición máxima de 8192.
- **HNetBit:** factor de expansión del estado recurrente 2, razón de la capa FFN 4, convolución corta causal de tamaño 4 con kernel compartido, posición máxima de 8192, y factor de submuestreo esperado del enrutamiento $N = 5,0$ (longitud promedio de palabra en español).

La Tabla 6.2 desglosa los parámetros totales y los no asociados a *embeddings*. El transformer paga un costo elevado por su vocabulario BPE: a 150M, 77.2 M de parámetros (40.5 %) corresponden a las tablas de *embeddings* y a la cabecera de lenguaje, que no contribuyen a las transformaciones del modelo; a 350M la sobrecarga es de 102.9 M (20.4 %). Los modelos byte-level, con vocabulario de 256 símbolos, tienen una sobrecarga de *embeddings* despreciable. En términos de parámetros no asociados a *embeddings*, el transformer y el MatMul-Free están prácticamente igualados a 150M (113.3 M frente a 113.4 M), mientras que el híbrido es un 22 % mayor; a 350M el MatMul-Free queda un 23 % por debajo del transformer y el híbrido un 4 % por encima.

Tabla 6.2. Conteo de parámetros (millones)

Modelo	Tamaño	Parámetros totales	Parámetros no- <i>embedding</i>
Transformer	150M	190.5	113.3
Transformer	350M	505.6	402.7
MatMul-Free	150M	113.8	113.4
MatMul-Free	350M	309.1	308.6
HNetBit	150M	138.3	138.0
HNetBit	350M	419.4	419.1

6.3. Metodología de entrenamiento

Todos los modelos se entrenaron con el mismo presupuesto de 25 000 millones de bytes de texto subyacente. Para igualar el consumo de datos entre esquemas de tokenización, el número de pasos se definió como:

$$\text{steps} = \frac{\text{total_bytes}}{\text{bytes_por_paso}}, \quad (6.1)$$

donde $\text{bytes_por_paso} = \text{lote efectivo} \times \text{bytes por muestra}$. Para los modelos byte-level, el lote efectivo de 32 con secuencias de 4096 bytes consume $32 \times 4096 = 131\,072$ bytes por paso. Para el transformer, el lote efectivo de 32 con secuencias de 1280 tokens consume $32 \times 1280 \times 3,16 \approx 129\,400$ bytes por paso, donde 3.16 es el promedio de bytes por token medido sobre el corpus. Los totales de pasos resultantes fueron 190 734 para los modelos byte-level y 193 247 para el transformer, de modo que todos consumen exactamente los mismos 25 000 millones de bytes al finalizar.

El tamaño de lote efectivo fue 32 en todos los casos (4 muestras por micro-lote con 8 pasos de acumulación de gradientes). Los hiperparámetros comunes se detallan en la Tabla 6.3.

Tabla 6.3. Hiperparámetros de entrenamiento comunes

Parámetro	Valor
Optimizador	AdamW ($\beta_1 = 0,9$, $\beta_2 = 0,95$, $\epsilon = 1 \times 10^{-8}$)
Decaimiento de peso	0.01 (sin decaimiento en sesgos, normas, <i>embeddings</i> ni parámetros de dimensión < 2)
Norma de gradiente máxima	1.0
Precisión mixta	bf16
Programador	WSD: 1 % warmup, 79 % estable, 20 % decaimiento co-seno
Semilla	42

Las tasas de aprendizaje se seleccionaron siguiendo la literatura de modelos ternarios (Ma et al., 2024; Zhu et al., 2024): la cuantización reduce la magnitud efectiva de las actualizaciones de pesos, por lo que los modelos ternarios requieren tasas un orden de magnitud mayores que el transformer:

- **transformer**: 3×10^{-4} .
- **matmulfree** y **hybrid**: 4×10^{-3} a 150M y $2,5 \times 10^{-3}$ a 350M, reduciendo la tasa con la escala para mantener la estabilidad.

El entrenamiento se ejecutó en una GPU NVIDIA A100 de 80 GB. Se habilitó el *gradient checkpointing* para el transformer y para **matmulfree**, cuyos pasos forward sí están instrumentados para recomputación de activaciones. En cuanto a **hybrid**, su paso forward jerárquico no está instrumentado para ello (ver Sección 5.6.1 del Capítulo 5). Como consecuencia,

los modelos híbridos consumen más memoria de entrenamiento (aproximadamente 15.5 GB para la versión de 150M) en comparación con el transformer (6.7 GB) y el MatMul-Free (3.4 GB) del mismo tamaño.

6.4. Métricas de evaluación

6.4.1. Calidad: bits por byte (BPB)

Para permitir la comparación entre arquitecturas con vocabularios distintos, se utilizó la métrica *bits per byte* (BPB). A partir de la pérdida de entropía cruzada media $\mathcal{L}$:

- Para modelos byte-level: $\text{BPB} = \mathcal{L} / \ln 2$.
- Para el transformer: $\text{BPB} = \mathcal{L} / (a \cdot \ln 2)$, donde $a = 3,16$ es el promedio de bytes por token medido sobre el corpus.

Un BPB menor indica mejor capacidad predictiva, y la métrica es independiente del vocabulario, lo que la hace preferible a la perplexidad en esta comparación.

La evaluación de validación se realizó cada 1000 pasos de optimizador sobre 50 lotes de validación (~ 0.8 MB de texto), y en los hitos de bytes del entrenamiento (6.25B, 12.5B, 18.75B y 25B). Al término del entrenamiento se ejecutó una evaluación final sobre 200 lotes (~ 3.3 MB) para obtener la medición reportada como resultado principal de cada modelo. El *checkpoint* de mejor validación se seleccionó con la evaluación periódica de 50 lotes.

6.4.2. Eficiencia: tamaño en despliegue

Para los modelos ternarios se exportó una versión compacta (`model_deploy.pt`) en la que los pesos se congelan, se precuantizan una sola vez y se empaquetan a 2 bits por parámetro mediante codificación ternaria $\{-1, 0, +1\}$, almacenando una escala por tensor (ver Sección 5.8 del Capítulo 5). El tamaño del archivo resultante y los bits por parámetro efectivos se reportan para cada modelo; la compresión se compara contra el equivalente en FP16. El transformer no se beneficia de este empaquetado y se exporta en FP32.

6.4.3. Eficiencia: rendimiento de inferencia

Mediante el script `profile_inference.py` se midieron dos aspectos de la inferencia, siempre bajo `torch.no_grad()` para evitar la retención de buffers del grafo de autograd:

- **Latencia de prefill** (*time-to-first-token*): tiempo de una pasada forward sobre contextos de 512, 1024, 2048 y 4096 bytes (o su equivalente en tokens para el transformer), con 3 ejecuciones de calentamiento y 5 temporizadas, medidas con eventos CUDA.
- **Rendimiento de decodificación** (*tokens por segundo*): generación autorregresiva de 256 tokens a partir de un *prompt* de 256 bytes, para tamaños de lote 1, 4 y 8. Se midió la memoria máxima asignada con `torch.cuda.max_memory_allocated()`.

Adicionalmente, durante el entrenamiento se registró, para cada ejecución, la memoria pico asignada y reservada en GPU (`training_stats.json`), y la memoria de inferencia con secuencias de 2048 bytes.

Capítulo 7

Resultados

En este capítulo se presentan los resultados experimentales de las tres arquitecturas evaluadas (Transformer, MatMul-Free y HNetBit) en los tamaños de 150M y 350M, tras consumir el presupuesto de 25 000 millones de bytes de texto en español. Se reportan métricas de calidad (BPB), eficiencia de despliegue, costo de entrenamiento y rendimiento de inferencia, así como el análisis del comportamiento del chunking dinámico. Salvo que se indique lo contrario, los valores de BPB corresponden a la evaluación final sobre 200 lotes de validación (~ 3.3 MB), descrita en el Capítulo 6.

7.1. Calidad de los modelos

La Tabla 7.1 muestra el BPB final de validación de cada arquitectura y tamaño.

Tabla 7.1. BPB de validación final (200 lotes; menor es mejor)

Modelo	150M	350M
Transformer (BPE)	1.484	1.386
MatMul-Free plano	1.474	1.370
HNetBit (híbrido)	1.622	1.437

Dos observaciones principales se desprenden de la tabla. Primero, el MatMul-Free plano obtiene el mejor BPB en ambas escalas, superando al Transformer por un margen pequeño pero consistente: 0.6 % a 150M y 1.2 % a 350M. Este resultado confirma que la eliminación de MatMul y la cuantización ternaria no impiden alcanzar calidad competitiva, e incluso ligeramente superior, en este régimen de entrenamiento.

Segundo, el modelo híbrido HNetBit queda detrás del Transformer, con una brecha del 9.3 % a 150M que se reduce a 3.7 % a 350M. La brecha se cierra con la escala, lo que es coherente con la evidencia de la literatura de que las jerarquías con chunking aprendido muestran su ventaja en escalas mayores (Hwang et al., 2025): a mayor profundidad y número de

parámetros, la compresión jerárquica compensa mejor la pérdida de señal que introduce el enrutamiento discreto.

La Figura 7.1 muestra las curvas de aprendizaje (BPB de validación periódica, 50 lotes, frente a bytes procesados). Dos patrones son relevantes:

- Los modelos de 350M descienden de forma monótona durante todo el entrenamiento, con el híbrido mostrando la mayor pendiente de mejora en la fase final de decaimiento del programador WSD: pasa de 1.546 a los 18.75B bytes a 1.432 a los 25B, una mejora de 7.4 % en el último 25 % del presupuesto.
- El híbrido de 150M muestra una curva no monótona (sube de 1.632 a los 6.25B bytes hasta 1.778 a los 18.75B, antes de caer a 1.617 al final), un comportamiento que sugiere que en la fase estable del programador el enrutamiento de una sola etapa explora configuraciones de segmentación que empeoran temporalmente la validación, para recuperarse con fuerza durante el decaimiento de la tasa de aprendizaje. El fenómeno no se repite a 350M, donde la jerarquía de dos etapas se estabiliza antes.

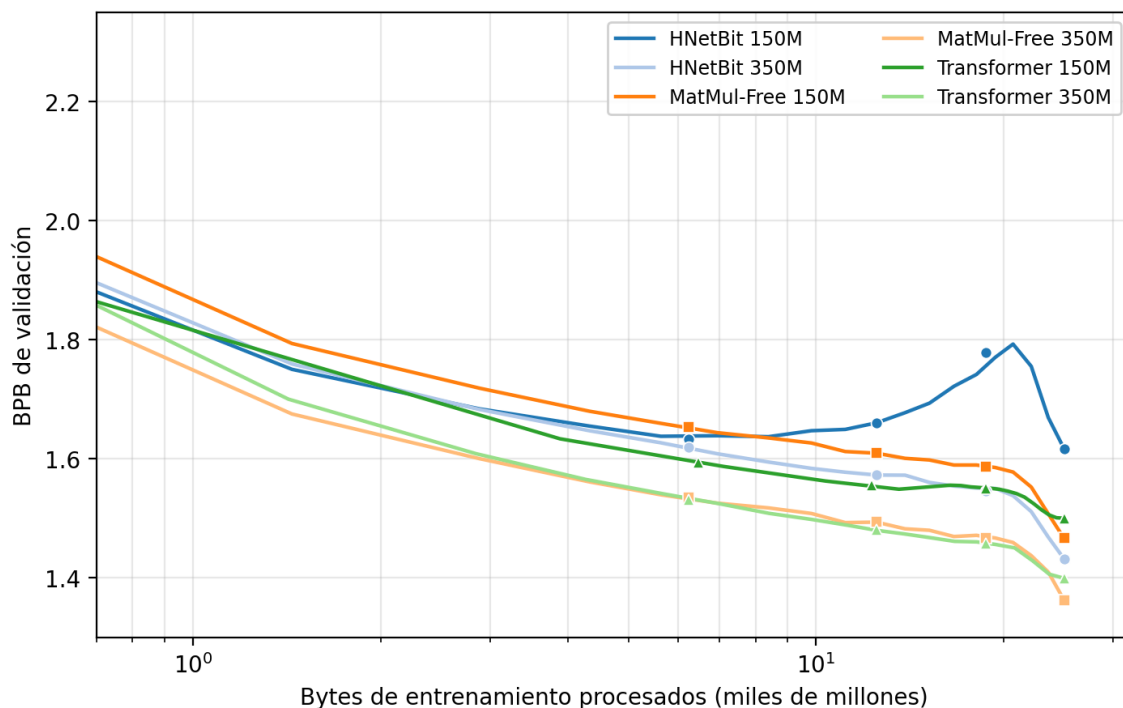


Figura 7.1. BPB de validación periódica (50 lotes) frente a bytes de entrenamiento procesados, en escala logarítmica. Las curvas muestran un subconjunto de puntos para legibilidad; los marcadores indican las evaluaciones en los hitos de 6.25B, 12.5B, 18.75B y 25B bytes.

Para evaluar la tendencia con la escala, la Figura 7.2 proyecta el BPB final de cada arquitectura hasta los 800M de parámetros mediante un ajuste de ley de potencias (recta en escala logarítmica) a través de los dos tamaños medidos. La proyección estima que a 800M el MatMul-Free alcanzaría un BPB de ~ 1.28 , HNetBit de ~ 1.28 y el Transformer de ~ 1.30 :

la brecha entre el híbrido y las líneas base prácticamente desaparecería en esa escala. Estas proyecciones se basan en solo dos puntos por arquitectura, por lo que son indicativas de tendencia y no predicciones calibradas.

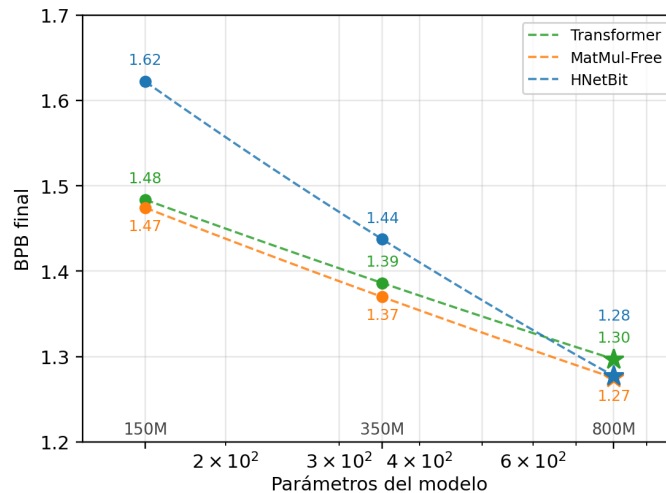


Figura 7.2. BPB final frente a parámetros del modelo, con extrapolación de ley de potencias (línea discontinua) desde los tamaños medidos (150M y 350M, marcadores redondos) hasta 800M (estrella). Los valores proyectados son 1.28 (MatMul-Free), 1.28 (HNetBit) y 1.30 (Transformer).

7.2. Eficiencia en parámetros y despliegue

El desglose de parámetros (Tabla 6.2 del Capítulo 6) evidencia el costo del vocabulario BPE del Transformer: a 150M, 77.2 M de sus 190.5 M de parámetros (40.5 %) son tablas de *embeddings* y cabecera de lenguaje que no participan en las transformaciones del modelo, y a 350M la sobrecarga es de 102.9 M (20.4 %). Los modelos byte-level tienen una sobrecarga de *embeddings* despreciable (vocabulario de 256 símbolos), de modo que todo su conteo de parámetros se dedica a cómputo efectivo.

La Tabla 7.2 presenta el tamaño de los pesos en disco y el resultado del empaquetado ternario para el despliegue de los modelos de 150M.

Tabla 7.2. Tamaño de despliegue y compresión (150M)

Modelo	Pesos bf16 (MB)	Despliegue (MB)	Bits/parámetro
Transformer	363.3	726.6	32.0
MatMul-Free	217.0	28.4	2.09
HNetBit	263.9	37.9	2.29

El Transformer se exporta en FP32 (32 bits por parámetro): su archivo de despliegue (726.6 MB) duplica el tamaño que ocuparían sus pesos en bf16 (363.3 MB). Los modelos ternarios,

en cambio, empaquetan cada peso en 2 bits: MatMul-Free reduce sus pesos de 217.0 MB a 28.4 MB (compresión de $7.6\times$ frente a FP16) y HNetBit de 263.9 MB a 37.9 MB ($7.0\times$). El ligero mayor costo del híbrido (2.29 frente a 2.09 bits/parámetro) se debe a que conserva en FP32 las escalas por tensor y los parámetros no ternarios (proyecciones residuales y módulos de enrutamiento).

7.3. Rendimiento de inferencia

La Tabla 7.3 resume las mediciones de latencia de prefill, rendimiento de decodificación y memoria pico, obtenidas sobre los modelos exportados para despliegue en una GPU A100 de 80 GB.

Tabla 7.3. Métricas de inferencia (A100 80 GB)

Modelo	Tamaño	Prefill 4096 (ms)	Decode BS1 (tok/s)	Decode BS8 (tok/s)	Mem. pico BS8 (MB)
Transformer	150M	96.3	125	922	1038
Transformer	350M	268.3	75	556	2723
MatMul-Free	150M	93.7	28	222	545
MatMul-Free	350M	210.5	17	133	1354
HNetBit	150M	109.4	49	372	653
HNetBit	350M	169.7	23	176	1769

En prefill, las tres arquitecturas de 150M son comparables (93.7–109.4 ms a 4096 bytes), mientras que a 350M el híbrido es el más rápido (169.7 ms) y el Transformer el más lento (268.3 ms), reflejando la diferencia entre el escaneo paralelo del HGRN por fragmentos y la atención cuadrática del Transformer.

En decodificación, el Transformer es claramente superior: 922 tok/s en lote 8 a 150M y 556 tok/s a 350M. El híbrido alcanza 372 y 176 tok/s, y el MatMul-Free plano 222 y 133 tok/s. El bajo rendimiento del MatMul-Free se explica por su recurrencia secuencial que ejecuta la red completa en cada token, mientras que el híbrido aprovecha la omisión condicional de la etapa interna en los tokens no-límite (Sección 4.8 del Capítulo 4): con un factor de límites de ~ 0.31 , la mayor parte de los tokens solo atraviesa el codificador y el decodificador superficiales de la etapa externa.

En memoria, la ventaja de las arquitecturas ternarias es contundente. En decodificación con lote 8, el híbrido usa 653 MB a 150M (37 % menos que el Transformer) y 1769 MB a 350M (35 % menos). El MatMul-Free es aún más frugal: 545 y 1354 MB. La causa es doble: los estados recurrentes de tamaño fijo eliminan el *KV cache* que crece con la secuencia, y los pesos ternarios empaquetados reducen el ancho de banda de memoria. La misma tendencia se observa en la memoria pico de prefill con contexto de 4096 bytes: 639 MB (MatMul-Free) y 738 MB (HNetBit) frente a 1822 MB (Transformer) a 150M; y 1479/1863 frente a 3507 MB a 350M.

7.4. Costo de entrenamiento

La Tabla 7.4 resume el costo computacional del entrenamiento de cada ejecución.

Tabla 7.4. Costo de entrenamiento (presupuesto de 25B bytes)

Modelo	Tamaño	Pasos	Tiempo (h)	Mem. pico (MB)	Throughput (tok/s)
Transformer	150M	193 247	40.3	6717	171 273
Transformer	350M	193 247	96.6	12 098	72 498
MatMul-Free	150M	190 734	120.3	3381	57 929
MatMul-Free	350M	190 734	228.6	7468	30 437
HNetBit	150M	190 734	80.6	15 453	86 417
HNetBit	350M	190 734	115.6	27 855	61 475

El Transformer es el más rápido en entrenar (40.3 h a 150M, 96.6 h a 350M), gracias a su mayor densidad de FLOPs por byte y a su *gradient checkpointing* funcional. El híbrido le sigue (80.6 y 115.6 h), y el MatMul-Free plano es el más lento (120.3 y 228.6 h): su recurrencia por fragmentos logarítmicos y su lote efectivo igual generan un costo por paso similar al del híbrido, pero sin la compresión de secuencia que este obtiene del chunking.

Para cuantificar el origen del escalado sublineal del costo del híbrido, la Tabla 7.5 estima los FLOPs por muestra a partir de la arquitectura y de las compresiones medidas. Cada bloque HGRN de dimensión d cuesta $8d^2 + 6dI$ FLOPs por token (cuatro proyecciones $d \times d$ del núcleo recurrente y las proyecciones del MLP, con dimensión intermedia $I = 256\lceil \frac{2}{3}dr/256 \rceil$ y $r = 4$), y la etapa s procesa $C_s L$ tokens, donde $C_s = \prod_{i<s} \hat{m}_i$ es la compresión acumulada de las etapas anteriores ($C_0 = 1$) y $\hat{m}_s$ es la fracción de límites medida de cada etapa de enrutamiento. El costo por muestra es entonces:

$$F = L \sum_{s=0}^{S-1} b_s \left(\prod_{i=0}^{s-1} \hat{m}_i \right) (8d_s^2 + 6d_s I_s), \quad (7.1)$$

donde S es el número de etapas, b_s los bloques de la etapa s y $L = 4096$ bytes. Se usan las configuraciones de la Sección 6.2 del Capítulo 6 y los promedios $\hat{m}_0 = 0,315$ (150M) y $(\hat{m}_0, \hat{m}_1) = (0,300, 0,219)$ (350M).

Tabla 7.5. FLOPs estimados por muestra (bloques HGRN; excluye enrutamiento, *embeddings* y pérdida)

Modelo	Etapas 0	Etapas 1	Etapas interna	Total
HNetBit 150M	260.9	—	182.4	443.3
HNetBit 350M	332.9	198.4	102.7	634.0
Denso equivalente 150M	260.9	—	579.8	840.7
Denso equivalente 350M	332.9	661.4	1565.5	2559.8

Instanciando la ecuación 7.1 con $\phi(d) = 8d^2 + 6dI$ y los valores de las Tablas 6.2 (Capítulo 6) y 7.5:

$$F_{150} = 4096 [8\phi(576) + 10 \cdot 0,315\phi(768)] = 443,3 \text{ GFLOPs}, \quad (7.2)$$

$$F_{350} = 4096 [8\phi(640) + 8 \cdot 0,300\phi(896) + 12 \cdot 0,066\phi(1152)] = 634,0 \text{ GFLOPs}. \quad (7.3)$$

El cociente entre los totales con chunking es $634.0/443.3 = 1.43$, que coincide con el cociente de tiempos de entrenamiento medidos ($115.6 \text{ h} / 80.6 \text{ h} = 1.43$). El equivalente denso (todas las etapas procesando los 4096 tokens) escalaría $2559.8/840.7 = 3.05$, proporcional al cociente de parámetros ($\times 3.03$): el chunking jerárquico elimina el 47.3 % de los FLOPs de bloques a 150M y el 75.2 % a 350M. La compresión medida de la secuencia (31.5 % a 150M; 30.0 % y 21.9 % por etapa a 350M) explica entonces por qué el costo de entrenamiento crece solo $\times 1.43$ mientras los parámetros se triplican.

En memoria pico de entrenamiento los papeles se invierten: el MatMul-Free es el más frugal (3.4 y 7.5 GB), seguido del Transformer (6.7 y 12.1 GB) y del híbrido, que alcanza 15.5 GB a 150M y 27.9 GB a 350M. Esta es la consecuencia directa de la ausencia de *gradient checkpointing* funcional en HNetBit (Sección 5.6.1 del Capítulo 5): todas las activaciones intermedias de la jerarquía (codificadores, decodificadores, módulos de chunking y etapas recursivas) se retienen hasta el paso backward.

7.5. Análisis del chunking dinámico

La Figura 7.3 muestra la evolución, durante el entrenamiento, de la fracción de límites predicha por cada etapa de enrutamiento de HNetBit (razón de compresión de la etapa).

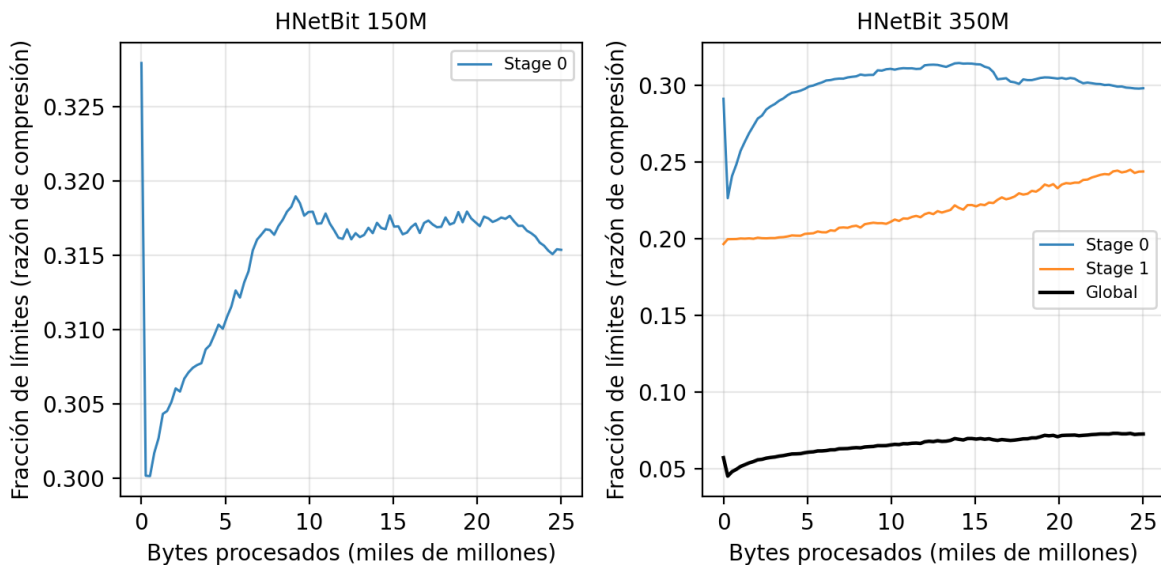


Figura 7.3. Fracción de límites (razón de compresión) por etapa de chunking durante el entrenamiento de HNetBit. Izquierda: 150M (una etapa). Derecha: 350M (dos etapas más el producto global). Valores cercanos a 0.3 significan que aproximadamente 3 de cada 10 posiciones se marcan como límite.

Dos hallazgos destacan. Primero, la etapa externa converge rápidamente a una fracción de límites estable de ~ 0.31 en ambas escalas, lo que equivale a fragmentos promedio de ~ 3.2 bytes: el modelo segmenta a nivel de caracteres o sílabas, como era de esperarse para una etapa que opera sobre bytes. Este valor aparece desde el inicio del entrenamiento y se mantiene esencialmente constante, lo que indica que la pérdida de balanceo de carga (Sección 4.4.5 del Capítulo 4) cumple su función de impedir soluciones degeneradas.

Segundo, la segunda etapa de la jerarquía de 350M establece una fracción de límites de ~ 0.22 – 0.24 sobre los fragmentos de la etapa anterior (fragmentos promedio de ~ 4.6 unidades de etapa 0), produciendo una compresión global promedio de la secuencia de $\sim 15\times$: el producto de las fracciones de ambas etapas promedió 0.066 durante el entrenamiento (0.300×0.219), de modo que la etapa más interna procesa en promedio el $\sim 6.6\%$ de los tokens de entrada. El producto global crece levemente durante el entrenamiento (de 0.064 a 0.072), lo que sugiere un refinamiento gradual hacia segmentos ligeramente más finos en la fase estable del programador; hacia el final, la etapa interna recibe $\sim 7.2\%$ de los tokens.

En conjunto, el enrutamiento aprendido produce una segmentación jerárquica plausible: la primera etapa agrupa bytes en unidades cercanas a caracteres o sílabas, y la segunda las agrupa en unidades de ~ 15 bytes en promedio, cercanas a la longitud típica de una palabra en español. La segmentación emerge de forma completamente end-to-end, sin ningún tokenizador predefinido.

7.6. Síntesis

La Tabla 7.6 consolida las métricas principales de las seis ejecuciones.

Tabla 7.6. Síntesis de resultados

Modelo	Tamaño	BPB	Despliegue (MB)	Mem. ent. (GB)	Mem. inf. (MB) ^b	Decode BS8 (tok/s)	Tiempo (h)
Transformer	150M	1.484	726.6	6.7	1822	922	40.3
Transformer	350M	1.386	~1010 ^a	12.1	3507	556	96.6
MatMul-Free	150M	1.474	28.4	3.4	639	222	120.3
MatMul-Free	350M	1.370	~81 ^a	7.5	1479	133	228.6
HNetBit	150M	1.622	37.9	15.5	738	372	80.6
HNetBit	350M	1.437	~120 ^a	27.9	1863	176	115.6

^a Estimado: el empaquetado ternario es de ~2.1–2.3 bits/parámetro independiente de la escala; los exports de 350M no fueron retenidos en disco. El Transformer se almacena en FP32 (32 bits/parámetro). ^b

Memoria pico de prefill con contexto de 4096 bytes, medida en proceso limpio sobre el export de despliegue (`profile_inference.py`). La medición intra-proceso de entrenamiento con contexto de 2048 bytes (1726/5010 MB para HNetBit) incluye los pesos FP32 y los momentos del optimizador aún vivos en el proceso de entrenamiento, por lo que no refleja la memoria de despliegue.

El panorama es el siguiente: el Transformer ofrece una calidad comparable (segundo lugar en BPB en ambas escalas) y la decodificación más rápida, pero a costa de 20–40 % de parámetros de vocabulario, un archivo de despliegue 19–26× mayor y 1.9–2.9× más memoria de inferencia. El MatMul-Free plano ofrece la mejor calidad y la menor huella de memoria, pero con la inferencia y el entrenamiento más lentos. El híbrido HNetBit se ubica entre ambos: calidad intermedia con brecha decreciente al escalar, despliegue 7× menor que FP16, memoria de inferencia casi tan baja como la del MatMul-Free y velocidad de decodificación 1.3–1.7× superior a la de este. La interpretación de estas compensaciones se desarrolla en el Capítulo 8.

Capítulo 8

Discusión

Este capítulo interpreta los resultados del Capítulo 7 a la luz de los objetivos del trabajo y de la literatura, discute los hallazgos inesperados, y examina las limitaciones metodológicas que condicionan la validez de las conclusiones.

8.1. Viabilidad de eliminar MatMul y tokenización

Los resultados responden de forma matizada, pero optimista, a la pregunta de investigación. Por un lado, confirman que la eliminación de MatMul mediante pesos ternarios no sacrifica calidad: el MatMul-Free plano superó al Transformer en BPB en ambas escalas (0.6 % a 150M y 1.2 % a 350M), con 23 % menos parámetros no asociados a *embeddings* a 350M. Este resultado es coherente con la literatura: Zhu et al. (2024) reportan paridad con Transformer++ a partir de 370M parámetros, y Ma et al. (2024) muestran que BitNet b1.58 iguala a LLaMA a partir de 3B. Nuestro trabajo extiende esta evidencia a un corpus en español y a un presupuesto de datos mucho menor (25B bytes frente a cientos de miles de millones de tokens).

Por otro lado, la combinación de chunking dinámico con pesos ternarios (HNetBit) quedó 9.3 % por debajo del Transformer a 150M y 3.7 % a 350M. Este resultado evidencia que la brecha se reduce con la escala y que la curva de aprendizaje del híbrido de 350M es la de mayor pendiente en la fase final del entrenamiento: si la tendencia se mantiene, el cruce ocurriría a escalas ligeramente mayores, consistente con Hwang et al. (2025), donde H-Net (con encoders Mamba en precisión completa y un presupuesto de decenas de miles de millones de bytes) cruza por debajo del Transformer tokenizado alrededor de los 30B bytes en modelos de 760M–1.3B parámetros.

La jerarquía con chunking aprendido introduce una pérdida de calidad inicial esperable que se amortiza con la escala, pero que en el régimen de 150M–350M con 25B bytes esa amortización aún no se completa. La eliminación de MatMul, en cambio, es gratuita en términos de calidad en este régimen: no se observó penalización alguna atribuible a los pesos ternarios.

8.2. Hallazgos inesperados

8.2.1. La curva no monótona del híbrido de 150M

El empeoramiento de la validación del híbrido de 150M entre los 6.25B y los 18.75B bytes (de 1.632 a 1.778) seguido de una recuperación abrupta hasta 1.617 en el decaimiento final no se observó en ningún otro modelo. La posible interpretación es que el módulo de enrutamiento, con una sola etapa de chunking y una tasa de aprendizaje alta (4×10^{-3}), explora continuamente configuraciones de segmentación durante la fase estable del programador WSD; esas exploraciones degradan temporalmente la validación, y solo el decaimiento de la tasa permite asentarse en una segmentación buena. La desaparición del fenómeno a 350M (dos etapas) sugiere que la jerarquía más profunda estabiliza el enrutamiento. Este resultado tiene una implicación práctica: los modelos con chunking dinámico en regímenes de alta tasa de aprendizaje pueden beneficiarse de fases de decaimiento más largas, y el BPB en los hitos intermedios no predice fielmente el resultado final.

8.2.2. La compresión aprendida es estable y lingüísticamente plausible

Un hallazgo positivo fue la convergencia rápida y estable de la fracción de límites de la etapa externa (~ 0.31 , es decir, fragmentos de ~ 3.2 bytes) desde el inicio del entrenamiento, y la emergencia de una segunda etapa que agrupa en unidades de ~ 15 bytes, cercanas a la longitud media de palabra en español. La segmentación aprendida end-to-end reproduce niveles de abstracción análogos a los de un tokenizador BPE, sin ningún preprocesamiento: este es el resultado más directo a favor del paradigma sin tokenización, y coincide cualitativamente con lo reportado por Hwang et al. (2025) y Slagle (2024) sobre la alineación de los límites aprendidos con fronteras lingüísticas.

8.2.3. El cuello de botella de los modelos ternarios es la velocidad, no la memoria

En decodificación el Transformer alcanzó 922 tok/s en lote 8 frente a 372 del híbrido y 222 del MatMul-Free a 150M. La explicación es que en la A100 los pesos ternarios se desempaquetan a FP32 para el cómputo (Sección 5.8 del Capítulo 5), de modo que la reducción de bits se traduce en menor memoria y menor ancho de banda, no en menos FLOPs efectivos; y la recurrencia secuencial del MatMul-Free no paraleliza bien en GPU. La ganancia real de estas arquitecturas en el hardware disponible es la huella de memoria ($1.9\text{--}2.9\times$ menor) y el tamaño de despliegue ($7\text{--}7.6\times$ menor), no la latencia. El aprovechamiento pleno de la aritmética ternaria requeriría kernels especializados para la representación empaquetada o hardware dedicado, como lo plantea Ma et al. (2024).

8.3. Validez interna

8.3.1. Igualdad del presupuesto de bytes

El diseño metodológico garantizó que las seis ejecuciones consumieran exactamente los mismos 25 000 millones de bytes subyacentes, mediante la fórmula de bytes por paso con el promedio medido de 3.16 bytes por token del corpus. Una consecuencia de usar el promedio medido (en lugar de la estimación inicial de 4.5) es que los contextos por muestra quedaron prácticamente simétricos: ~4043 bytes efectivos para el Transformer frente a 4096 bytes para los modelos byte-level. Esto elimina el riesgo de que el Transformer disfrutara de una ventaja de contexto más largo, y fortalece la comparación: las diferencias de BPB no pueden atribuirse a asimetría de contexto.

8.3.2. Desigualdad de parámetros

La comparación no está emparejada en parámetros: a 150M el híbrido tiene 22 % más parámetros no asociados a *embeddings* que el Transformer, y a 350M el MatMul-Free tiene 23 % menos. Las configuraciones se eligieron para cubrir el presupuesto de parámetros objetivo de cada categoría, no para igualar los conteos. Esto no invalida los hallazgos cualitativos, pero debe tenerse en cuenta al leer las magnitudes de las diferencias.

8.3.3. Asimetría en gradient checkpointing

El Transformer y el MatMul-Free entrenaron con recomputación de activaciones; el híbrido no, porque su paso forward no está instrumentado para ello. Esta diferencia no afecta la calidad de los modelos (el checkpointing no cambia la función aprendida), pero sí el consumo de memoria de entrenamiento: el híbrido necesitó 15.5 GB a 150M y 27.9 GB a 350M, frente a 6.7/3.4 y 12.1/7.5 GB de las líneas base. Documentarlo es importante para la comparabilidad de las métricas de memoria de entrenamiento, que no deben leerse como costo intrínseco de la arquitectura sino como costo de la implementación actual.

8.3.4. Comparabilidad de la métrica BPB

El BPB del Transformer se convirtió desde nats por token usando el promedio de bytes por token del corpus (3.16). Este valor se midió sobre el corpus completo; si la distribución de longitudes de token difiriera entre entrenamiento y validación, la conversión introduciría un sesgo pequeño. Dado que la división es contigua y el corpus es homogéneo, el riesgo bajo, pero es una fuente de incertidumbre adicional para el Transformer que no existe en los modelos byte-level.

8.4. Validez externa y limitaciones

8.4.1. Una sola ejecución por configuración

Todas las ejecuciones usan la semilla 42, sin réplicas. Las diferencias de BPB entre modelos a 150M (0.6 % entre MatMul-Free y Transformer) podrían ser comparables a la varianza entre ejecuciones con semillas distintas, por lo que la afirmación de superioridad del MatMul-Free a 150M debe tomarse con cautela; a 350M las diferencias (1.2 % y 3.7 %) son más difíciles de explicar por varianza, pero no contamos con intervalos de confianza que lo demuestren.

8.4.2. Comparación no emparejada en FLOPs

La igualación se hizo por bytes de entrenamiento, no por FLOPs, siguiendo el protocolo de Hwang et al. (2025). Dado que las arquitecturas tienen costos computacionales por byte muy distintos (el Transformer entrenó en 40.3 h frente a 120.3 h del MatMul-Free a 150M), una comparación emparejada en FLOPs podría mover los resultados en direcciones no anticipadas. En particular, el MatMul-Free dispuso de más tiempo de cómputo por byte, lo que favorece su BPB.

8.5. Implicaciones

Los resultados respaldan una conclusión práctica clara para el despliegue: si el objetivo es un modelo de lenguaje compacto para un entorno con recursos limitados (memoria y almacenamiento), las arquitecturas ternarias ofrecen reducciones de 7–7.6× en tamaño de despliegue y de 1.9–2.9× en memoria de inferencia con una penalización de calidad nula (MatMul-Free) o moderada y decreciente (HNetBit). Si el objetivo es máximo rendimiento por token en una GPU moderna, el Transformer sigue siendo la opción dominante.

Capítulo 9

Conclusiones

Este capítulo cierra el documento respondiendo la pregunta de investigación, evaluando la hipótesis planteada y sintetizando las contribuciones del trabajo.

9.1. Respuesta a la pregunta de investigación

La pregunta que guió el trabajo fue cómo diseñar e implementar una arquitectura de modelo de lenguaje que, integrando mecanismos MatMul-Free y chunking dinámico, logre un rendimiento competitivo con un costo computacional reducido y capacidad de procesar secuencias sin tokenización. La respuesta, sustentada en seis ejecuciones de entrenamiento a 25 000 millones de bytes en español, es que ambas ideas son viables, pero con matices distintos:

- **La eliminación de MatMul es gratuita en calidad.** El MatMul-Free plano, con pesos ternarios $\{-1, 0, +1\}$ y recurrencia HGRN, obtuvo el mejor BPB de todas las arquitecturas (1.474 a 150M y 1.370 a 350M), superando al Transformer de referencia (1.484 y 1.386). Lo logró con un despliegue de 28.4 MB ($7.6\times$ menor que el FP16 equivalente), un pico de memoria de entrenamiento de 3.4 GB y una memoria de inferencia $2.9\times$ menor.
- **El chunking dinámico aprende una segmentación jerárquica plausible** HNet-Bit aprendió, de forma end-to-end y sin tokenizador, a agrupar bytes en fragmentos de ~ 3.2 bytes y estos en unidades de ~ 15 bytes, cercanas a palabras en español, con una compresión de secuencia de $\sim 15\times$ a 350M (la etapa más interna procesa el $\sim 6.6\%$ de los tokens en promedio). Sin embargo, su BPB quedó 9.3% (150M) y 3.7% (350M) por encima del Transformer. La brecha decreciente y la mayor pendiente de mejora del híbrido en la fase final del entrenamiento sugieren que la ventaja del enfoque jerárquico aparecería a escalas mayores, en línea con la literatura de H-Net.

9.2. Evaluación de la hipótesis

La hipótesis central sostenía que la combinación de una arquitectura libre de MatMul con cuantización a 1.58 bits y un mecanismo de segmentación dinámica permitiría construir modelos más eficientes sin sacrificar calidad. La evidencia la confirma parcialmente:

- **Confirmada en eficiencia:** los modelos ternarios reducen el tamaño de despliegue en $7\text{--}7.6\times$ frente a FP16, la memoria de inferencia en $1.9\text{--}2.9\times$, y el pico de memoria de entrenamiento hasta 3.4 GB. La ventaja en huella de memoria y almacenamiento es robusta, medible y reproducible.
- **Confirmada parcialmente en calidad:** la arquitectura MatMul-Free plana iguala o supera al Transformer sin costo alguno de calidad; la combinación con chunking jerárquico, en cambio, impone una penalización moderada que desaparecería a escalas más grandes.
- **Confirmada en aprendizaje end-to-end:** la segmentación jerárquica emergió espontáneamente del entrenamiento sobre bytes crudos, validando el principio de que la tokenización no es indispensable para aprender representaciones multinivel.

9.3. Contribuciones

Las contribuciones concretas del trabajo son las siguientes:

1. **Una arquitectura nueva documentada en detalle:** HNetBit, que integra pesos ternarios, recurrencia HGRN y chunking dinámico multi-etapa, con su formulación matemática completa, sus kernels de cómputo, su caché de inferencia jerárquica y su variante con atención deslizante (Capítulos 4 y 5).
2. **Un marco experimental modular y reproducible:** un sistema de entrenamiento unificado para las tres arquitecturas (además de la variante `hybrid_attn` implementada pero no evaluada), con igualación estricta del presupuesto de datos entre tokenización BPE y byte-level, checkpointing, registro de métricas y exportación de despliegue compacto.
3. **Evidencia experimental en español:** los primeros resultados conocidos de arquitecturas MatMul-Free y de chunking dinámico entrenadas sobre un corpus de miles de millones de bytes en español, con métricas de calidad (BPB), eficiencia de despliegue, memoria de entrenamiento e inferencia, y costo de entrenamiento.
4. **Un análisis honesto de limitaciones:** ausencia de *gradient checkpointing* en el híbrido, conteos de parámetros no emparejados, comparación por bytes y no por FLOPs, una sola semilla y un solo idioma, todas documentadas explícitamente para su consideración en trabajos futuros.

9.4. Síntesis final

El trabajo demuestra que es posible construir modelos de lenguaje eficientes en español que prescinden de la multiplicación de matrices sin penalización de calidad, y que el aprendizaje de segmentación jerárquica sin tokenizador es factible y produce estructuras lingüísticamente plausibles, aunque su rentabilidad en calidad requiere escalas mayores que las exploradas aquí. La combinación de ambas ideas ofrece hoy una alternativa atractiva para el despliegue de modelos compactos: un modelo de 150M parámetros que cabe en 28–38 MB, infiere con menos de 700 MB de memoria y predice el español con un BPB comparable al de un Transformer de referencia. Esa es la evidencia central de que la eficiencia extrema no exige abandonar la calidad, y de que los modelos que aprenden directamente de bytes son una dirección de investigación con fundamento experimental sólido.

Capítulo 10

Trabajo Futuro

Los resultados y las limitaciones identificadas en la Discusión abren las siguientes líneas de trabajo:

10.1. Evaluación empírica más amplia

- **Múltiples semillas y estimación de varianza.** Repetir cada configuración con varias semillas permitiría calcular intervalos de confianza sobre el BPB y determinar si las diferencias de 0.6 % observadas a 150M son significativas. Es el primer paso para consolidar las afirmaciones de superioridad del MatMul-Free.
- **Escalas mayores.** La hipótesis central pendiente es que la brecha del híbrido continúa cerrándose con la escala. Entrenar HNetBit y las líneas base a 750M y, si los recursos lo permiten, con presupuestos de datos mayores (50B–100B bytes), permitiría verificar o descartar la tendencia decreciente observada (9.3 % $\rightarrow$ 3.7 %). Un análisis preliminar de escalado del costo de entrenamiento sitúa el cruce de costos en torno a los $\sim$ 770M de parámetros.
- **Presupuesto de datos.** El log de validación del híbrido de 150M muestra que el $\sim$ 11 % del tiempo total de entrenamiento captura $\sim$ 96 % de la calidad final (1/BPB), una propiedad del protocolo de bytes iguales (todos los modelos se sobre-entrenan de 4 a 9 veces sobre Chinchilla), no de la arquitectura. Fijar puntos de parada óptimos por arquitectura reduciría el costo sin pérdida apreciable de calidad.
- **Enmarcado de las comparaciones futuras.** Las extensiones deben preservar el enmarcado del benchmark —comportamiento restringido por el protocolo (bytes, optimizador, WSD, semilla, lote, tasas por defecto, sin barridos), no óptimo por arquitectura (Capítulos 6 y 8)— o declarar explícitamente en qué eje se desvía.
- **Otros idiomas y dominios.** Evaluar el comportamiento en dominios no lingüísticos (código fuente, secuencias biológicas, notación matemática) donde la motivación del trabajo predice la mayor ventaja de los modelos byte-level; y en idiomas con morfología distinta al español, donde el sesgo del BPE se manifiesta de otra forma.

- **Tareas posteriores.** El BPB mide la calidad del modelado de lenguaje, no su utilidad. Evaluar los modelos en tareas de clasificación, generación o comprensión sobre benchmarks en español complementaría la imagen con métricas orientadas a aplicación.

10.2. Mejoras arquitectónicas

- **Evaluar la variante `hybrid_attn`.** La variante con atención deslizante en la etapa más interna quedó implementada pero sin evaluar. Su ejecución de 150M es la forma más barata de medir el beneficio de reintroducir atención local en la jerarquía, dado que la motivación teórica (Sección 4.7 del Capítulo 4) lo predice.
- **Modulación de la tasa de aprendizaje por etapa.** H-Net asigna tasas mayores a las etapas externas que procesan secuencias más largas (Hwang et al., 2025). Nuestro entrenamiento usó una tasa única, y la curva no monótona del híbrido de 150M sugiere que el enrutamiento es sensible a este hiperparámetro.
- **Fase de decaimiento más larga o programadores alternativos.** Dado que el híbrido de 350M mejoró 7.4 % solo en el último 25 % del entrenamiento, un programador con mayor fracción de decaimiento podría extraer mejoras adicionales sin datos nuevos.
- **Enrutamiento suave o con temperatura.** El enrutamiento usa una decisión discreta con STE. Explorar versiones suaves (selección ponderada por probabilidad) podría suavizar el paisaje de optimización que presumiblemente causa la no-monotonía observada.

10.3. Mejoras de implementación y eficiencia

El diagnóstico del cuello de botella del entrenamiento del híbrido indica un régimen limitado por eficiencia: el tiempo por paso escala en proporción a los FLOPs ($1.53\times$ de tiempo frente a $1.55\times$ de FLOPs al pasar de 150M a 350M), con overhead fijo despreciable y una utilización estable de $\sim 10\%$ del pico bf16 de la A100. Las prioridades atacan por tanto esa constante de eficiencia, y el kernel ternario queda supeditado a la validación previa del resto.

- **Gradient checkpointing para el híbrido.** Honrar la bandera de recomputación en el forward jerárquico (solo durante el entrenamiento) eliminaría la asimetría frente a las líneas base y reduciría la memoria de entrenamiento (15.5 GB a 150M, 27.9 GB a 350M), habilitando escalas mayores en una sola GPU.
- **Kernels sobre pesos empaquetados.** Una ruta ternaria opcional que opere directamente sobre los pesos de 2 bits, con gradiente denso en el backward, evitaría el desempaquetado a FP32 en cada paso y convertiría la ventaja de memoria en ventaja de latencia; su techo está acotado mientras los GEMM operen al $\sim 10\%$ de eficiencia, por lo que se pospone hasta validar las mejoras anteriores.
- **Recurrencia chunk-paralela en el forward jerárquico.** Conectar la variante chunk-paralela de la recurrencia al entrenamiento y prefill, con caída automática al

modo recurrente en la generación paso a paso, aceleraría el entrenamiento y corregiría un caso límite numérico del dechunking.

- **Fusión de proyecciones.** Fusionar las tres proyecciones de la convolución corta en un solo peso compartido, con reconstrucción automática al cargar checkpoints antiguos.
- **Eliminación de sincronizaciones CPU–GPU.** Mantener los escalares por micro-paso (ratios de límites, pérdida acumulada) como tensores en GPU y extraerlos solo al momento del logging (+5–15 %).
- **Cómputo único del ordenamiento de límites.** Calcular una sola vez el ordenamiento de la máscara de límites y compartirlo entre el chunking y el dechunking (+1–3 %).
- **Configuración del asignador CUDA.** Usar segmentos expandibles para reducir la fragmentación de la memoria reservada (18.1 GB reservados frente a 15.4 asignados a 150M).
- **Recurrencia paralela del MatMul-Free.** El bajo rendimiento de decodificación del MatMul-Free (28 tok/s a 150M) es el principal costo de su excelente BPB. Aplicar las técnicas de omisión condicional del híbrido o kernels de escaneo más agresivos podría recuperar parte de esa brecha.
- **Comparación emparejada en FLOPs.** Replicar el protocolo de comparación con presupuestos de FLOPs iguales (además de bytes iguales) aclararía cuánto de la ventaja del MatMul-Free proviene de su mayor cómputo por byte, y ubicaría las arquitecturas en una frontera de Pareto honesta.
- **Despliegue en hardware edge.** El tamaño de despliegue de 28–38 MB para 150M hace viable probar los modelos exportados en dispositivos de bajo consumo, validando la motivación de accesibilidad del trabajo.

El objetivo combinado es reducir el entrenamiento del híbrido de 150M de 80.6 h a ~27–45 h (2–3×) y su memoria pico a 11–13 GB a lote 8 (6–8 GB a lote 4), y a 350M de ~123 h a ~58–85 h.

10.4. Optimizaciones de velocidad de inferencia

El cuello de botella de los modelos ternarios en GPU es la velocidad, no la memoria (Capítulo 8). Las optimizaciones propuestas, en orden de esfuerzo:

- **Inferencia en BF16.** Reduce a la mitad la memoria de pesos y mejora el rendimiento entre 1.5 y 2 veces.
- **`torch.compile`.** Fusiona operaciones y reduce el overhead de Python en el forward, con una mejora de 1.3–2× en GPU.
- **Fijación de hilos de CPU.** Variables de entorno que acotan los hilos de cómputo, con 1.5–2× en CPU, relevante para el despliegue edge.

- **Caché de estado HGRN optimizada.** Evita recomputación en bloques cuyo estado apenas cambió, con $2\text{--}3\times$ en contextos largos.
- **Salida temprana sin límites.** Omite la etapa interna cuando no se detectan límites en un paso ($\sim 1.2\times$).
- **Batching de la etapa interna.** Acumula varios pasos de generación antes de despachar la etapa interna, convirtiendo su cuello de botella secuencial en procesamiento por lotes ($2\text{--}4\times$ en decodificación).
- **Decodificación especulativa.** Un modelo auxiliar pequeño propone candidatos baratos, verificados en lote por la jerarquía completa ($2\text{--}4\times$).
- **Kernel fusionado del camino por token.** Fusiona codificador, enrutamiento, chunking, etapa interna, dechunking y decodificador en un solo kernel que mantiene los intermedios en SRAM; es la mejora individual más grande disponible ($3\text{--}5\times$).
- **Pipeline de etapas en varias GPU.** Asigna cada etapa de la jerarquía a una GPU distinta, con comunicación de límites y estados ($2\text{--}4\times$ con 4 GPU).
- **Decodificación asíncrona.** Solapa la etapa i del token t con la etapa $i - 1$ del token $t + 1$ ($\sim 2\times$).
- **Pre-cuantización de pesos en inferencia.** Cuantiza los pesos una sola vez y omite la cuantización de activaciones, eliminando dos de cada tres operaciones por capa lineal ($2\text{--}3\times$).

Referencias

- Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The Long-Document Transformer. <https://arxiv.org/abs/2004.05150>
- Bengio, Y., Léonard, N., & Courville, A. (2013). Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. <https://arxiv.org/abs/1308.3432>
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Google. (2024). *SentencePiece: Unsupervised Text Tokenizer* [Repositorio oficial en GitHub]. <https://github.com/google/sentencepiece>
- Google DeepMind. (2024). *Gemma: Open Models Based on Gemini Research and Technology* [Blog de Google]. <https://blog.google/technology/developers/gemma-open-models/>
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., ... Sifre, L. (2022). Training Compute-Optimal Large Language Models. <https://arxiv.org/abs/2203.15556>
- HuggingFace. (2024a). *Byte-Pair Encoding Tokenization* [Recuperado de Hugging Face LLM Course]. <https://huggingface.co/learn/llm-course/es/chapter6/5>
- HuggingFace. (2024b). *WordPiece Tokenization* [Recuperado de Hugging Face LLM Course]. <https://huggingface.co/learn/llm-course/en/chapter6/6>
- Hwang, S., Wang, B., & Gu, A. (2025). Dynamic Chunking for End-to-End Hierarchical Sequence Modeling. <https://arxiv.org/abs/2507.07955>
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling Laws for Neural Language Models. <https://arxiv.org/abs/2001.08361>
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., & Chen, Z. (2020). GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. *International Conference on Learning Representations (ICLR)*.
- Ma, S., Wang, H., Ma, L., Wang, L., Wang, W., Huang, S., Dong, L., Wang, R., Xue, J., & Wei, F. (2024). The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. <https://arxiv.org/abs/2402.17764>

- Microsoft Research. (2023). *Phi-2: The Surprising Power of Small Language Models* [Blog de Microsoft Research]. <https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/>
- Peng, B., Alcaide, E., Anthony, Q., et al. (2023). RWKV: Reinventing RNNs for the Transformer Era. <https://arxiv.org/abs/2305.13048>
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. <https://arxiv.org/abs/1910.01108>
- Shazeer, N. (2020). GLU Variants Improve Transformer. <https://arxiv.org/abs/2002.05202>
- Slagle, K. (2024). SpaceByte: Towards Deleting Tokenization from Large Language Modeling. <https://arxiv.org/abs/2404.14408>
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568, 127063.
- Tillet, P., Kung, H. T., & Cox, D. (2019). Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., & Lewis, M. (2023). MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers. <https://arxiv.org/abs/2305.07185>
- Zhang, B., & Sennrich, R. (2019). Root Mean Square Layer Normalization. *Advances in Neural Information Processing Systems*, 32.
- Zhang, P., Zeng, G., Wang, T., & Lu, W. (2024). TinyLlama: An Open-Source Small Language Model. <https://arxiv.org/abs/2401.02385>
- Zhu, R.-J., Zhang, Y., Abreu, S., Sifferman, E., Sheaves, T., Wang, Y., Richmond, D., Shrestha, S. B., Zhou, P., & Eshraghian, J. K. (2024). Scalable MatMul-free Language Modeling. <https://arxiv.org/abs/2406.02528>